z0 无人机自主飞行项目报告

本文件由 `tools/report/assemble.py` 从 `docs/report/chapters/` 装配生成；不要手改，改章节源文件后重跑。

第 0 章 摘要

z0 是一台涵道四旋翼，机架为 OddityRC XI35 Pro；飞控是 MicoAir NxtPX4v2，运行 PX4 的 MicoAir 厂商构建；机载电脑是 Jetson Orin NX，室内定位由 Nokov 动捕经 VRPN 提供。项目的目标不是再造一台能飞的机器，而是把 z0 建成一个可复用的自主飞行平台，并让 Model Predictive Control（MPC）接管飞控内置 PID。

四阶段路线遵循「每个阶段只引入一个新变量」。Phase 1 在真机上打通「动捕 → 机载电脑 → offboard → 飞控 → 飞机」的位置链路，控制仍是 PX4 内置 PID；Phase 2 把 (C) 级 MPC 接进控制回路，但只在 SITL 中闭环；Phase 3 整理入口层、`docs/` 与未结票，不引入新变量；Phase 4 让 MPC 在真机上完成悬停、正方形与 8 字三条轨迹的自由飞，阶梯 2 未达标后由操作者裁定跳过。本报告（Phase 5）把散落在话题包、SITL 证据与文档里的结论收成一条可重跑的数据管线，并在同一时间零点与同一轨迹原点下比较目标、offboard、MPC 仿真与 MPC 实飞四个源。

第 1 章 目录

本目录由 `tools/report/assemble.py` 从正文标题生成；链接指向标题前的显式锚点。

- 第 0 章 摘要
- 第 2 章 项目背景与目标
  - 2.1 平台与目标
  - 2.2 四阶段路线
    - 2.2.1 Phase 1 —— offboard 位置定点
    - 2.2.2 Phase 2 —— SITL 验证 MPC
    - 2.2.3 Phase 3 —— 整理与收尾
    - 2.2.4 Phase 4 —— MPC 上实机
  - 2.3 三条关键工程约束
    - 2.3.1 室内无 GPS，动捕是唯一位置源
    - 2.3.2 mavros 是唯一的机载链路
    - 2.3.3 控制层级选 (C)：总推力 + 机体三轴角速率
  - 2.4 与后续章节的分工
- 第 3 章 无人机零件与物理参数
  - 3.1 证据等级与符号约定
    - 3.1.1 三类标记
    - 3.1.2 符号表
  - 3.2 零件清单
  - 3.3 几何与质量
  - 3.4 动力与推力
    - 3.4.1 四个容易混淆的推力量
    - 3.4.2 推重比与电池工作点
    - 3.4.3 最大起飞重量与推重比
  - 3.5 飞控限制
  - 3.6 延迟与外参
    - 3.6.1 端到端延迟
    - 3.6.2 刚体原点与 IMU 的 z 向偏移
  - 3.7 电池
  - 3.8 未测量项
  - 3.9 参数文件中的三处已知口径
  - 3.10 数据来源与复现
  - 第 3 章引用清单
- 第 4 章 参考项目与资料来源
  - 4.1 证据分层与引用纪律
  - 4.2 参考的开源项目
    - 4.2.1 acados —— 求解器与代码生成
    - 4.2.2 agilicious —— 架构范本，不是代码来源
    - 4.2.3 Fast-Drone-XI35 —— 同硬件的整机参考
    - 4.2.4 OmniNxt —— 同系列硬件的平台佐证
    - 4.2.5 Sim2real-UAV-Params —— 参数与动捕链路的资料库
    - 4.2.6 MPC_uav —— 本报告描述的仓库本身
    - 4.2.7 px4-mpc —— 10 状态 OCP 的直接蓝本
    - 4.2.8 Nxt-FC 与 z0 固件 —— 飞控硬件与固件身份
  - 4.3 论文
    - 4.3.1 `Sun2022_TRO_NMPC_vs_DFBC.pdf`
    - 4.3.2 `Foehn2022_SciRob_agilicious.pdf`
    - 4.3.3 `Torrente2021_RAL_DataDrivenMPC.pdf`
    - 4.3.4 `Houska2011_Automatica_RTI.pdf`
    - 4.3.5 `Houska2011_OCAM_ACADOToolkit.pdf`

        - 4.3.6 Liu2024_IROS_OmniNxt.pdf
    - 4.4 移植边界与原创性
        - 4.4.1 已落地的复用与改写
        - 4.4.2 只有概念、没有代码
        - 4.4.3 本项目的原创部分
    - 4.5 资料来源与复现
    - 第 4 章引用清单
- 第 5 章 系统架构与实验平台
    - 5.1 符号与缩写
    - 5.2 三层架构
    - 5.3 硬件连接与软件栈
        - 5.3.1 硬件连接
        - 5.3.2 软件栈
    - 5.4 话题表
    - 5.5 两条数据链
        - 5.5.1 共同点
        - 5.5.2 区别：两档各一张表
    - 5.6 坐标系与转换发生在哪一侧
    - 5.7 安全边界：谁能让飞机动
    - 5.8 本章引用清单
- 第 6 章 项目结构与代码结构
    - 6.1 符号、口径与快照
    - 6.2 仓库结构
    - 6.3 七个 ROS 2 包
        - 6.3.1 输入与定位：vrpn_listener 与 mocap
        - 6.3.2 任务与参考：traj 与 control
        - 6.3.3 控制器：nmpc
        - 6.3.4 编排与测试：bringup 与 bench
    - 6.4 两条数据流：从 /traj/reference 到两个 setpoint 话题
    - 6.5 MPC 的层分界：零 ROS 核心层与节点层
        - 6.5.1 哪些模块属于核心层
        - 6.5.2 改哪一层要不要 build
    - 6.6 测试布局
        - 6.6.1 包内测试：seam 1 与 seam 2
        - 6.6.2 仓库级测试：run/tests/ 与 tools/*/test_*.py
    - 6.7 证据目录的角色
    - 6.8 本章引用清单
- 第 7 章 启动脚本与遥控等交互（含安全/FSM）
    - 7.1 术语与通道编号
    - 7.2 一键起链：两条链与一个问句
    - 7.3 启动顺序与判读
    - 7.4 输出路径：同一套任务、两个最后执行者
    - 7.5 录制：六条场景与一次收口
    - 7.6 遥控通道：四个开关，四条职责
    - 7.7 状态机：两个正交维度
    - 7.8 失效路径与复位
    - 7.9 安全路径的第一手证据
    - 7.10 现场边界：这条链里没有什么
    - 7.11 本章引用清单
- 第 8 章 MPC 公式推导与应用
    - 8.1 坐标系与符号
        - 8.1.1 三个坐标系与转换边界
        - 8.1.2 符号与代码变量对照

  - 8.2 刚体动力学
    - 8.2.1 平动
    - 8.2.2 转动：四元数运动学
    - 8.2.3 三条建模假设及其边界
  - 8.3 十状态模型
    - 8.3.1 状态与输入
    - 8.3.2 输入为什么是 “角速率指令” 而不是力矩
    - 8.3.3 这套控制能不能称为 NMPC
  - 8.4 离散化
    - 8.4.1 连续模型的离散形式
    - 8.4.2 RK4 与 ERK 的分工
  - 8.5 最优控制问题
    - 8.5.1 一般形式
    - 8.5.2 代价函数与权重
    - 8.5.3 参考与前馈
  - 8.6 约束
    - 8.6.1 推力上下限
    - 8.6.2 角速率上下限
    - 8.6.3 没有实现的约束
  - 8.7 求解器
    - 8.7.1 acados 配置与实时迭代语义
    - 8.7.2 实时预算与实测耗时
  - 8.8 应用到 PX4
    - 8.8.1 CTBR 接口
    - 8.8.2 ENU/FLU 到 FRD 的符号映射
    - 8.8.3 推力归一化链
  - 8.9 参考姿态为什么是水平 + 纯偏航
  - 8.10 模型失配与鲁棒性
    - 8.10.1 参数失配的证据现状
    - 8.10.2 结构假设与参数误差不是一回事
  - 8.11 求解失败兜底
    - 8.11.1 “可用” 与 “退化” 的判据
    - 8.11.2 兜底阶梯
  - 8.12 已实现与规划中
  - 8.13 本章引用清单
- 第 9 章 实验设计与数据处理
  - 9.1 数据组织
    - 9.1.1 四个源
    - 9.1.2 三份产物
  - 9.2 数据集
    - 9.2.1 六份话题包与场景映射
    - 9.2.2 三趟 nominal 仿真
  - 9.3 复现命令
  - 9.4 本章引用清单
- 第 10 章 实验结果与汇总
  - 10.1 读数口径
    - 10.1.1 公共窗与各源完成时刻
    - 10.1.2 图与表的生成方式
  - 10.2 轨迹图（T1–T3）
  - 10.3 姿态图（A1–A3）
  - 10.4 位置误差（E1–E3）
    - 10.4.1 误差–时间图
    - 10.4.2 位置指标与悬停稳态

- 10.5 控制量（C1–C3）
- 10.6 三组对比
  - 10.6.1 offboard 与 MPC（位置层）
  - 10.6.2 MPC 仿真与实飞
  - 10.6.3 求解耗时与退化
- 10.7 边界与未验证项
  - 10.7.1 五张 `wontfix` 票的缺口
  - 10.7.2 本轮读数的边界
  - 10.7.3 未验证项与不作外推的部分
- 10.8 小结

第 2 章 项目背景与目标

2.1 平台与目标

z0 是一台已经组装完成、并且做过人工飞行验证的 152 mm 涵道四旋翼，机架为 OddityRC XI35 Pro。飞控是 MicoAir NxtPX4v2，运行 PX4 1.14.3 系的 MicoAir 厂商构建——固件真身是 `Minderring/PX4-Autopilot @ 4a0e65f2`，由飞控自报的版本字段核对确认（refs-manifest.md §2.5(d)）。机载电脑是 Jetson Orin NX 8GB，本仓库就在这台机器上；室内定位由 Nokov 动捕系统提供，经 VRPN 送到机载侧。平台与版本的完整清单见 README.md 的「项目基础信息」。

本项目的目标不是再造一台能飞的机器，而是把 z0 建成一个可复用的自主飞行平台，并让 MPC 接管飞控内置的 PID。平台建设优先于单次演示：模块边界是否清晰、接口是否稳定、后续任务能否复用，本身就是验收标准的一部分（PROJECT.md §1.1）。

平台目标有两条：

- **整条链路可替换**：从动捕感知、参考规划、MPC 到 PX4 执行，做成一套首尾相接的完整系统；控制算法可以整块换掉，而感知、状态估计、轨迹与安全机制不需要跟着改（分层与接口见第 5 章）。
- **硬件平台大推重比、可高速飞行**：推重比决定机动与速度的物理上限，电池的可持续放电能力是当前最硬的约束；z0 的实际边界见第 3 章 §3.4.2。

四阶段路线遵循「每个阶段只引入一个新变量」：Phase 1 只验证「动捕 → 机载电脑 → offboard → 飞控 → 飞机」这条基础链路，控制律仍是 PX4 内置 PID；Phase 2 才把 MPC 接进控制回路，但只在 SITL 中验证，不碰真机；Phase 3 不引入新变量，负责把前两阶段摊开的文档与未结事项收敛掉；Phase 4 才让 MPC 在真机上飞，并在离地之前把每个可能摔机的未知量单独测出来（README.md 的四阶段路线）。

这条路线还带着一条贯穿全项目的纪律：结论必须能指回证据。项目早期有过两次结论站不住的教训——把未授权私有仓库的 404 当成「仓库不存在」，以及在只有 README、LICENSE 与 PDF 二进制的情况下声称读过论文全文——此后关键判断都要求落到第一手来源：源码路径、行号、commit、日志或数据文件（open-source-survey.md §0）。本报告中的读数同样要求带来源与窗口。

术语约定。本报告里 `SITL` 专指 PX4 软件在环仿真；「仿真」只作形容词或与「实飞」对举，不单独指代 SITL。`offboard` 小写指本项目的位置控制链路，`Offboard` 指 PX4 的飞行模式，`OFFBOARD` 指代码常量。正文叙述统一用「实飞」指 z0 真机飞行、用「Phase」指四阶段路线中的阶段；「真机」「上机」「阶段」只在引用原文或强调动作时出现。`MPC` 指 Model Predictive Control，与 PX4 的 `MPC_*`（Multicopter Position Controller）参数前缀无关。

打造一台全链（动捕感知 → 参考规划 → MPC → 飞控执行）无人机平台，并且可灵活切换和添加新的控制方式，硬件选型注重大推重比。

2.2 四阶段路线

表 1 给出四个阶段各自做了什么、控制接口是什么，以及截至 2026-10-17 的结论。

表 1　四阶段路线。

阶段	做了什么	控制接口	结论（截至 2026-10-17）
Phase 1	用动捕让 z0 稳定悬停并跟踪简单轨迹；PX4 跑位置、姿态、角速率全部内环 PID	**(A) 级**：`/mavros/setpoint_raw/local`（位置 + yaw）	2026-09-28 真机自由飞一次成功，以目测为准、数值未量；票面无未结，但阶段 spec 仍为 `ready-for-agent`
Phase 2	MPC 节点 + acados；在 z0 同构 SITL 机体上验证闭环	**(C) 级**：机体三轴角速率 + 总推力（SITL 阶段经 px4_msgs/uXRCE-DDS；Phase 4 起改 mavros）	SITL 闭环跑通，最高一级验收的四条判据都有结论（不都是通过）；票面无未结，但阶段 spec 仍为 `ready-for-agent`
Phase 3	整理入口层与 `docs/`，并结清 Phase 2 留下的未结票	无（不引入新的控制接口）	spec `done`（2026-10-09）

阶段	做了什么	控制接口	结论（截至 2026-10-17）
Phase 4	四项前置测量（部分未完成）；按拆桨 → 系绳 20 cm → 系绳悬停 30 s → 自由飞三条轨迹推进（阶梯 2 未达标，裁定跳过）	同 Phase 2（**(C) 级**）	自由飞三条轨迹完成；spec `done`（2026-10-17，操作者裁定收口）；5 张票记为 `wontfix` 的缺口

2.2.1 Phase 1 —— offboard 位置定点

Phase 1 把自主飞行限制在「位置」这一层：机载电脑只发位置与 yaw 设定点，位置环、姿态环、角速率环全部由 PX4 内置 PID 承担。2026-09-28 的首次真机自由飞一次成功，定点、定高、按键与三条轨迹均按目测判为正常；这次验收的口径是**目测为准、不补量数字**，因此它能证明的是「链路通了」，不是「跟踪精度是多少」（phase1 票 09）。

Phase 1 名下的拆桨、系绳 20 cm、系绳悬停 30 s 三级阶梯没有做；操作者 2026-09-28 裁定不再补做，这几级在（A）级下的读数至今为空（README.md 的四阶段路线）。票据层面 Phase 1 已没有未结票，但**阶段并未收口**：`phase1/spec.md` 的 `Status` 行仍是 `ready-for-agent`（phase1 spec）。这里必须区分「票收口」与「阶段收口」：前者指每张票有了处置，后者指 spec 本身被翻成 `done`，两者不是一回事。

结论：（A）级的动捕定点与轨迹飞行链路已在真机上打通，但精度未量、阶梯读数缺失，阶段按 spec 的实际状态只能算「未收口」。

一句话总结：Phase 1 搭建了一台动捕作为定位来源的 offboard 模式（控制部分为飞控固有 PID）的 PX4 无人机。

2.2.2 Phase 2 —— SITL 验证 MPC

Phase 2 才把 MPC 接进控制回路。控制层级选 **(C)**：MPC 输出总推力与机体三轴角速率，PX4 只保留 1 kHz 的角速率环与混控。被控对象与 z0 同构，但其物理参数当时并没有对齐真机实测口径——满量程推力明显偏小，直到 Phase 3 才按实测基线重搭（phase2 spec；phase3 票 04）。这一阶段只在仿真里闭环，不碰真机。

Phase 2 的最高一级验收把状态源换成动捕：参考轨迹取自真实动捕包，测量则由 SITL 真值叠加上从包里提取的噪声与延迟合成——没有把录好的帧直接当测量流，因为那样会断开反馈，注入延迟的判据也就失去意义（phase2 票 11；tier-b-sitl-seam3.md §3）。四条判据都留下了结论，其中包括未通过项，而不是「看着还行」。票据层面同样已无未结票，但 `phase2/spec.md` 的 `Status` 仍是 `ready-for-agent`（phase2 spec）。Phase 2 未收口，主要不是因为控制律还有悬案，而是因为「仿真能飞」与「真机能飞」之间隔着一批只能在真机上读出的量：真实的动捕到飞控延迟、推力标定、动捕与 IMU 的外参、电池在负载下的电压跌落（phase4 spec 的 Problem Statement）。

结论：（C）级 MPC 在 SITL 中完成了闭环验证，能摔机的错法被逐条验过；阶段未收口，未收口的部分是「真机上的量还没量」。

一句话总结：MPC 算法在仿真 SITL 中完成了闭环验证，但并未同步实机的物理参数。

2.2.3 Phase 3 —— 整理与收尾

Phase 3 不引入新变量，也不改控制律。它做两件事：把入口层与 `docs/` 裁到「一个上线的人或 agent 读得完」；把 Phase 2 留下的未结票结掉，因为这些票是 Phase 4 上机的前置（phase3 spec）。这一阶段的 spec 已于 2026-10-09 翻成 `done`，票面全部结清。

它不产生新的飞行证据，作用是把「散落的证据」整理成后续阶段可以站上去的地面：文档里读到的每句话要仍然成立，未结事项要有明确处置。没有这一步，Phase 4 就会带着已知的未知量起飞。

结论：阶段按 spec 的实际状态是 `done`；产出是整理后的入口层、`docs/` 与结清的票，而不是新的控制能力。

一句话总结：Phase 3 并未引入新功能，而是将前俩阶段的工作优化，并同步实机的物理参数。

2.2.4 Phase 4 —— MPC 上实机

Phase 4 是 MPC 第一次上真机。上机前安排了四项前置测量：电机实际位置、悬停油门、动捕与 IMU 之间的外参、动捕到飞控的端到端延迟（phase4 spec）。实际落地并不完整：悬停油门有实测回填；延迟有来源记录，但没有做 A/B；电机位置由「机架规格 + 操作者现场复认」推得，不是卡尺读数；外参只落了 z 方向的一部分——xy 未测，「动捕 z=0 平面即地面」的前提未证，倾斜段的实飞判据从未跑过。未完成的部分按操作者 2026-10-17 裁定记为 `wontfix`，没有证据作锚（phase4 票 01；phase4 票 02；phase4 票 03）。

然后按「拆桨看方向 → 系绳离地 20 cm → 系绳闭环悬停 30 s → 自由飞三条轨迹」四级阶梯推进。阶梯并不是每一级都过了：阶梯 1（拆桨看方向）用 `/mavros/rc/out` 的仪器读数判过，六段角速率指令的方向全部与预期一致；阶梯 2（系绳离地 20 cm）在 2026-10-16 实跑未达标——系绳长度没有把高度限制在 20 cm，飞机没有稳定悬停在该高度，随后下坠，并在水平方向跑出明显距离后撞地。操作者当天裁定**跳过这一级、直接进闭环**，并明令豁免判据页「哪级不过不进下一级」的规则（phase4 票 09；ladder-acceptance-criteria.md §5.1）。之后阶梯 3、4 才继续推进，具体读数见票 09 与第 10 章。

2026-10-17 的自由飞留下了三份 MPC 轨迹包（悬停、正方形、8 字；另有一份起飞包），供第 10 章与 Phase 1 的 offboard 数据同口径对比（bag/README.md）。Phase 4 的 spec 于同日翻成 `done`，依据是操作者裁定「全都收工」；其中 5 张票按裁定记为 `wontfix`。这 5 张并不都是同一种「没做」：多数是「裁定收、不做了」，票 14 是「被票 15 取代后收口」；无论哪一种，未做项都没有证据作锚，并不等于每一项都做过。具体缺口分别记在票 01（几何与外参）、票 04（固件不重刷）、票 09（阶梯 2）与票 16（阶梯 3/4 的未取证项），第 10 章会把它们作为「未验证项」列出。

结论：（C）级 MPC 完成了自由飞三条轨迹，阶段按操作者裁定收口；但前置测量与阶梯都有未完成项，阶梯 2 未达标后由裁定跳过，另有若干未取证项——这些缺口单独记账，不用「基本完成」掩盖。

一句话总结：MPC 算法在实机上完成了三种轨迹的验证，并进一步通过实机测量优化物理参数模型。

2.3 三条关键工程约束

2.3.1 室内无 GPS，动捕是唯一位置源

飞行场地在室内，没有 GPS。z0 的绝对定位只有动捕一条来源：Nokov 动捕主机把刚体位姿经 VRPN 送到机载电脑，再进入 PX4 的 EKF2；GPS 这条线在飞控参数上被显式关掉（`EKF2_GPS_CTRL = 0`、`SYS_HAS_GPS = 0`，见 params-changes.md §I.1）。这条约束有两个直接后果：第一，动捕链路中断时没有卫星定位可以作为备份；第二，所有位置与轨迹都是相对于某个局部原点的量，比较不同架次的数据时必须先对齐时间零点与轨迹原点（方法见第 9 章）。动捕链路的设计与延迟标定见 mocap-setup.md。

一句话总结：室内没有 GPS，动捕是唯一位置源，因此也没有卫星定位可以作为备份。

2.3.2 mavros 是唯一的机载链路

在 z0 真机上，机载电脑与飞控之间只走 mavros 一条链路。Phase 2 的 SITL 验证曾经用 px4_msgs/uXRCE-DDS 把 (C) 级指令发到 `/fmu/in/vehicle_rates_setpoint`；到 Phase 4 上机时这条路走不通——飞控的 `UXRCE_DDS_CFG` 是 `0`，串口又被 MAVLink 占着——于是 2026-10-12 起 (C) 级下行改为 mavros 的 `/mavros/setpoint_raw/attitude`（phase4 票 13；README.md 的「项目基础信息」）。所以：Phase 1 的 (A) 级、Phase 4 的 (C) 级实飞，以及今天代码里的 (C) 级下行，都经 mavros；uXRCE-DDS 从未在真机上使用。代价是 (C) 级要经过 mavros 的消息转换；收益是真机只有一条需要维护、需要排查的链路。Phase 2 的 SITL 证据是在旧载体上产出的，第 9、10 章引用那批证据时要把这一点写明。

一句话总结：真机只走 mavros 一条链路，SITL 阶段的 uXRCE-DDS 载体没有延续到实机。

2.3.3 控制层级选 (C)：总推力 + 机体三轴角速率

控制层级的编号描述的是 MPC 把控制量送到 PX4 的哪一层，四个选项的完整对照见 mpc-roadmap.md §1 与 PROJECT.md §4.1。本项目选 **(C) 级：总推力 + 机体三轴角速率**，也就是 CTBR（collective thrust + body rate）。它的含义是：MPC 接管位置环与姿态环，输出总推力，以及绕机体三轴的角速率期望；PX4 只保留 1 kHz 角速率环与混控（CONTEXT.md 的 CTBR 与 (C) 方案）。

不选另外三级的理由分别是：

- **(A) 级**只发位置与速度，MPC 的输出会被 PX4 的位置环「抹平」，模型等于没有进回路——这正是 Phase 1 的状态；
- **(B) 级**发姿态角与推力，仍然隔着 PX4 的姿态环，姿态环带宽会与 MPC 的预测时域打架，四元数设定点的偏航处理也更麻烦；
- **(D) 级**直接发电机指令，绕过 PX4 的 `control_allocator` 与安全网，摔机风险与标定工作量都超出本项目的范围。

(C) 级是这三者之间唯一「模型真正起作用、又不拆掉飞控安全机制」的落点（PROJECT.md §4.2）。需要强调：(C) 级定义的是**输出形态**，不是某一条具体话题；本项目当前经 mavros 的 `mavros_msgs/AttitudeTarget` 下发，将来更换载体不改变方案本身（CONTEXT.md；phase4 票 13）。

一句话总结：(C) 级（最终给到飞控的控制量：总推力 + 机体三轴角速率）让模型真正进入回路，同时保留 PX4 的角速率环与混控安全机制。

2.4 与后续章节的分工

本章只回答「为什么做、做到哪」，不展开「怎么做」：

- 动捕 → mavros → 控制节点 → PX4 的两条链、话题频率与坐标系转换发生在哪一侧，见第 5 章；
- 10 状态模型、四元数运动学、代价函数、约束与 acados/RTI 的配置，见第 8 章的推导；
- 时间零点、轨迹原点、公共窗、重采样与指标定义，见第 9 章；
- 三条轨迹 × 四种来源的图与指标，以及本章点到的未验证项，见第 10 章。

本章出现的阶段结论只指向仓库里的 spec、票与录制包，不复述具体数值；数值统一由第 10 章从数据生成。

一句话总结：本章只讲为什么做、做到哪；架构、推导、方法与结果分别留给后面四章。

第 3 章 无人机零件与物理参数

本章回答三个问题：z0 由哪些部件组成；其中哪些物理量真正进入了控制模型、约束与状态估计；哪些量至今没有测过。前两问决定报告里的数能不能被复现，第三问决定结论的适用边界。

本章不使用插图。每个数值都标了证据等级；数据出处与复跑命令集中在 § 3.10。

3.1 证据等级与符号约定

3.1.1 三类标记

标记	含义
实测	在 z0 上直接测得，或从飞控 / 工作树参数文件直接读出的**现值**。后者在备注里写「参数」。
推导	由实测值或厂商规格按本章写出的公式算出。公式随行给出，读者可以当场复算。
未测	没有 z0 上的证据。厂商规格、数据手册值也归入这一类，备注里写「厂商规格」。

「未测」**不等于 0，也不等于不可用**。它只表示这个量没有被 z0 的测量锚住：凡是要把它写进模型、约束或结论的地方，都应当先测再用，或者明确写出所用的替代口径。

正文里随行出现的中间值（例如 20.474 V、523 W、−2.3 ms）都取自下文列出的实测或台架来源；凡是不能归入 z0 实测的中间值，已在括号里注明其口径。

一句话总结：证据只分实测、推导、未测三类；未测不等于零，写进模型前必须先取得 z0 的证据。

3.1.2 符号表

符号	含义	单位
m	起飞质量（含电池）	kg
g	重力加速度，本章取 9.81	m/s²
T_{hover}	悬停所需总推力	N
T	推力（单电机或整机，按上下文；如 $P \propto T^{1.5}$）	N
F	模型中的总推力（优化变量与约束）	N
h	悬停归一化推力（`hover_thrust`）	1
$t_{\max}$	推力归一化常数（归一化满量程）	N
$f_{\max}$	电池可持续口径的推力上限	N
d	轴距（电机对电机对角线）	mm
a	电机位置在机体 x / y 轴上的分量	mm
P	电功率	W
V	电池电压	V
ω	机体角速率	rad/s
$\omega_{\max}$	飞控允许的最大机体角速率	rad/s
k_ω	等效角速率跟踪增益（13 状态模型的输入环节）	1/s
τ_{lat}	动捕到飞控的端到端延迟	ms
$\mathbf{r}$	动捕刚体原点 → IMU 中心的外参（机体系）	m
I_{xx}, I_{yy}, I_{zz}	三轴转动惯量	kg·m²
k_t, k_q	推力系数、反扭矩系数	—
τ_m	电机时间常数	s

本章的 g 取 9.81 m/s²，与模型代码 `src/nmpc/nmpc/params.py` 的 `G` 一致。若改用 9.80665 m/s²，T_{hover} 为 7.3550 N，与本章的 7.3575 N 相差 0.03%，不改变任何结论。

3.2 零件清单

类别	型号	关键规格	数量	证据等级与来源
机架	怪象 OddityRC **XI35 Pro**	3.5 英寸**涵道**机架（cinewhoop 类），轴距 152 mm，机架自重 118 g（含螺丝 / 打印件）	1	轴距：实测（操作者现场复认，非卡尺）；自重：未测（厂商规格）
飞控	微空科技（MicoAir）**NxtPX4v2**	STM32H743VIH6 @ 480 MHz；双 BMI088；SPL06 气压计；**无磁罗盘**；7 路 UART；无以太网；8 路 DShot/BDShot；27 × 32 × 8 mm，6.5 g；20 × 20 mm 安装孔	1	未测（厂商规格）；板级目标名 `hkust_nxt-dual` 为实测（飞控自报）
电调	微空科技 **33A 四合一**（「蓝鸟 48K」）	4-in-1 ESC，33 A/路；DShot，`DSHOT_MIN = 0.055`、`DSHOT_3D_ENABLE = 0`；遥测关闭（`DSHOT_TEL_CFG = 0`）	1	33 A/路与固件型号：未测（厂商规格，固件型号描述含糊）；DShot 参数：实测（参数）
电机	OddityRC **Spinnybois 2006-2150KV**（升级款）	定子 20 × 6 mm；**12 槽 14 极**；KV = 2150；6S 定位	4	未测（厂商规格）；`MOT_POLE_COUNT = 14` 与 12 槽 14 极一致（实测，参数）
螺旋桨	HQProp **DT90MMX3**（用户也叫 `T90MMX3`）	直径 90 mm（3.5 英寸级）；螺距 3 英寸；三叶；PC 材质；2.35 g/支；1.5 mm T-mount 轴孔	2 正 + 2 反	未测（厂商规格）；正反桨数量为实测（操作者确认）
电池	6S1P **Sony/Murata US18650VTC6** 3000 mAh	标称 22.2 V；满电 25.2 V；实用截止 18.0 V；持续放电 15 A（Mooch 实测）/ 30 A（厂商标称）	1	配置：实测（操作者确认）；电压：推导（6×3.7 / 6×4.2 / 6×3.0 V）；容量、内阻、放电能力：未测（厂商 / 第三方测试规格）
遥控器	RadioMaster **Pocket**	ELRS 2.4 GHz；EdgeTX；CRSF over UART，典型 420000 baud；不标配接收机	1 套	未测（厂商规格）；通道映射见第 7 章
机载电脑	**Jetson Orin NX 8GB** + ORIN NX V2 底板	6 核；7.4 GB 内存（可用 5.0 GB）；233 GB 磁盘；Ubuntu 22.04 / L4T R36.5.0 / 内核 5.15.185-tegra；ROS 2 Humble；无 `nvcc`、无 TensorRT	1	实测（环境侦察，2026-09-18）
动捕	Nokov 光学动捕	VRPN/UDP 端口 3883；配置说明的出包频率约 200 Hz；z0 刚体由 5 颗反光球组成，刚体名 `z0`	1 套	端口与刚体名：实测（配置与现场核对）；出包频率：未测（配置说明）；相机数量与布置：未测（本次未记录）

下面对表中三处容易产生误解的地方作补充。

涵道不单独建模。 XI35 Pro 是涵道机，但本项目的动力学模型按普通四旋翼处理，把涵道对推力和阻力的影响吸收进后续辨识出的系数里。单独建涵道气动模型的成本远高于收益，而且这架飞机本来就不是为高速飞行设计的。

桨的型号要写对。用户口语里的「翼飞 ProTek35 桨叶」并不准确：ProTek35 的原厂配桨是 HQProp Nazgul 3535，`DT90MMX3` 是被卖家标注为「ProTek35 兼容」的第三方桨。两者不是同一个桨，查推力数据时不能混用。

飞控的板级目标名不是上游型号。飞控自报的硬件名是 `HKUST_NXT_DUAL`，对应 PX4 目标 `hkust_nxt-dual`。该目标不在上游 PX4 1.14.3 里，来自厂商分支；本项目不刷固件，沿用机上现有的 PX4 1.14.x 厂商构建。

硬件配置的参考来源有一条共同限制。表中的厂商规格来自项目早期的硬件调查：当时 `WebFetch` 被网络策略全线拦截，没有打开过任何官方页面，型号与参数全部取自搜索结果摘要，存在过时、转述

或串型号的风险（见 `hardware.md` §0）。因此凡标「未测（厂商规格）」的条目，在写进模型前都应当复核一次；轴距、起飞质量、正反桨数量与电池配置这类由操作者确认的量不受这条限制。

3.3 几何与质量

项	值	证据等级与来源
构型	四旋翼 X 型（Quad X，无正前方电机）	实测（操作者确认）
PX4 机型	`SYS_AUTOSTART = 4001`（Generic Quad X），`MAV_TYPE = 2`	实测（参数）
轴距 d	**152 mm**（电机对电机对角线）	实测（机架规格 + 操作者 2026-10-09 现场复认；**不是卡尺读数**）
臂长	**76 mm**（机体中心到电机的对角线距离，即 $d/2$）	推导：$152/2$
电机位置（机体 x 前 / y 右）	$(\pm 0.0537401,\ \pm 0.0537401)$ m	推导：$a = 76/\sqrt{2} = 53.7401$ mm
布局形状	正方形、正 X（四个象限各一）	实测（操作者确认，定性）
机架自重	118 g	未测（厂商规格）
起飞质量 m	**0.750 kg**	实测（操作者称重）
电机序	物理输出口 → 逻辑电机 = **`3-2-1-4`**	实测（参数 `PWM_MAIN_FUNC1..4 = 103, 102, 101, 104`）

750 g 是整机称重；除机架 118 g 之外，电机、桨、电池、飞控、机载电脑没有逐项称重，质量分解在本项目里**未测**。

一句话总结：152 mm 是电机对角线，轴上的坐标分量是 ±53.74 mm；整机 750 g 已称重，其余部件质量未分解。

3.4 动力与推力

3.4.1 四个容易混淆的推力量

表里的四个量分属三个层次：物理推力（T_{hover}）、归一化（h、$t_{\max}$）与电池约束（$f_{\max}$）。它们不能互相替代。

量	符号	值	证据等级	用途
悬停所需总推力	T_{hover}	**7.3575 N**	推导：mg	悬停工作点、推力坐标系的原点
悬停归一化推力	h	**0.1589**	推导：实测悬停点 → 满电归一化	归一化推力，等价于 `hover_thrust`
推力归一化常数	$t_{\max}$	**46.30 N**	推导：T_{hover}/h	把模型里的 F（N）换成 PX4 的 `thrust_body`（归一化）
电池可持续推力上限	$f_{\max}$	**11.8 N**	推导：15 A 持续放电外推	优化问题的 F 上限（`ocp.py` 的 `ubu[0]`）

$$T_{\text{hover}} = mg = 0.750 \times 9.81 = 7.3575\ \text{N} \tag{3-1}$$

$$t_{\max} = \frac{T_{\text{hover}}}{h} = \frac{7.3575}{0.1589} = 46.30\ \text{N} \tag{3-2}$$

$h = 0.1589$ **是怎么来的**。它不是从参考机抄来的常数，而是从 z0 自己的悬停日志反推的。两段可用的稳态悬停段（电压中位数分别为 20.474 V 与 20.526 V）给出 $h = 0.221992$ 与 0.220884，互差 0.50%。按同一批数据就地标定的推力—电压关系 $T \propto V^{1.608}$ 归一到满电 25.2 V：

$$h(25.2) = h(V_{\text{窗口}}) \left(\frac{V_{\text{窗口}}}{25.2} \right)^{1.608}, \tag{3-3}$$

得到 0.15896 与 0.15882，取 **0.1589**。所以更准确的写法是：**两段实测点 + 满电口径的归一化外推**。表里把它归入「推导」，因为 0.1589 本身不是在 25.2 V 下直接读到的。

h **是归一化推力，不是电机油门信号**。两者之间还隔着一层 `THR_MDL_FAC` 曲线，不能把 0.1589 直接读成「电机油门 15.89%」。飞控参数 `MPC_THR_HOVER` 与本章的 h 处在**同一个归一化推力域**（PX4 对 `thrust_body` 不做重标定），这正是它应当回填为 0.1589 的理由；但 `docs/assets/z0.params` 里的现值仍是 PX4 默认的 **0.5**，不能拿它当实测值。在 (C) 级接口下，`MPC_THR_HOVER` 不参与我们发出的推力，它的实际消费者是位置模式、悬停推力估计器与着陆检测（见 `params-changes.md` C1、C5）。

$t_{\max} = 46.30$ **N 不是「实测电机最大推力」**。它是由实测悬停点反推出来的**归一化满量程**：悬停时 PX4 打在 `thrust_body` 上的归一化值恰好是 $T_{\text{hover}}/t_{\max}$，所以知道了 T_{hover} 和这个归一化读数，就能反解出 $t_{\max}$。它只负责把牛顿换算成归一化推力；PX4 在 offboard 分支对 `thrust_body` 不做重标定，这个常数错了，悬停前馈就会整体偏。

$f_{\max} = 11.8$ **N 是电池口径，不是电机口径**。它由 VTC6 的 15 A 持续放电能力（Mooch 第三方测试）反推：$15\ \text{A} \times 22.2\ \text{V} = 333\ \text{W}$，四电机平分即 $333/4 = 83.25$ W/电机；按台架效率点（523 W / 1020 g；**GFD90-3 桨，不是 z0 的 DT90MMX3，也未在 z0 上复测**）与 $P \propto T^{1.5}$ 外推，单电机推力 $1020 \times (83.25/523)^{2/3} \approx 299.5$ g，四电机总推力约 1198 g ≈ 11.75 N，取三位为 **11.8 N**。这个数回答的是「电池能持续兑现多少推力」，不是「电机和桨最多能推出多少」。后者由 $t_{\max}$ 反映：按满量程 46.30 N 算，电机 / 桨口径的推重比约 **6.29**。

> **$t_{\max}$ 与 $f_{\max}$ 之比是 3.92**。2026-09-29 之前，项目把它们合用一个常数；回填实测悬停点时顺手把 F 上限从 11.8 N 抬到 46.3 N（3.9 倍），而所有测试仍然全绿。这是一个静默的可行域错误：优化器会算出物理上供不出来的推力。两者必须分开使用——$t_{\max}$ 进归一化，$f_{\max}$ 进约束。

注意：悬停推力的归一化值、满量程常数、电池上限分属三层，混用会让参数与实机有出入。

3.4.2 推重比与电池工作点

口径	电流依据	可用总推力	推重比 $T/(mg)$	证据等级
电池第三方测试（Mooch，保守）	第三方测试，15 A 持续	≈ 11.8 N	**1.60**	推导
电池厂商规格（乐观）	厂商规格，30 A 持续	≈ 18.7 N	**2.54**	推导
归一化满量程口径（$t_{\max}$）	由实测悬停点反推	46.30 N	**6.29**	推导；**不是静态拉力实测**

从电机与桨的能力看，z0 是一台大推重比平台。按归一化满量程 $t_{\max} = 46.30$ N 计，推重比达到 **6.29**，约为悬停所需推力的 6.3 倍；这正是第 2 章把「大推重比」写成平台目标的硬件依据。⚠ 但 6.29 是**能力上限**，不是可持续飞行的边界：电池持续放电把可兑现的推重比压到 1.60–2.54。**判断平台潜力看 6.29，判断轨迹与机动边界看 1.60–2.54**。

电池口径的推重比只有 1.60–2.54，这是全机最硬的边界。文献里 20 m/s 级机动飞行通常需要 4–6 的推重比裕度（文献经验值，未在本项目验证）；本项目在 1.60 下可以做很好的轨迹跟踪和抗扰，但不应当设计竞速级机动，参考轨迹的加速度指令必须保守。

实测电池工作点（从归档飞控日志读取，不是台架测量）：

量	读数	证据等级
悬停功耗	76–84 W	实测（归档日志，稳态悬停锚点）
悬停电流	3.36–4.13 A	实测（归档日志；**继承 `BAT1_A_PER_V = 17.0` 这个未验证的电流标度**）
外推到 $f_{\max}$ 所需电流	7.0–8.5 A，占 15 A 额定的 47–57%	推导

这组读数说明：在已飞过的电量区间里，$f_{\max}$ 对应的电流还没有接近 15 A 限额。**但这不等于 $f_{\max}$ 一定保守**。如果 `BAT1_A_PER_V = 17.0` 把电流低估了约 2 倍，那么 $f_{\max}$ 处的真实电流就正好压在 15 A 上。日志里没有拉力台、钳形表或第二条电流通路，无法分开这两种解释；「$f_{\max}$ 保不保守」在本项目里仍是未判定项，归档里最低也只飞到 19.26 V，离 18.0 V 的实用截止还有距离。

一句话总结：实测并推导了电池 满电 ~ 亏电 全过程的电压-总推力曲线。

3.4.3 最大起飞重量与推重比

把「最大起飞重量」定义为**可用推力刚好等于重力**时的整机质量：

$$m_{\max} = \frac{F_{\text{可用}}}{g}, \qquad \frac{T}{W} = \frac{F_{\text{可用}}}{mg}. \tag{3-4}$$

按这个定义，三种推力口径给出的上限如下（当前起飞质量 750 g 作对照）：

推力口径	可用推力	最大起飞重量 $m_{\max} = F/g$	相对 750 g 的余量	当前质量下的推重比
电池 15 A（Mooch，保守）	11.8 N	**≈1.20 kg（1203 g）**	≈450 g	**1.60**
电池 30 A（厂商标称，乐观）	≈18.7 N	≈1.91 kg（1906 g）	≈1.16 kg	**2.54**
归一化满量程（$t_{\max}$）	46.30 N	≈4.72 kg（4720 g）	≈3.97 kg	**6.29**

只有第一行是可持续口径。15 A 是 VTC6 的持续放电能力（Mooch 第三方测试），对应电池能长期兑现的功率；30 A 一行改用厂商标称值，比保守口径乐观一倍；46.30 N 一行只是电机与桨的归一化满量程，既不代表电池能持续供电，也没有计算机架结构、电调与温升的承载。**能用的最大起飞重量取第一行，其余两行只作上界对照**。

⚠ $T/W = 1$ 是没有爬升、抗扰与加速度裕度的悬停线，因此 $m_{\max}$ 是理论上界，不是推荐起飞重量。当前 750 g 在第一行口径下仍有 1.60 的推重比；把质量加到 1.20 kg，悬停就会正好压在 15 A 持续限额上，没有任何余量。若需要机动裕度，应当从 1.20 kg 往下取，而不是把 1203 g 当作可挂载的负载上限。

一句话总结：按三种推力口径算出的最大起飞重量约 1.20、1.91、4.72 kg，但只有电池持续口径的 1.20 kg 是可兑现上界。

3.5 飞控限制

这些参数定义了 MPC 输出的角速率与推力可行域。它们是从飞控参数文件读到的**现值**，不是本章新测的量。

限制	参数	值	换算	证据等级
最大横滚角速率	`MC_ROLLRATE_MAX`	220 °/s	**3.84 rad/s**	实测（参数）
最大俯仰角速率	`MC_PITCHRATE_MAX`	220 °/s	**3.84 rad/s**	实测（参数）
最大偏航角速率	`MC_YAWRATE_MAX`	360 °/s	**6.28 rad/s**	实测（参数）
最大空中倾角	`MPC_TILTMAX_AIR`	**45°**	—	实测（参数）

限制	参数	值	换算	证据等级
推力曲线因子	`THR_MDL_FAC`	**0.45**	—	实测（参数）；形状由飞行数据支持
归一化推力下限 / 上限	`MPC_THR_MIN` / `MPC_THR_MAX`	0.12 / 1.0	—	实测（参数）

`THR_MDL_FAC = 0.45` 有飞行数据支持，不只是「参数里写着」。把该因子当自由参数在 $[0, 1]$ 上扫描，两条真机日志的拟合优度峰值落在 0.450 与 0.455（网格步长 0.005），明显高于对照档 $fac = 0$（0.9952 / 0.9955）与 $fac = 1$（0.9871 / 0.9836）。这支持保留 0.45，但它是**形状**结论，不能反过来推悬停推力：推力曲线工作在「归一化推力 ↔ 电机信号」这一段，悬停归一化推力工作在更上游的归一化推力域，两者必须各自独立量。

> ⚠ 参数名前缀 `MPC_` 是 PX4 **Multicopter Position Controller** 的缩写，与 MPC 控制器无关；本报告的 MPC 指 **Model Predictive Control**。本章只在代码 / 参数名里保留这个前缀，正文一律写「MPC」。

3.6 延迟与外参

项	值	证据等级与来源
动捕到飞控的端到端延迟 τ_{lat}	**≈ 50 ms**	实测：z0 飞控日志，IMU 陀螺 × 动捕四元数差分的互相关峰
飞控视觉延迟补偿	`EKF2_EV_DELAY = 50.0` ms	实测（参数）
动捕刚体原点 → IMU 外参 **r**	**(0, 0, −0.080) m**（机体系 FLU）	未测（操作者给定并落地在 `mocap` 节点；推导值 ≈ −0.0885 m）
飞控外参参数	`EKF2_EV_POS_X/Y/Z = 0/0/0`	实测（参数）；这个 0 是**由构造保证正确**的，不是漏标
刚体反光球	5 颗，铺开约 114 × 100 × 23 mm	实测（`bag/1_static`）

3.6.1 端到端延迟

延迟的测法是交叉相关：把飞控 IMU 的陀螺读数与动捕四元数差分得到的体轴角速度对齐，取相关峰。两者是同一个物理量，不需要换算。作为对照，用飞控陀螺与飞控自身姿态差分做同一计算，结果必须落在 0 附近；实测对照为 **−2.3 ms**（$r = 0.999$），说明方法本身没有系统性偏移。主口径给出 **≈ 50 ms**（相关峰 $r = 0.967$；点估计的原始读数与分段离散度见 `mocap-setup.md` §6.3）。

2026-10-17 在三趟新飞行的日志上复测，悬停为 +45.9 / +47.0 ms（与主口径点估计的差在 ±10 ms 分辨率内），8 字为 +68.4 / +69.4 ms（高出约 20 ms，超出分辨率）。所以**这个延迟不是常数**，它随飞行状态变化，机理尚未钉死。`EKF2_EV_DELAY = 50.0` 落在复现范围的下沿；没有做过「改这个参数前后可观测差异」的实飞 A/B，因此本章不建议改它。

一句话总结：动捕到飞控的延迟约 50 ms，且随飞行状态变化；`EKF2_EV_DELAY` 保持 50，没有做改参 A/B。

3.6.2 刚体原点与 IMU 的 z 向偏移

动捕刚体原点取 5 颗反光球的无权质心，它并不在 IMU 上。飞机停在地面时（`bag/1_static`，2026-09-29），动捕原始 z 为 **+0.10850 m**（20346 帧，标准差 0.06 mm）。以「飞控离地约 20 mm」为估计，可以推出飞控侧应该填的外参约为 **−0.0885 m**（FRD，参考点是 IMU，z 向下为正）。

实际操作者给定的落地值是 **−0.080 m**，做在 `mocap` 节点里（`body_to_imu_z = -0.080`，机体系 FLU，z 向上为正）。两者相差 8.5 mm，来源是「离地 20 mm」本身是估的；**两个数都保留**：80 mm 是代码里的落点，−0.0885 m 是推导依据。

因此飞控侧的 EKF2_EV_POS_X/Y/Z 保持 0/0/0。☒ 不要两边都做：节点侧和飞控侧各做一次等于偏移双计，错一倍，而且两边都「看起来对」。x、y 两个分量至今**未测**，现按零处理。

> 前提「动捕 $z = 0$ 就是地面」仍待验证，而且这个偏移是**每场次现读**的：2026-09-21 那次原始 z 为 +0.0883 m，与 09-29 的 +0.10850 m 相差 20.2 mm。换场地或重新摆放刚体后必须复核。

一句话总结：刚体原点比 IMU 高约 80 mm，外参只做在动捕节点一侧；两边都做会把偏移双计。

3.7 电池

项	值	证据等级
配置	6S1P	实测（操作者确认）
电芯	Sony/Murata US18650VTC6，3000 mAh	未测（厂商规格）
标称 / 满电 / 实用截止	22.2 V / 25.2 V / 18.0 V	推导：6×3.7 / 6×4.2 / 6×3.0 V
单节内阻	≈ 13 mΩ	未测（厂商规格：数据手册）
整包内阻	≈ 78 mΩ（13×6）	推导
持续放电	15 A（Mooch 实测）/ 30 A（厂商标称）	未测（第三方测试 / 厂商规格）
归档日志的电压中位数	**20.48–23.15 V**；最低单点 **19.26 V**	实测（归档日志）
悬停功耗	76–84 W	实测（归档日志）

两处读数不能按字面使用。 BAT1_R_INTERNAL = 0.005 Ω（5 mΩ）比整包内阻（$13 \times 6 \approx 78$ mΩ）小约 15 倍：它只影响 PX4 对电压跌落的补偿估算，不进入本章的推力推导，但做 sag 补偿前必须先改对；归档最低单点 19.26 V，低电量端的行为没有被触发过。所有安培读数又都继承未验证的 BAT1_A_PER_V = 17.0，日志里没有第二条电流通路，内阻回归的 90–200 mΩ 与数据手册同量级，却分不开「标度正确」与「低估两倍」两种解释。BAT1_N_CELLS = 6 是参数，voltage_cell_v[*] 整份为 0，不能用来判断单节状态。

一句话总结：内阻参数比真值小约 15 倍，电流标度也未验证；两者都不进入推力推导，但用电池电压做补偿前必须先改对。

3.8 未测量项

下面每一项都**没有** z0 上的测量证据。它们在报告里不以 0 出现；需要用到时，必须写明替代口径。

项目	状态	影响与说明
转动惯量 I_{xx}, I_{yy}, I_{zz}	未测	当前的 10 状态模型把机体角速率 ω 当作**输入**，不出现转动惯量；若升级到 13 状态（角速率作为状态），必须先用双线摆、CAD 估算或角速率辨识得到它。
推力系数 k_t、反扭矩系数 k_q	未测	参数文件里的 CA_ROTOR*_CT = 6.5、CA_ROTOR*_KM = ±0.05 是 PX4 模块默认值，不是 z0 的辨识结果。当前 10 状态模型不直接使用 k_t, k_q；它们属于 (D) 方案与力矩饱和估算的输入。
电机时间常数 τ_m	未测	10 状态模型假设内环完美跟踪，不出现 τ_m；电机 / 执行器层面的模型需要它，13 状态升级则需要等效角速率跟踪增益 k_ω。两者都未辨识。
气动阻力	未测	第一版模型忽略阻力，低速室内飞行可以接受；高速或长距离轨迹需要辨识。
桨转向（props-in / props-out）	未测	只知道「2 正 2 反」。它直接影响力矩符号；不能从 CA_ROTOR*_KM 的正负推断（那是模板默认值），必须靠肉眼观察迎风面或单电机低速点转确认。

项目	状态	影响与说明
DT90MMX3 的全量推力—油门曲线	未测	现有台架数据对应的是 GFD90-3 桨，不是本项目用的 DT90MMX3；现有证据只覆盖悬停工作点与 `THR_MDL_FAC` 的曲线形状，不覆盖全量程。
动捕 ↔ IMU 的 x、y 外参	未测	现按零处理；z 已有 80 mm 的落点（见 § 3.6.2）。
DShot / ESC 遥测	未测	`DSHOT_TEL_CFG = 0`，电调固件型号也含糊；若开启，可能拿到每电机转速 / 电流，对辨识有价值，但本项目没有验证过。

此外还有四项与本章结论直接相关、但同样没有闭合的量：

- $f_{\max} = 11.8$ N **是否保守**：未判定。它本身是从 15 A 反推的，而电流标度未验证（§ 3.7）。
- **`BAT1_A_PER_V = 17.0` 是否正确**：未验证，所有安培读数都继承它。
- **电机位置的逐轴卡尺值**：未测。$(\pm 53.7401,\ \pm 53.7401)$ mm 是由「对角线 152 mm + 正方形」推得的。
- **动捕相机数量与布置**：未记录。本章只登记了 VRPN 端口、配置说明的出包频率和刚体构成。

一句话总结：凡是本章标为未测的量都没有 z0 证据，报告不以零代替；写进模型或约束前必须先测。

3.9 参数文件中的三处已知口径

`docs/assets/z0.params` 是飞控现状的第一手导出，但下面三处不能按字面直接使用。它们不是「文件过时」，而是**有意保留**或**尚未回填**的口径差异。

参数	文件里的值	本章使用的值 / 口径	为什么不一致
`CA_ROTOR0..3_PX/PY`	±0.15 m	真值 ±0.0537401 m，但**不改参数**	两者是纯均匀缩放，控制分配器按列范数归一化后完全约掉；真值只给 (D) 方案与力矩饱和估算用。
`BAT1_R_INTERNAL`	0.005 Ω	整包内阻 ≈ 0.078 Ω（推导）	参数比真实值小约 15 倍；只在做电压 sag 补偿时才需要改。
`MPC_THR_HOVER`	0.5	归一化悬停推力 $h = 0.1589$	0.5 是 PX4 默认值，尚未回填。它与 h **同域**（同一归一化推力），但 (C) 级接口下不参与我们发出的推力；其消费者是位置模式、悬停推力估计器与着陆检测。回填理由与四处连带效应见 `params-changes.md` C1。

3.10 数据来源与复现

本章的数来自参数文件、模型代码和几件测量工具，按用途列出。

参数现值：`docs/assets/z0.params`。它已于 2026-09-21 与飞控真身逐条对账；本章只引用参数名与值，不引用文件行数（行数会随现场重导出变化）。

模型参数：`src/nmpc/nmpc/params.py`。其中 `G = 9.81`、`HOVER_THRUST = 0.1589`、`F_MAX_BATTERY = 11.8`，并由 `t_max_from_hover()` 把 $t_{\max}$ 与 h 锁成一条导出关系。

悬停推力与推力曲线：

```
# 从 z0 自己的悬停日志反推 hover_thrust
python3 tools/hover-thrust/hover_thrust_from_ulog.py <悬停日志.ulg> --model-hover 0.1589

# 反解 THR_MDL_FAC（形状，不是悬停推力）
python3 tools/hover-thrust/thrust_curve_check.py <悬停日志.ulg>
```

证据与判据说明见 tools/hover-thrust/README.md 与 tools/hover-thrust/evidence/README.md。

电池与电流：

```
# 单份归档日志
python3 tools/battery-thrust/battery_from_ulog.py bag/3_traj1_hover/fcu/<日志>.ulg

# 整批汇总（会核对 bag/MANIFEST.md 的 sha256）
python3 tools/battery-thrust/evidence/summarise.py
```

读法与「哪些量取不到」见 tools/battery-thrust/README.md 与 tools/battery-thrust/evidence/README

延迟：

```
python3 tools/tau-analysis/tau_from_ulog.py <飞控日志.ulg> --dt 0.002 --fc 2.0 --segment
```

方法与三条口径（主口径、交叉、对照）见 docs/reference/mocap-setup.md §6.3，原始证据在 tools/tau-analysis/evidence/。

几何与外参：判定全文见 phase4/01；外参落点在 src/mocap/README.md 与 params-changes.md §C2。

一句话总结：本章每个数都能从参数文件、模型代码或仓库内工具复跑；复跑命令与证据清单一并列出。

第 3 章引用清单

主题	文件
硬件调查、建模参数汇总、待测清单	docs/reference/hardware.md
项目总纲与硬件平台	docs/PROJECT.md §2
参数逐项分析（几何、内阻、速率限制）	docs/reference/params-analysis.md
参数改动与落地状态	docs/reference/params-changes.md
飞控参数现值	docs/assets/z0.params
悬停推力反推工具	tools/hover-thrust/
电池可持续推力工具	tools/battery-thrust/
动捕延迟实测工具	tools/tau-analysis/
动捕链路与外参	docs/reference/mocap-setup.md、src/mocap/README.md
几何与外参判定	phase4/01
延迟实测票	phase4/03

第 4 章 参考项目与资料来源

本章回答三个问题：本项目的哪些部分站在外部项目与论文之上；每个来源借了什么、没借什么；以及哪些结论能回到源码或数据。

参考仓库的只读副本放在 `~/refs/`，不进 git。本章凡写版本号，都以 `refs-manifest.md` 为准；系统的分层、代码结构与公式推导分别留给第 5、6、8 章。

4.1 证据分层与引用纪律

本章把证据分成四类，让读者知道一句话该信到什么程度。

等级	定义	典型锚点
第一手·源码	克隆到钉住的 commit，直接读文件或看许可证	仓库结构、状态机枚举、板级默认值
第一手·运行或全文	在本机构建、实测，或从 PDF 全文抽取	acados 构建、飞控自报版本、论文摘要
推断	由第一手事实外推，但没有直接验证	“这套方法在 z0 上也会有效”
二手	搜索摘要、旧索引、第三方转述	只用于定位线索，不作为结论判据

写作时遵守四条：版本只认钉住的 commit 或飞控自报字段；论文数字写成 “论文报告了什么”，不写成 “本项目验证了什么”；推断必须显式标注；本仓库的状态以 git 提交和票面 `Status` 为准。

一句话总结：本章把证据分为源码、运行/全文、推断与二手四级；版本只认锚点，推断必须写明。

4.2 参考的开源项目

下表给出八个项目的来源、锚点与角色。“是否复用代码” 只回答文件是否进入本仓库，不评价借鉴深度。

项目	来源实验室	钉住的 commit / 版本	在本项目中的角色	是否复用代码
`acados/acados`	弗莱堡大学 syscop（Diehl 组）	`00947ddc`（`v0.6.0-41-g00947ddc3`）	求解器与代码生成	否，外部依赖
`uzh-rpg/agilicious`	UZH-RPG	`50bba45`	架构与选型佐证	否
`Longer95479/Fast-Drone-XI35`	哈工大 HIT-K1928-LAB	`8c80ddb`（分支 `board`）	同硬件整机参考，代码框架和接口	否，状态机重写
`HKUST-Aerial-Robotics/OmniNxt`	港科大 Aerial Robotics Group	`79ce1d4`	同系列硬件佐证，代码框架和接口	否
`Huan-qun/Sim2real-UAV-Params-`	个人仓库，未注明实验室	`b7aeebb`	参数、动捕文档与监听器	是，`vrpn_listener` vendored
`weisama/MPC_uav`	本项目	`21eb4a7`（Phase 4 收口）	本报告描述的仓库本身	是，本仓库
`DISCOWER/px4-mpc`	KTH / ETH（ATMOS 团队）	`3e541af`	10 状态 OCP 与节点骨架	是，模型与 OCP 改写

4.2.1 acados —— 求解器与代码生成

`acados/acados` 是基于 CasADi 的嵌入式最优控制求解器，能把最优控制问题生成成 C 代码。它用 HPIPM 解 QP、BLASFEO 做线性代数、`t_renderer` 生成 C 代码，许可证为 BSD 2-Clause。

借了什么。求解器与代码生成整条链路都作为外部依赖使用，不重新实现：`acados_template` 搭问题，HPIPM、BLASFEO 与 `t_renderer` 在生成和求解时工作。求解器配置沿用成熟选项，具体参数留给第 8 章。机载电脑上只需两处适配：BLASFEO 目标取 `ARMV8A_ARM_CORTEX_A76`（上游没有 A78 目标），HPIPM 取 `GENERIC`。

没借什么。不走 CUDA，也不用预编译二进制：z0 的机载电脑没有 `nvcc` 与 TensorRT，acados 在本机按源码编译，安装到 `~/acados`，构建目录在仓库之外。

锚点。`00947ddc394b30c956d9a7794d0bbcbc44c7c050`，2026-09-25。构建细节见 `acados-orin-build.md`。

一句话总结：acados 是一个强大的广泛被引用的 MPC 求解器，极大地加速了 MPC 求解，提供 MPC 代码范式与代码生成。

4.2.2 agilicious —— 架构范本，不是代码来源

`uzh-rpg/agilicious` 是 UZH-RPG 的敏捷四旋翼框架，对应论文 `Foehn2022`。仓库钉在 `50bba45`（2023-03-07），只有 38 个文件，全是文档、容器脚本与图片，没有任何 `.cpp`、`.hpp`、`.yaml` 或 `.launch`。

借了什么。借的是四个概念：

- guard：一条与主链路并行的备份链路，主链路异常时接管。z0 没有实现独立的 guard pipeline，实际安全网是 PX4 的 EKF2、遥控接管与 `AUTO.LAND`。
- 求解器选型：它的 MPC 使用 acados + HPIPM，为本项目的选型提供旁证。
- INDI 内环：它的控制器列表把 INDI 放在内环跟踪角速率。本项目把它列为 “(C) → (C)+INDI“ 的下一步，**尚未实现**。
- 多 pipeline：它支持多条 pipeline 与运行时切换；本项目只按阶段选择启动配置。

没借什么。一行代码都没有借：仓库里没有代码，许可证也只授权 UZH 批准的研究者学术非商业使用，并限制再分发。接口层同样不能照抄：它走 SBUS、自家控制板与 RotorS，不走 PX4 或 mavros。

锚点与证据。`50bba456156f6f438bea212c643b02e162442fa7`；细节见 `open-source-survey.md` §2。

一句话总结：agilicious 只借 guard、求解器选型与内环演进思路；代码和接口都没有借用。

4.2.3 Fast-Drone-XI35 —— 同硬件的整机参考

`Longer95479/Fast-Drone-XI35` 是哈工大 HIT-K1928-LAB 的自主飞行栈，钉在 `8c80ddb`（2026-02-02，分支 `board`）。它与 z0 的硬件高度重叠：XI35 机架、NxtPX4v2/Nxt-FC 飞控、Jetson Orin NX、6S 电池；控制器是 PID，框架是 ROS 1。

借了什么。主要是 `px4ctrl` 的状态机：五个状态 `MANUAL_CTRL`、`AUTO_HOVER`、`CMD_CTRL`、`AUTO_TAKEOFF`、`AUTO_LAND` 的划分，以及消息超时、电池电压、遥控打杆接管、offboard 掉线这些实飞判据。z0 的 `traj` 节点从 `WAIT_START` 进 `TRACK` 时接收一次开始边沿，也来自这里。

改了什么。没有搬运 C++，而是在 `fsm.hpp` 里用 ROS 2 重写，并把 XI35 扁平枚举里的两个维度拆开：控制权（`MANUAL` / `AUTO`）与任务态（`INIT` → `STANDBY` → `TAKEOFF` → `HOVER` → `WAIT_START` → `TRACK` → `LAND`）。状态名与判据沿用，结构是本项目自己的。

没借什么。PID 控制器、`ctrl_param_fpv.yaml` 的参数值、视觉与规划链、ROS 1 构建体系都没有采用。推力模型只在 XI35 的参数文件注释里出现，`controller.cpp` 实际用的是线性映射加电压补偿；本项目也没有实现它。动捕插值对齐同样只是读过，没有落地。

许可证边界。`px4ctrl` 目录是 GPL-3.0；本项目没有把它的源文件或构建依赖引入仓库，只按判据重写。

一句话总结：XI35 参考了其 FSM（状态划分与实飞判据）；PID、参数与推力模型都没有落地。

4.2.4 OmniNxt —— 同系列硬件的平台佐证

`HKUST-Aerial-Robotics/OmniNxt` 是港科大 Aerial Robotics Group 的全向视觉平台，对应论文 `Liu2024`，仓库钉在 `79ce1d4`（2025-03-09）。它只有 4 个文件（README、许可证、论文 PDF 与一张总览图），README 指向的控制与规划仓库都已 404，参数文件也取不到。

借了什么。只借硬件事实：它用的飞控是 **Nxt-FC**，机载电脑是 **NVIDIA Jetson Orin NX**，与 z0 的飞控同系列、机载电脑同款。论文里的多鱼眼相机与全向感知算法本项目不用。

没借什么。没有代码或参数可借；也不能把它的预编译固件刷到 z0 上：Nxt-FC 仓库里的固件是给 `nxt-v1` 板的，z0 是双 BMI088 + SPL06 的 NxtPX4v2。

一处更正。旧索引把 OmniNxt 写成 “全向倾转旋翼、带舵机”。PDF 全文里 `tilt`、`tilting`、`servo` 各 0 次，构型是常规四旋翼；“omnidirectional” 指全向视觉。

一句话总结：OmniNxt 只用于佐证 Nxt-FC 与 Orin NX 这条硬件线；参考其硬件（无人机动力套件）选项。

4.2.5 Sim2real-UAV-Params —— 参数与动捕链路的资料库

`Huan-qun/Sim2real-UAV-Params` 是同一套配件的动捕实飞资料库，钉在 `b7aeebb`（2026-08-30）。它提供参数、文档与一个可用的 VRPN 监听器，不是控制算法；仓库没有许可证文件。

借了什么。`uav1.params` 作为 z0 参数文件的基线，归档为 `docs/assets/z0.params` 后逐项对账；`mocap.md` 整理成 `mocap-setup.md`；`BOM.md` 的物料信息整理进 `hardware.md`；`vrpn_listener.zip` 原样 vendored 到 `src/vrpn_listener/`，9 个文件与本仓库逐文件 sha256 一致，一行未改。它的 `stamp = now()` 是刻意设计：动捕主机与机载电脑约有 4 秒时钟差，用 `msg_time` 会毁掉 EKF2 融合。

没借什么。上游物理参数表标注 “待更新”，其 0.3 kg 是另一台更轻的机器，只能作量级参考；Genesis + `rsl_rl` 的强化学习路线也没有采用。

边界。 `uav1.params` 与 z0 是 “近似同构”，不是同一台机器的实测快照。

一句话总结：Sim2real 提供参数基线、动捕手册与 VRPN 监听器；参数表和强化学习路线未采用。

4.2.6 MPC_uav —— 本报告描述的仓库本身

`weisama/MPC_uav` 是**本项目自己的仓库**，不是外部参考项目。把它列进本章，是为了说明 “哪些东西来自上游、哪些是本项目新增”，而不是把它包装成第三方参考实现。

本章描述的实现状态以 Phase 4 收口提交 **`21eb4a7`**（2026-10-17）为准。四个版本分支分别钉在阶段性落点：`v1.0` → `5ff9709`（Phase 1 收口）、`v2.0` → `175e298`（Phase 2 票面全部 `done`，阶段 spec 未收口）、`v3.0` → `cdddeb5`（Phase 3 收口）、`v4.0` → `f137472`（Phase 4 之后的在制品快照）。

从上游借来的部分。求解器来自 acados；10 状态模型、OCP 与节点骨架来自 `px4-mpc`；状态划分与实飞判据来自 XI35；参数基线、动捕文档与监听器来自 Sim2real；飞控板级事实来自 Nxt-FC 与厂商固件。

属于本项目的部分。七个 ROS 2 包的组织与实现；`control` 把控制权与任务态拆开；`nmpc` 走 mavros 下行；`bench` 的假 PX4 夹具与 SITL 同构机体；三条解析轨迹；报告的数据管线、图与结果表。包与节点见第五、六章，报告管线见第九、十章。

边界声明。 `model.py` 保留 `px4-mpc` 的 BSD 版权头，`vrpn_listener` 的字节与上游 zip 一致；其余重写部分在代码注释里标注出处。飞行结果只属于 z0。

一句话总结：MPC_uav 是本项目自己的仓库；上游内容与原创部分在此划清。

4.2.7 px4-mpc —— 10 状态 OCP 的直接蓝本

`DISCOWER/px4-mpc` 是 KTH / ETH ATMOS 团队把 acados 与 PX4 连起来的示例，钉在 `3e541af`（2026-06-15），BSD 3-Clause。它是为 x500 写的，价值在于 “怎么把模型、OCP 与节点接起来”。

借了什么。 10 状态模型：状态取位置、速度、四元数共 10 维，控制量取总推力与机体三轴角速率共 4 维；角速率不作状态，等价于假设内环完美跟踪。世界系 ENU、机体系 FLU，模型方程与约定可以原样复用。求解器配置与节点骨架也来自它。输出侧的符号约定以 `sign-map.md` 为唯一权威。

改了什么。参数、上限、代价与载体都按 z0 改过：质量与悬停推力换成 z0 的值；删掉硬编码的 `0.07` 推力标度；角速率上限换成 3.84 / 3.84 / 6.28 rad/s；推力上限按电池取 11.8 N；求解失败与 NaN 分级兜底；控制代价改罚 `u − u_ref`；状态源改成 `/mocap/pose_enu`；删掉 NED→ENU 转换与 `_v1` 双订阅；下游从 `/fmu/in/vehicle_rates_setpoint` 改成 mavros 的 `/mavros/setpoint_raw/attitude`。

没借什么。 x500 的参数、`0.07` 魔数、占位的 `max_rate = 0.5`、`MPC_HORIZON = 15` 死变量都没有沿用。截至 2026-10-17，上游 23 条远端分支里没有 `VehicleAttitudeSetpoint` 的参考实现，四旋翼分支都发角速率设定点；要做 (B) 级姿态回退，是全新工作。

一句话总结：px4-mpc 是本项目的 MPC 动力学代码框架的主要参考，参考了 10 状态模型、求解器配置与节点骨架；参数与下行载体按 z0 改过。

4.2.8 Nxt-FC 与 z0 固件 —— 飞控硬件与固件身份

`HKUST-Aerial-Robotics/Nxt-FC` 是港科大 Aerial Robotics Group 的飞控仓库，钉在 `153b7d1`（2024-08-13），GPL-3.0。它包含预编译固件、README 与两个未初始化的 submodule 指针；硬件设计在 `NxtPX4-Hardware` 里，**它本身不是飞控固件的源码仓库。**

借了什么。借的是板级事实：板级目标名 `hkust_nxt-dual`；`rc.board_defaults` 的 18 条默认值中，15 条与 `z0.params` 相同，3 条被改过（`BAT1_N_CELLS`、`BAT1_V_DIV`、`RC_INPUT_PROTO`）；电池电流通道在厂商加入这块板时就已经是 `ADC12_CH(8)`，不需要打补丁。

没借什么。不刷它的预编译固件：那是给 `nxt-v1` 板的（启动脚本启用 BMI088、ICM42688P 与 BMP388），而 z0 是 NxtPX4v2。也不把 submodule 指针当成 z0 的固件身份：gitlink 指向 `Peize-Liu/PX4-Autopilot` @ `5ba2e91f`，z0 不跑这支 fork。

z0 固件的真身。飞控自报 `flight_sw_version = 0x010e0300`,即 `1.14.3`;`flight_custom_version` 的 8 字节还原出 `4a0e65f233`，对应 `Minderring/PX4-Autopilot` @ `4a0e65f2`（2024-11-18），是一份 v1.14.3 系的 MicoAir 厂商构建。其 `msg/` 与上游 v1.14.3 逐字节相同。2026-10-09 操作者已裁定不重刷固件，本章只作事实存档。

一句话总结：Nxt-FC 本项目飞控的开源仓库

4.3 论文

`docs/assets/papers/` 里有 6 篇 PDF。英文原题按 PDF 首页保留，中文译名放在括号里；引用的数字只写 “论文报告了什么”。

4.3.1 `Sun2022_TRO_NMPC_vs_DFBC.pdf`

原题： A Comparative Study of Nonlinear MPC and Differential-Flatness-based Control for Quadrotor Agile Flight（中文译名：《非线性 MPC 与基于微分平坦的控制用于四旋翼敏捷飞行的对比研究》）

出处与实验室： IEEE Transactions on Robotics (T-RO), 2022；苏黎世大学机器人与感知组（UZH-RPG），与 4.2.2 的 agilicious 同一研究组。

链接： DOI: 10.1109/TRO.2022.3177279 ·arXiv:2109.01365

内容。在同一台四旋翼上对比非线性 MPC 与微分平坦控制，速度到 20 m/s；两者加上 INDI 内环与气动阻力模型后，论文报告跟踪误差降低 78% 以上。

用途与对应关系。这是 `(C) → (C)+INDI` 路线的直接依据。它没有单独的开源仓库；同组工作见 agilicious（4.2.2）。本项目没有复现 INDI，也没有在 z0 上验证这个数字。

一句话总结：Sun2022 对比论证 MPC 加 INDI 内环的收益，是本项目控制器内（PID 角度环）/外环（MPC）的选型依据，对应同组 agilicious。

4.3.2 `Foehn2022_SciRob_agilicious.pdf`

原题： Agilicious: Open-Source and Open-Hardware Agile Quadrotor for Vision-Based Flight（中文译名：《Agilicious：面向视觉飞行的开源开放硬件敏捷四旋翼》）

出处与实验室： Science Robotics, Vol. 7, Issue 67, 2022；UZH-RPG。

链接： DOI: 10.1126/scirobotics.abl6259 ·arXiv:2307.06100

内容。一套软硬件协同设计的敏捷四旋翼平台，MPC 输出总推力与机体三轴角速率（CTBR），内环用 INDI；论文报告 5 g 加速度、70 km/h 的轨迹跟踪，并描述了 guard 模块。

用途与对应关系。它直接对应 4.2.2 的 agilicious 仓库，是 (C) 级控制与 acados + HPIPM 选型的理论来源。仓库没有可借用的代码，本项目只借分层与控制层级的判断。

一句话总结：Foehn2022 是 agilicious 平台对应的论文，为 (C) 级控制与 acados 选型提供了理论依据。

4.3.3 `Torrente2021_RAL_DataDrivenMPC.pdf`

原题： Data-Driven MPC for Quadrotors（中文译名：《面向四旋翼的数据驱动 MPC》）

出处与实验室： IEEE Robotics and Automation Letters (RA-L), 2021；UZH-RPG。

链接： DOI: 10.1109/LRA.2021.3061307 ·arXiv:2102.05773

内容。用高斯过程学习气动效应并嵌入 MPC，论文报告在 14 m/s、加速度超过 4 g 时，高速跟踪误差最多降低 70%。

用途与对应关系。这是第二阶段气动补偿的参考；与 agilicious（4.2.2）同组，但没有单独的开源仓库。本项目当前没有气动项，70% 不是 z0 的读数。

一句话总结：Torrente2021 给出用高斯过程补偿气动效应的路线；与 agilicious 同组，本项目暂未参考。

4.3.4 `Houska2011_Automatica_RTI.pdf`

原题： An auto-generated real-time iteration algorithm for nonlinear MPC in the microsecond range（中文译名：《一种把非线性 MPC 压到微秒级的自动生成实时迭代算法》）

出处与实验室： Automatica, 47 (2011) 2279–2285；K.U. Leuven（Diehl 组）。

链接： DOI: 10.1016/j.automatica.2011.08.020

内容。提出实时迭代（RTI）：把 SQP 的准备放在两次采样之间，采样时刻只做一次廉价反馈迭代，使非线性 MPC 的单周期计算压到微秒级。

用途与对应关系。它是 acados（4.2.1）里 `SQP_RTI` 求解器的理论出处，也解释了机载 100 Hz 求解为什么可行。算法由 acados 实现，本项目没有自己写 RTI。

一句话总结：Houska2011 提出的实时迭代算法，是 acados 中 `SQP_RTI` 求解器的直接理论出处。

4.3.5 Houska2011_OCAM_ACADOToolkit.pdf

原题： ACADO toolkit—An open-source framework for automatic control and dynamic optimization（中文译名：《ACADO Toolkit：一个用于自动控制与动态优化的开源框架》）

出处与实验室： Optimal Control Applications and Methods (OCAM), 32:298–312, 2011；K.U. Leuven（Diehl 组）。

链接： DOI: 10.1002/oca.939

内容。介绍 ACADO Toolkit：一个 C++ 的最优控制与动态优化框架，覆盖直接最优控制（含 MPC）以及状态与参数估计，重点讲软件设计与符号表达式处理。

用途与对应关系。 ACADO Toolkit 是 acados（4.2.1）的前身；读它能理解 acados 为什么强调符号建模与代码生成。本项目没有使用 ACADO Toolkit 的代码。

一句话总结：Houska2011 介绍的是 acados 的前身 ACADO Toolkit；本项目只借它来理解设计取舍。

4.3.6 Liu2024_IROS_OmniNxt.pdf

原题： OmniNxt: A Fully Open-source and Compact Aerial Robot with Omnidirectional Visual Perception（中文译名：《OmniNxt：一款全开源、紧凑、带全向视觉感知的空中机器人》）

出处与实验室： IEEE/RSJ IROS 2024；港科大 Aerial Robotics Group。

链接： DOI: 10.1109/IROS58592.2024.10802134 ·arXiv:2403.20085

内容。面向全向视野的空中机器人平台：多鱼眼相机组、飞控 Nxt-FC、Jetson Orin NX，软件提供 Omni-VINS 与 Omni-Depth。

用途与对应关系。它对应 4.2.4 的 OmniNxt 仓库，也佐证 4.2.8 的 Nxt-FC 硬件线。它的感知与规划面向视觉，本项目不采用；构型是常规四旋翼，不是旧索引写的倾转旋翼。

一句话总结：Liu2024 是 OmniNxt （港科大 Aerial Robotics Group）对应的论文，也佐证了 Nxt-FC 与 Orin NX 这条硬件线有工程先例。

4.4 移植边界与原创性

4.4.1 已落地的复用与改写

上游	落点	方式	本项目改动
`acados` @ `00947ddc`	`~/acados`	外部依赖	机载电脑适配：BLASFEO A76、HPIPM GENERIC、不走 CUDA
`px4-mpc` @ `3e541af` 的模型与 OCP	`src/nmpc/nmpc/`	复制后改写	参数、上限、代价、兜底、状态源、下行载体
`px4-mpc` 的节点骨架	`src/nmpc/nmpc/node_core.py`	结构参考后重写	ROS 2 节点、mavros 下行、健康度与兜底
XI35 `px4ctrl` 的状态机	`src/control/include/control/fsm.hpp`	按判据重写	控制权与任务态显式拆开
Sim2real `vrpn_listener.zip`	`src/vrpn_listener/`	原样 vendored	无；新增 `VENDORED.md` 说明来源
Sim2real `uav1.params` 与文档	`docs/assets/z0.params`、`docs/reference/`	基线后整理	按 z0 板级默认值与日志改值

真正逐字节进入仓库的上游代码只有 `vrpn_listener`；`px4-mpc` 是改写，XI35 是重写，acados 始终是外部依赖。

一句话总结：真正进入仓库的上游代码只有 VRPN 监听器；px4-mpc 是改写，XI35 是重写。

4.4.2 只有概念、没有代码

- **agilicious 的 guard 与 INDI**：只有架构思路；z0 的安全网是 PX4 的 EKF2、遥控接管与 `AUTO.LAND`，内环仍是 PX4 角速率环。
- **XI35 的推力模型与动捕插值**：读过、记录过，当前代码没有实现。
- **Nxt-FC 的预编译固件**：没有刷；z0 跑的是 Minderring 的 v1.14.3 厂商构建。
- **论文里的方法**：只引用结论，不复现实验；78%、70%、14 m/s、4 g 都是论文读数，不是 z0 读数。

报告里写 “参考了某项目”，不等于 “某项目替我们飞了”；飞行结果、参数现值与失效判据都来自 z0 自己的数据与代码。

一句话总结：guard、INDI、XI35 推力模型与固件都没有落地；论文只提供参考结论。

4.4.3 本项目的原创部分

- **ROS 2 包组织与实现**：七个包的分工、节点边界、话题与参数；`control` 把控制权与任务态拆成两个维度。
- **mavros 下行**：MPC 节点走 `/mavros/setpoint_raw/attitude`，把总推力与机体角速率交给 PX4。
- **轨迹与台架**：悬停、正方形、8 字三条解析轨迹；假 PX4 夹具、脚本化动捕、故障注入与同构 SITL 机体。
- **数据与报告**：四源对齐的数据管线、图与结果表、文档与票面记录。

控制与节点部分以 Phase 4 收口提交 `21eb4a7` 为代码锚点，见第五、六章；报告管线属于 Phase 5，见第九、十章。

一句话总结：包组织、mavros 下行、轨迹、台架与报告管线是本项目原创。

4.5 资料来源与复现

本章的依据集中在五份文档与一类目录：

- `docs/reference/refs-manifest.md`：仓库清单、钉住的 commit、z0 固件身份。**版本以它为准。**
- `docs/reference/open-source-survey.md`：项目级调查与源码事实。
- `docs/reference/code-inventory.md`：逐文件的 “可复用 / 参考 / 不用” 取舍。
- `docs/assets/papers/README.md`：6 篇论文的中文标注；标题与译名以 PDF 首页复核为准。
- `src/vrpn_listener/VENDORED.md`：vendored 监听器的来源、md5、逐文件 sha256 与复核命令。
- `~/refs/`：参考仓库的本地只读副本，在仓库之外、不进 git。

复核命令：

```
# 任一参考仓库的钉住 commit
git -C ~/refs/px4-mpc log -1 --format=%H
git -C ~/refs/Nxt-FC  log -1 --format=%H

# acados 的版本串
git -C ~/refs/acados describe --tags

# 论文标题与摘要（第一手全文抽取）
pdftotext docs/assets/papers/Sun2022_TRO_NMPC_vs_DFBC.pdf -
```

使用纪律与 `refs-manifest.md` §3 一致：只读、不 `git pull`、需要新版时另建目录并登记。例外是 acados：上游在 cmake configure 时会往自己的 `lib/` 写文件，所以构建走 out-of-source，产物落在 `~/acados-build*`。

一句话总结：来源、版本与复现命令集中在五份文档与只读参考副本。

第 4 章引用清单

主题	文件
参考仓库清单与钉住的 commit	`docs/reference/refs-manifest.md`
开源项目逐项调查	`docs/reference/open-source-survey.md`
逐文件取舍	`docs/reference/code-inventory.md`
6 篇论文的中文标注	`docs/assets/papers/README.md`
原仓库 README（归档）	`docs/assets/MPC_uav-README.md`
固件身份与 `px4_msgs` 选版	`refs-manifest.md §2.5(d)、§2.6`
acados 在机载电脑上的构建	`docs/control/acados-orin-build.md`
px4-mpc 的逐行解读与改动	`docs/control/phase2-implementation.md`
输出侧符号映射的唯一权威	`docs/control/sign-map.md`
状态机实现与出处注释	`src/control/include/control/fsm.hpp`
模型实现与 BSD 版权头	`src/nmpc/nmpc/model.py`
vendored 监听器的来源与字节核对	`src/vrpn_listener/VENDORED.md`

一句话总结：本章每条来源都能回到仓库内文档、源码或钉住的只读副本。

第 5 章 系统架构与实验平台

本章回答两个问题：z0 的自主飞行栈分成哪几层，层与层之间用什么接口连接；哪些话题、坐标系与安全边界构成了两条控制链——offboard 位置控制与 MPC。第 3 章讲零件与物理参数，第 6 章讲代码结构，第 7 章讲起链、遥控与状态机；本章只讲它们之间的连接关系与数据流，不重复各章的细节。

按操作者裁定，本章不画架构图；结构用文字与表描述，话题表是本章的主干。

5.1 符号与缩写

表 2 列出本章反复出现的缩写与符号。物理量的完整符号表在第 3 章；本章只补充与接口、频率、坐标系有关的几个。

表 2　本章缩写与符号。

缩写/符号	含义
ENU / NED	East-North-Up / North-East-Down，分别为 ROS 侧与 PX4 侧的世界系
FLU / FRD	Forward-Left-Up / Forward-Right-Down，分别为 ROS 侧与 PX4 侧的机体系
VRPN	Virtual Reality Peripheral Network，动捕数据的传输协议
EKF2	PX4 的扩展卡尔曼滤波器，融合 IMU、气压计与外部视觉
CTBR	collective thrust + body rate，总推力 + 机体三轴角速率
OCP	optimal control problem，最优控制问题
SQP_RTI	sequential quadratic programming，real-time iteration
HPIPM	acados 使用的高性能内点法 QP 求解器
DShot	数字电调协议
CRSF	Crossfire，遥控接收机与飞控之间的串行协议
QGC	QGroundControl，地面站
VP / LP	本章对 `/mavros/vision_pose/pose` 与 `/mavros/local_position/pose` 的简称
f	话题频率，单位 Hz；表中所有频率都带测量时点
t_0	时间零点，取 `/traj/start` 的包记录时间戳
τ	端到端延迟，单位 s；本报告只在第 3、9 章给数值
T	总推力，单位 N
$\omega = (\omega_x, \omega_y, \omega_z)$	机体三轴角速率，单位 rad/s

本章统一用 ENU/FLU 指 ROS 侧，用 NED/FRD 指 PX4 侧；出现 “机体角速率” 时，默认是 FLU 口径，除非明说 PX4 侧。

一句话总结：本章统一用 ENU/FLU 指 ROS 侧、NED/FRD 指 PX4 侧，VP 与 LP 是两条位姿话题的简称。

5.2 三层架构

z0 的飞行栈按**失效域**分成三层：定位层在地面，机载层与飞控层都在飞机上。表 3 给出每层的组成、输入输出，以及这一层失效时最先失去什么。

表 3　三层架构。

层	主要组成	输入	输出	失效时先失去什么
定位层	Nokov 光学动捕相机、Nokov 软件、VRPN 服务（`10.1.1.198:3883`）	机体上 5 颗反光球的图像	刚体位姿与速度（VRPN）	两条动捕路径同时失去状态；飞控失去视觉位置；只能遥控或降落
机载层	Jetson Orin NX 8GB；ROS 2 Humble；`vrpn_listener`、`mocap`、`traj`、`control`、`nmpc`、`mavros`	VRPN/UDP；`/mavros/*` 的飞控状态与遥控通道	setpoint（位置，或推力 + 角速率）、`/control/*`、录制包	控制链停止发帧；PX4 在 1 s 后退出 Offboard；遥控仍然可用
飞控层	NxtPX4v2；PX4 1.14.3 厂商构建；EKF2、位置/姿态/角速率环、控制分配、失效保护、DShot	MAVLink setpoint、遥控、IMU 与气压计	四路电机指令	飞机失去全部控制；最后手段是 ch8 杀车或 PX4 自动降落

三层是按失效后果切的，不是按机箱切的。飞控与机载电脑都装在飞机上，动捕主机在地面；把它们并列成三层，是因为它们的失效后果不同：定位层失效，飞机失去外部位置；机载层失效，飞机失去任务指令；飞控层失效，飞机失去全部控制。

层间只有两条物理连接：动捕主机到机载电脑的 WiFi，机载电脑到飞控的串口。机载层没有任何一条线能直接驱动电调；飞控层也不运行我们的控制律。这个边界是后面所有安全论证的前提。

机载层内部，`control` 是任务总管，`traj` 只发参考，`mpc_node` 只解优化问题，`mavros` 是唯一与飞控说话的进程。第 6 章按包展开这些职责，本章只按角色使用这些名字。

一句话总结：定位层给位姿，机载层做任务与计算，飞控层保留角速率环、混控与失效保护。层级之间的输入输出标准参考成熟开源项目。

5.3 硬件连接与软件栈

5.3.1 硬件连接

表 4 给出本章架构涉及的物理连接。零件型号与物理规格见第 3 章，这里只列 “谁连谁、走什么协议、哪个参数锁住它”。

表 4　硬件连接。

连接	近端	远端	物理/协议	关键参数	证据
动捕 ↔ 机载电脑	Nokov 主机 `10.1.1.198`	机载电脑 WiFi `10.1.1.114/24`（2026-09-21 读数）	VRPN over UDP，端口 3883	同一网段；不需要额外网卡	mocap-setup.md §9
机载电脑 ↔ 飞控	`/dev/ttyTHS1` @ 921600	飞控 TELEM2 / Onboard	MAVLink 串口	`MAV_1_CONFIG=102`、`SER_TEL2_BAUD=921600`、mavros 独占	mocap-setup.md §5、params-changes.md §E1
飞控 ↔ 电调	4 路 DShot	33A 四合一电调	DShot	`DSHOT_MIN=0.055`、`DSHOT_3D_ENABLE=0`、`DSHOT_TEL_CFG=0`（遥测关）	`docs/assets/z0.params`
遥控 ↔ 飞控	RadioMaster Pocket（ELRS 2.4 GHz、EdgeTX）	飞控遥控输入	CRSF	`RC_INPUT_PROTO=6`；ch5 解锁、ch6 模式、ch8 杀车、ch9 开始	params-changes.md §H
机载电脑 ↔ QGC（可选）	mavros `gcs_url`	地面站	UDP 转发 MAVLink	只在需要 QGC 时启用；不参与控制	mocap-setup.md §8

5.3.2 软件栈

表 5 列出两条链上实际运行的软件与版本。版本以本报告证据产出时的实际值为准；没有一手记录的项标 “未记录”，不按默认值补。

表 5　软件栈。

组件	版本/标识	跑在哪	角色
Ubuntu / L4T	22.04.5 LTS / R36.5.0 / 内核 5.15.185-tegra	机载电脑	宿主系统
ROS 2	Humble	机载电脑	节点间通信
mavros	2.15.1 @ `22ae5b7`，源码构建在 `~/mavros_ws`	机载电脑	唯一的 MAVLink 桥
acados	`v0.6.0-41-g00947ddc3`，SQP_RTI + HPIPM	机载电脑，aarch64 原生	优化问题求解
PX4	1.14.3 系厂商构建 @ `4a0e65f2`	飞控	EKF2、控制环、混控、失效保护
VRPN 客户端	`ros-humble-vrpn` 7.35.0	机载电脑	接收动捕
Nokov 软件	版本未记录；VRPN banner 自报 7.34	动捕主机	VRPN 服务端
Python / rclpy	3.10	机载电脑	`mpc_node.py` 等节点

5.4 话题表

表 6 是本章的主干：两条控制链经过的话题、类型、生产者与消费者、频率、坐标系，以及它何时存在。

表 6　两条控制链的主干话题。

话题	类型	生产者 → 消费者	频率（2026-10-17 六份包实测）	坐标系 / 语义	何时存在
`/vrpn/z0/pose`	`geometry_msgs/PoseStamped`	`vrpn_client_node` → `mocap`	181–193 Hz（n/span）	动捕原生 ENU/FLU；`frame_id="world"`	VRPN 数据到达后
`/traj/select`	`std_msgs/UInt8`	`fly.py` / 手动 → `traj`	事件（每次 1 帧）	无量纲，1/2/3/4；0 = 清空	选轨迹时
`/traj/start`	`std_msgs/Empty`	`control` → `traj`	事件（每趟 1 帧）	无载荷；包记录时间戳定义 t_0	进 TRACK 时
`/traj/reference`	`mavros_msgs/PositionTarget`	`traj` → `control`	50 Hz（中位 49.9–50.1；最大缺口 44–154 ms）	ROS ENU，相对轨迹原点；`frame_id="traj_origin"` 仅标注；`type_mask=2496`	select 且 start 后
`/traj/progress`	`std_msgs/Float32`	`traj` → `control`	50 Hz（与参考同帧）	无量纲 0–1	同上
`/control/state`	`std_msgs/String`	`control` → `fly.py` / `z0.py` / 录制	1 Hz 心跳；整包均值 1.05–1.40 Hz	`"< 控制权 >/< 任务态 >"`，如 `AUTO/TRACK`	节点启动后
`/control/target`	`mavros_msgs/PositionTarget`	`mpc_node` → `mavros`	活跃期 50 Hz；整包 39.0–45.9 Hz（LAND 停发 7.2–7.9 s）	ROS ENU 绝对量；`type_mask=2496`	仅 `output_mode=external`
`/mavros/setpoint_raw/local`	`mavros_msgs/PositionTarget`	`mavros` → `mavros`	活跃期 50 Hz；整包 40.9–50.0 Hz（LAND 停发 6.6–7.1 s）	ROS ENU；`type_mask=2552`（位置 + yaw）；mavros 转 NED	仅 offboard 档
`/mavros/setpoint_raw/attitude`	`mavros_msgs/AttitudeTarget`	`mpc_node` → `mavros`	98.8–99.8 Hz（n/span）	`body_rate` 为 FLU；`thrust` 为正号 0..1；`type_mask=128`	仅 MPC 档

话题	类型	生产者 → 消费者	频率（2026-10-17 六份包实测）	坐标系 / 语义	何时存在
/mavros/vision_pose/pose	geometry_msgs/PoseStamped	mocap → mavros	181–193 Hz（n/span）	ROS ENU/FLU；mavros 转 NED/FRD	mocap 启动后
/mocap/pose_enu	geometry_msgs/PoseStamped	mocap → mpc_node / 录制	181–193 Hz（n/span）	动捕原生 ENU/FLU，经外参、原点与 yaw 处理	同上
/mocap/velocity_enu	geometry_msgs/TwistStamped	mocap → mpc_node / 录制	181–193 Hz（n/span）	ENU；角速度分量实测恒 0	同上
/mavros/local_position/pose	geometry_msgs/PoseStamped	mavros → control	29.9–30.1 Hz	ROS ENU（EKF2 输出）	mavros 连上后
/mavros/local_position/velocity_local	geometry_msgs/TwistStamped	mavros → control	29.9–30.0 Hz	ROS ENU	同上
/mavros/state	mavros_msgs/State	mavros → control / mocap / mpc_node / z0.py	1.00 Hz	connected / armed / mode	同上
/mavros/rc/in	mavros_msgs/RCIn	mavros → control	19.95–20.04 Hz	原始 PWM 读数（μs）；归一化在 control 侧（low_us=1000 / high_us=2000）；ch5 / ch6 / ch8 / ch9	同上

5.5 两条数据链

两条链共用同一个任务总管、同一条轨迹源和同一套起飞/降落流程，差别只在“最后谁给 PX4 发指令、发到哪一层”。表 7 先列共同点，表 8 与表 9 按同一组项目分别展开两档。

5.5.1 共同点

表 7　两条链的共同点。

项目	两档共同的做法
任务总管	control 负责选轨迹、解锁、切模式、状态机、轨迹原点与降落；mpc_node 不 arm、不 set_mode。
轨迹源	traj 发布 /traj/reference（50 Hz，相对轨迹原点）与 /traj/progress；control 在进入 TRACK 时发 /traj/start。
操作流程	ch6 选模式；第 1 次按 ch9 解锁起飞；判稳后第 2 次按 ch9，**松手**才进 TRACK；飞完自动降落。
OFFBOARD 前置	切模式前 setpoint 已持续预流 ⩾1 s；预流频率 ⩾2 Hz（offboard 50 Hz，MPC 100 Hz）。
轨迹原点	在 AUTO/WAIT_START 判稳时锁存，取当时指令的恒定值；不取 /traj/start 时刻的飞机位置。
下行链路	只有 mavros 一条；不使用 uXRCE-DDS；MAV_FWDEXTSP=1 是硬前置。
结束与失效	任务完成时 control 停发并请求 AUTO.LAND；输入类失效（动捕、EKF 位置、mavros 状态超时等）同样停发，已解锁时请求 AUTO.LAND；打杆接管或模式被夺则只停发、交还控制权。
互斥	同一时刻只有一条链写 setpoint：external 档下 control 不写 /mavros/setpoint_raw/local；offboard 档下不启动 mpc_node。

一句话总结：两档共用任务总管、轨迹源、起飞降落流程与安全边界，只有最后一级的执行者与接口不同。

5.5.2 区别：两档各一张表

表 8　offboard 档（位置 + yaw）。

项目	offboard 档
谁发指令	`control`
下行话题	`/mavros/setpoint_raw/local`（`mavros_msgs/PositionTarget`）
控制量	位置 (x, y, z) + yaw；速度字段被忽略
`type_mask`	2552（位置 + yaw）
指令频率	活跃期 50 Hz（2026-10-17 实测）；LAND 停发 6.6–7.1 s 拉低整包均值
参考	`/traj/reference`（相对轨迹原点）+ 轨迹原点，绝对 ENU
状态反馈	`/mavros/local_position/pose`、`/mavros/local_position/velocity_local`（EKF2，29.9–30.1 Hz）
PX4 承担	位置环 → 姿态环 → 角速率环 → 控制分配 → DShot
预流	`control` 从 STANDBY 起以 50 Hz 发当前位置与当前机头
停止条件	`/traj/progress` 首次 ⩾1.0；输入类失效（已解锁时请求 `AUTO.LAND`）；打杆接管或模式被夺（只停发）

表 9　MPC 档（总推力 + 机体三轴角速率）。

项目	MPC 档
谁发指令	`mpc_node`
下行话题	`/mavros/setpoint_raw/attitude`（`mavros_msgs/AttitudeTarget`）
控制量	总推力 T + 机体三轴角速率 ω（CTBR）
`type_mask`	128（忽略姿态四元数）
指令频率	求解 100 Hz；2026-10-17 实测 98.8–99.8 Hz
参考	`/control/target`（绝对 ENU 位置、速度与 yaw，`type_mask=2496`，50 Hz）
状态反馈	`/mocap/pose_enu`、`/mocap/velocity_enu`（原始动捕，181–193 Hz）
PX4 承担	角速率环（1 kHz）→ 控制分配 → DShot；位置环与姿态环由控制器承担
预流	`mpc_node` 以 `offboard_prestream=true` 发纯悬停帧；`control` 仍负责 arm 与 set_mode
停止条件	`/control/target` 断流 0.5 s → 停流；PX4 在 `COM_OF_LOSS_T=1.0` s 后退出 Offboard；求解器不可用时走 hold→hover 阶梯

一句话总结：offboard 档交位置与 yaw，MPC 档交总推力与机体三轴角速率，PX4 保留的环路因此不同。

5.6 坐标系与转换发生在哪一侧

表 10 只回答“转换由谁做、往哪个方向做”。ENU↔NED 与 FLU↔FRD 的公式、四元数写法以及模型内部的推导，见第 8 章。

表 10　坐标系与转换边界。

量	ROS 侧（本仓节点之间）	mavros 边界	PX4 侧	谁做转换
位置、速度	ENU	`vision_pose` 入：ENU→NED；`local_position` 出：NED→ENU；`setpoint_raw/local` 入：ENU→NED	NED	mavros
姿态四元数	ENU/FLU	`vision_pose` 入：ENU→NED + FLU→FRD；`local_position` 出：NED→ENU + FRD→FLU	NED/FRD	mavros
机体角速率	FLU	`setpoint_raw/attitude` 入：FLU→FRD	FRD	mavros

量	ROS 侧（本仓节点之间）	mavros 边界	PX4 侧	谁做转换
总推力	标量，正号 0..1	不转换	`thrust_body_z` 为负	PX4 `fill_thrust`
模型内部状态	ENU/FLU	—	—	无（`pose_to_state` 不做转换）

一句话总结：ROS 侧统一 ENU/FLU，ENU↔NED 与 FLU↔FRD 只在 mavros 边界转换一次。

5.7 安全边界：谁能让飞机动

表 11 按“谁能直接让飞机动”列出动作、执行者与接口。状态机的完整转移与失效路径见第 7 章；这里只划责任边界。

表 11　安全边界。

动作	谁执行	接口 / 触发	证据
解锁（程序）	`control`	`/mavros/cmd/arming`；ch9 第 1 次上升沿，或台架 `/control/takeoff`	phase1-implementation.md §4
解锁（硬件开关）	遥控器 → PX4	ch5；`RC_MAP_ARM_SW=5`	`docs/assets/z0.params`
起飞	`control`	状态机 `AUTO/TAKEOFF`；到 1 m 悬停	同上
切 OFFBOARD	`control`	`/mavros/set_mode`；切之前 setpoint 已预流 ⩾1 s	同上
切模式（硬件开关）	遥控器 → PX4	ch6；`RC_MAP_FLTMODE=6`	`docs/assets/z0.params`
切 AUTO.LAND	`control`	`/mavros/set_mode`；任务完成、超时、失效，或在飞时长按 ch9 ⩾1 s	`fsm.hpp`
发 setpoint（offboard）	`control`	`/mavros/setpoint_raw/local`	`run/README.md`
发 setpoint（MPC）	`mpc_node`	`/mavros/setpoint_raw/attitude`	同上
停发（offboard）	`control`	打杆接管、模式被夺、动捕/状态超时、进 LAND	phase1-implementation.md §4
停发（MPC）	`mpc_node`	`/control/target` 断流 0.5 s、状态源超时、离开 OFFBOARD、掉 armed	`node_core.py`
OFFBOARD 丢失	PX4	`COM_OF_LOSS_T=1.0 s`；`COM_OBL_RC_ACT=0` → Position	`docs/assets/z0.params`
打杆接管	PX4	`COM_RC_OVERRIDE=3`	同上
杀车	PX4	ch8；`RC_MAP_KILL_SW=8`	同上
遥控失联	PX4	`NAV_RCL_ACT=1` → Hold（2026-09-28 实测）	params-changes.md
上锁	PX4	`COM_DISARM_LAND=2.0`；`control` 不发上锁	`fsm.hpp`
起链 / 录制	`run/z0.py`	只起停进程与录制；没有 arm / mode / setpoint 入口	`run/README.md`

一句话总结：解锁、切模式与停发由 control、遥控器或 PX4 保护承担；MPC 节点只发指令。

5.8 本章引用清单

表 12 把本章的接口、参数与判定结论按主题归到仓库内的文件。正文里已经随行给过链接，这里只是便于回查。

表 12　本章引用清单。

主题	文件
三层架构、两条链路的解耦理由	docs/PROJECT.md §3
话题与服务、坐标与原点	docs/control/phase1-implementation.md §2–3
mavros 转换、下行定案、MAV_FWDEXTSP	docs/reference/px4-interface.md §4.3、§5.1
动捕链路、VRPN、串口与网段	docs/reference/mocap-setup.md
起链、两档、run/z0.py 的安全边界	run/README.md
MPC 下行与闸门	src/nmpc/nmpc/mpc_node.py、node_core.py
符号映射与推力正负号	src/nmpc/nmpc/sign_map.py、docs/control/sign-map.md
control 的发布接口	src/control/include/control/px4_interface.hpp、px4_interface.cpp
动捕节点的预处理	src/mocap/README.md
轨迹参考的频率与掩码	src/traj/src/traj_node.cpp、src/traj/config/traj.yaml
飞控参数现值	docs/assets/z0.params
数据口径与已知坑	docs/report/data/README.md
表 6 的频率读法与复算脚本	tools/report/topic_frequency.py、topic-frequency-20261017.json
下行切换票	.scratch/phase4/issues/13-mavros-downlink-switch.md
任务总管与 /control/target	.scratch/phase4/issues/15-trajectory-flight-roles.md
术语	CONTEXT.md

第 6 章 项目结构与代码结构

第 5 章讲层与接口，本章讲这些接口落在哪些代码里。全章按「仓库目录—七个包—两条数据流—MPC 的层分界—测试布局—证据目录」展开。目录树与计数都在 2026-10-17 16:18 UTC 从当前工作树现生成；它们描述的是这一刻的仓库，不是长期有效的清单。

6.1 符号、口径与快照

表 13 先固定本章反复出现的几个词。数学符号在本章几乎不出现；物理量符号表在第 3 章，架构缩写表在第 5 章。

表 13　本章缩写与符号。

符号/缩写	含义
colcon 根	运行 `colcon build` 的目录；本仓就是仓库根
包	`src/` 下带 `package.xml` 的目录
入口脚本	由 `install(PROGRAMS)` 装到 `install/< 包 >/lib/< 包 >/` 的可执行文件
零 ROS 核心层	不 import `rclpy` 或任何具体消息包的纯 Python 模块
seam 1 / seam 2	节点级话题边界 / 纯函数边界；定义见第 1 阶段 spec
vendored	原样纳入的上游代码，不改
CTBR	collective thrust + body rate，总推力 + 机体角速率
SITL	software-in-the-loop，软件在环
OCP	optimal control problem，最优控制问题
EKF2	PX4 的扩展卡尔曼滤波器

本节还固定三件事：什么算「包」、什么算「飞行代码」、哪些数字是现算的。**包**按上表取 `package.xml` 的定义。**飞行代码**指会进入飞行链、能影响飞机动作的节点与模块；测试夹具、启动脚本与离线工具不算。这条区分在 6.3 节会反复用到，尤其是 `bench`。

本章引用的目录树和计数都按写稿时的工作树现算，未从 `README.md` 或旧票里抄。表 14 给出快照时点与复算命令；`build/`、`install/`、`log/` 是 colcon 的生成物，不在快照内。

表 14　写稿快照与复算命令。

项	值或命令
快照时点	2026-10-17 16:18 UTC
ROS 2 包数	`find src -maxdepth 2 -name package.xml | wc -l` → 7
目录树	`find . -maxdepth 2 …`（排除 `.git`、`build`、`install`、`log` 与缓存目录）
包内测试文件	`find src -path '*/test/*' -type f \( -name 'test_*.py' -o -name 'test_*.cpp' \) | wc -l` → 26
仓库级测试文件	`find run/tests -maxdepth 1 -name 'test_*.py' | wc -l` → 5
工具测试文件	`find tools -name 'test_*.py' | wc -l` → 23

这些数字只用于说明量级与测试分布；改动仓库后必须按同一命令重算，不能把表里的值当成长期事实。

一句话总结：本章的目录树与计数都取自 2026-10-17 16:18 UTC 的工作树，复算命令随表给出。

6.2 仓库结构

仓库根就是 colcon 根：没有单独的 `src` 工作空间，`colcon build` 直接在仓库根执行，包在 `src/` 下。表 15 给出顶层目录的职责。

表 15　顶层目录的职责。

目录	内容	是否进入飞行链
src/	7 个 ROS 2 包的源码、测试、launch 与配置	其中 5 个包会进入飞行链；bringup 负责起链；bench 只在测试里用
bag/	已录制的飞行数据：场景目录含 rosbag 与 session.json，飞控 .ulg 归档在 fcu/（有则存）；根目录有自动生成的总表 MANIFEST.md，old/ 存归档	否，是证据
docs/	项目总纲、控制与接口文档、参考资料、报告章节与插图	否，是文档与证据
run/	人工起链与录制层：z0.py、rec.py、up.sh、down.sh、tests/、logs/	间接进入：它只起停进程，不发控制指令
tools/	离线测量、参数辨识、SITL 扫描与报告数据管线	否，是离线工具
.scratch/	仓库内的 issue tracker：每阶段的 spec 与票	否，是过程记录

README.md 的目录段落写「6 个包」，但同一棵树下实际列了 7 个目录；本节按 src/*/package.xml 现算的 7 个写。旧目录树只作对照，不直接复制。

下面的树只展开到能看清职责的层级，不逐文件列全；src/ 内部的文件在 6.3 节按包展开。

```
ros_ws/                              ← colcon 根 = 仓库根
├── src/                             ← 7 个 ROS 2 包
│   ├── bench/                       测试夹具（不是飞行代码）
│   ├── bringup/                     启动层与操作台
│   ├── control/                     任务总管与控制节点
│   ├── mocap/                       动捕相对位姿节点
│   ├── nmpc/                        MPC（核心层 + 节点层）
│   ├── traj/                        参考轨迹发生器
│   └── vrpn_listener/               VRPN → ROS 2 桥（vendored）
├── bag/                             录制数据与归档
├── docs/
│   ├── PROJECT.md                   项目总纲
│   ├── agents/                      协作代理的 issue tracker / triage 规则
│   ├── assets/                      参数快照、论文、演练记录
│   ├── control/                     控制实现、台架、安全路径文档
│   ├── reference/                   飞控、动捕、接口参考
│   └── report/                第 5 阶段报告：chapters / data / figures / generated
├── run/                             人工起链与录制层
├── tools/                           离线测量与数据处理
├── .scratch/                        仓库内 issue tracker
├── README.md                        项目入口
├── CONTEXT.md                       术语表与书写约定
├── operation_manual.md              现场一页纸
└── CLAUDE.md                        给协作代理的项目约定
```

bag/、docs/、run/、tools/ 与 src/ 的分离不是按文件类型，而是按**变更节奏与失效后果**：src/ 改一次要重新构建、可能要重新试飞；bag/ 与 docs/assets/ 基本只增不改；run/ 与 tools/ 可以在不碰飞行代码的情况下迭代。把这三类放在不同顶层目录，是为了让「这次改动要不要上飞机」一眼可判。

6.3 七个 ROS 2 包

表 16 先给全景。职责一列只写这个包**唯一负责**的事；入口/运行方式一列写 `ros2 run`、`ros2 launch` 实际解析到的名字，或模块运行方式。

表 16　七个包的职责、入口与主要依赖。

包	形态	职责	入口/运行方式	主要依赖
`bench`	Python（`ament_cmake` + `ament_cmake_python`）	台架夹具：假 mavros、假 PX4（现无 DUT）、脚本化动捕输入、被控对象模型、故障时序	没有 `ros2 run` 入口；由测试进程 import	`rclpy`、`geometry_msgs`、`sensor_msgs`、`std_msgs`、`mavros_msgs`、`px4_msgs`
`bringup`	Python launch + 脚本	启动层：按顺序拉起节点，提供操作台；不含飞行逻辑	`ros2 launch bringup z0.launch.py`、`mavros.launch.py`；`ros2 run bringup fly.py`、`mavros_set_thrust_scaling.py`	`launch`、`launch_ros`、`rclpy`、`std_msgs`、`mavros_msgs`；exec 依赖 `vrpn_listener`、`mocap`、`traj`、`control`
`control`	C++	任务总管：权限与任务两维状态机、Offboard 位置控制、解锁/起飞/跟踪/降落	`ros2 run control control_node`；launch `control.launch.py`	`rclcpp`、`mavros_msgs`、`geometry_msgs`、`sensor_msgs`、`std_msgs`、`std_srvs`
`mocap`	C++	VRPN 刚体位姿 → 相对位姿（ENU/FLU），输出 vision pose 与两条控制用话题	`ros2 run mocap mocap_rel_pose_node`；launch `mocap.launch.py`	`rclcpp`、`geometry_msgs`、`std_msgs`、`mavros_msgs`
`nmpc`	Python（调用 acados）	MPC 节点 + 零 ROS 核心层	`ros2 run nmpc mpc_node.py`、`offboard_smoke.py`、`preflight_check.py`；`python3 -m nmpc.timing`（模块运行）	`rclpy`、`geometry_msgs`、`std_msgs`、`nav_msgs`、`visualization_msgs`、`mavros_msgs`、acados 运行时
`traj`	C++	四条解析参考轨迹：悬停、1 m 正方形、8 字、起飞；只发参考	`ros2 run traj traj_node`；launch `traj.launch.py`	`rclcpp`、`geometry_msgs`、`std_msgs`、`mavros_msgs`、`visualization_msgs`
`vrpn_listener`	C++（vendored）	VRPN → ROS 2 桥：自动发现刚体并发布位姿、速度、加速度	`ros2 run vrpn_listener vrpn_listener`；launch `sync_entity_state.launch`	`rclcpp`、`geometry_msgs`、`tf2`、系统 VRPN 库

「谁调用谁」不能只看依赖清单，还要看进程是谁起的：

- `bringup` 的 `z0.launch.py` 按需 include `mavros`，再依次 include `vrpn_listener`（可选）、`mocap`、`traj`、`control`；它不启动 `nmpc`。
- `run/z0.py` 是人工层：起链时按 `UP_CHAIN` 逐条起 `mavros`、`vrpn_listener`、`mocap`、`traj`、`control`；菜单里另有一项可以直接 `ros2 launch bringup z0.launch.py`。MPC 档再另起 `mpc_node.py`，并把 `control` 的 launch 参数设为 `output_mode:=external`。两档的差异在第 7 章展开。
- `bench` 不被任何飞行包依赖。它只被测试 import，并由测试用它起的假 mavros 与假动捕充当「外部世界」。
- 运行期的调用关系是话题，不是函数调用：`vrpn_listener → mocap → control`，`traj → control`，`control → mavros` 或 `control → nmpc → mavros`。

6.3.1 输入与定位：vrpn_listener 与 mocap

vrpn_listener 是 vendored 包：来源、commit 与 zip 校验和钉在 VENDORED.md，逐文件 sha256 与上游一致。它自动发现 VRPN 刚体，为每个刚体发布 /vrpn/< 刚体 >/pose、/twist、/accel 三条话题。三处看似「可以顺手清理」的行为——时间戳用接收时刻 now()、角速度刻意留零、frame_id 走参数——都是上游原有设计，不能改；尤其 now() 是为了避开动捕主机与机载电脑约 4 s 的时钟差，改回 msg_time 会毁掉 EKF2 融合。

mocap 是本地重写的节点。它订阅其中两条话题：位姿先平移到飞控 IMU 所在点，再减去两阶段原点（启动时锁一次、解锁时只归零偏航）；速度则原样转发。然后分三路输出：/mavros/vision_pose/pose 喂 EKF2，/mocap/pose_enu 与 /mocap/velocity_enu 给控制器与记录。它**不做任何坐标转换**：ENU→NED 与 FLU→FRD 全部留给 mavros 在自己的边界上完成。这条分工让 mocap 可以只用纯函数测试，也避免同一套转换在两处各写一遍。

一句话总结：动捕代码功能包。

6.3.2 任务与参考：traj 与 control

traj 只回答「第 t 秒该在哪、以多快的速度经过」。它提供四条解析轨迹：原地悬停、边长 1 m 的正方形、Gerono 8 字、以及从原点匀速直上 0.2 m 的起飞轨迹。/traj/select 选轨迹，/traj/start 定参考时钟零点；输出是 50 Hz 的 /traj/reference，另有 /traj/progress 与 RViz 用的 /traj/marker。参考**相对轨迹原点**，traj 不知道飞机在哪，也不碰 mavros。因此它可以在地面、没有飞控、没有动捕的条件下，把几何与限速跑一遍核对。

control 是任务总管。它把 /traj/reference 与轨迹原点相加，得到绝对目标，再按 output_mode 决定从哪里出去：position 档直接写 /mavros/setpoint_raw/local（位置 + yaw，type_mask=2552）；external 档只写 /control/target（绝对位置 + 速度 + yaw，type_mask=2496），由外部控制器接手。两种后端在构造时就**结构互斥**：external 档下根本不会创建 /mavros/setpoint_raw/local 的发布者，不是「创建了但发零」。control 还负责解锁、切 Offboard、起飞、跟踪、降落与失效停发；这些判据在零 ROS 的 fsm.hpp、rc_stick.hpp、rc_switch.hpp 里，节点层只做接线。

一句话总结：traj 是轨迹话题包，control 负责飞行流程任务流程 和 状态机。

6.3.3 控制器：nmpc

nmpc 是两层结构：sign_map、commands、params、model、ocp、timing、node_core 是零 ROS 核心层；mpc_node.py、offboard_smoke.py、preflight_check.py 是节点层入口脚本。核心层里，ocp 只输出内部 FLU 量 u = [F, ωx, ωy, ωz]，sign_map 负责把它变成下行字段；两者分开，是为了让「符号错了」与「模型解错了」可分别定位。

节点层按参数条件订阅：状态源固定为 /mocap/pose_enu 与 /mocap/velocity_enu，闸门源是 /mavros/state，参考源三选一（hold、traj、control），对应 /traj/reference 或 /control/target。输出只有一条执行器话题 /mavros/setpoint_raw/attitude：body_rate 为 FLU 原样、thrust 归一化到 0–1，type_mask=128（忽略姿态四元数）。它不 arm、不切模式、不读遥控器；这些动作留在 control 与 PX4。RViz 用的 /mpc/predicted_path 与 /mpc/reference 属于观测面，不进入飞行链。

一句话总结：nmpc 包含 数学公式层 和 ROS 对接层，主要由 acodos 生成。

6.3.4 编排与测试：bringup 与 bench

bringup 只回答「按什么顺序、用什么参数把已有进程摆到台上」。z0.launch.py 默认 use_mavros:=false、use_vrpn:=true，依次拉起 mocap、traj、control（以及可选的 vrpn_listener），可选的 mavros 走本仓的薄封装 mavros.launch.py。fly.py 是操作台：只发 /traj/select 并显示 /control/state 与完成度；/traj/start 由 control 在进入跟踪时自己发。它没有解锁、切模式或直接发 setpoint 的入口。mavros.launch.py 额外

挂 mavros_set_thrust_scaling.py，补上上游 px4.launch 因插件不读参数文件而漏掉的 thrust_scaling；这是起链补丁，不是飞行逻辑。

bench 是测试夹具，**不是飞行代码**。它提供假 mavros、假 PX4、脚本化动捕源 VrpnSource、假轨迹源 FakeTraj、被控对象 Plant、时间线 Timeline 与场景 Scenario，让节点级测试能在没有飞机、没有飞控、没有动捕棚的条件下跑。假 PX4 自 2026-10-13 起在本仓库已没有 DUT，只剩 bench 自测与 Scenario(px4=True) 的回退项会起它；节点级测试走假 mavros 的对应 setpoint 话题。它的安全边界是默认 ROS_DOMAIN_ID=73 且硬拒域 0，避免测试话题落到现场域。bench 没有可执行入口，也不被任何飞行包依赖；它只在 pytest 进程里被 import。

一句话总结：bringup 是节点启动功能包。

6.4 两条数据流：从 /traj/reference 到两个 setpoint 话题

两条控制路径共享前半段，在 control 的输出后端分叉。表 17 按「参考—任务层—控制器—载体—飞控」五段对照。

表 17　两条数据流。

段	offboard 位置档	外部控制器档（MPC）
1. 参考	traj 发 /traj/reference（相对轨迹原点，50 Hz）	同左
2. 控制目标	control 加轨迹原点，写 /mavros/setpoint_raw/local（type_mask=2552）	control 加轨迹原点，写 /control/target（绝对量，type_mask=2496）
3. 控制器	PX4 位置环直接消费位置 + yaw	mpc_node 的 node_core 做闸门与外推，ocp 求解，sign_map 映射为总推力 + 机体角速率
4. 载体	目标点，mavros 做 ENU→NED 转换后转发位置 setpoint	三轴角速率，/mavros/setpoint_raw/attitude（type_mask=128）→ mavros 做 FLU→FRD
5. 飞控	PX4 位置/速度环 → 姿态环 → 角速率环 → 混控	PX4 角速率环 → 混控

两条路径的反馈也不同：位置档读 /mavros/local_position/pose（EKF2 输出约 30 Hz），外部控制器档读 /mocap/pose_enu（动捕原生约 190 Hz），不经过 EKF2；两个频率的测量口径在第 5 章。安全网（/mavros/vision_pose/pose → EKF2）仍然存在，但它只保证 PX4 还能估计位置、还能执行失效保护，不是控制器的备用状态源。

互斥不仅在话题上，也在进程上：control 的 position 档不会启动 mpc_node；external 档不创建位置 setpoint 发布者。外部控制器断流 0.5 s 时 node_core 自己关闸，PX4 的 Offboard 丢失保护（COM_OF_LOSS_T=1.0 s）是最后一道。两条路径都不依赖对方还活着。

一句话总结：两条路径共享 traj 与 control，分叉点在后端：自己发位置，或交给 nmpc 发推力与角速率。

6.5 MPC 的层分界：零 ROS 核心层与节点层

6.5.1 哪些模块属于核心层

nmpc 按「有没有 ROS」切层：sign_map、commands、params、model、ocp、timing、node_core 是核心层，不许 import rclpy 或具体消息包，因此离线求解、台架与纯函数测试都能直接 import。node_core.py 决定「这一拍发不发、发什么」——新鲜度闸门、外推、参考构造与失效降级；它不 import acados 或 ROS，可用普通 pytest 断言。ocp.py 只求解，sign_map.py 只映射，符号错误与求解错误不会混在一起。

节点层是 mpc_node.py：声明参数、按档订阅、把消息搬进核心层、把输出填进 AttitudeTarget。offboard_smoke.py 与 preflight_check.py 也是入口脚本，不属于核心层。

「零 ROS」目前只由测试断言 rclpy 与 px4_msgs，尚未盯 mavros_msgs；缺口记在 票 13。设计完整，守卫未齐，不能把「现有断言全绿」当成边界已守住。

一句话总结：核心层用纯函数决定发不发、发什么，节点层只做接线；零 ROS 边界尚缺 mavros_msgs 断言。

6.5.2 改哪一层要不要 build

判断依据是进程实际 import 到哪一份，不是改了哪个文件。ros2 run nmpc mpc_node.py 解析 install/ 里的入口脚本；拷贝式安装下改 mpc_node.py、offboard_smoke.py、preflight_check.py 必须 colcon build。包模块看 PYTHONPATH：跑场脚本前置 src/nmpc 时，改 node_core.py、params.py、ocp.py 不用 build；只 source install/ 时读的是安装副本，仍要 build。--symlink-install 下入口脚本也会即时生效。

因此「入口脚本要 build、核心层不要」只在源码路径被前置时成立。判断错了，旧入口配新核心层会静默少字段，读数却看着和以前一样；先核对进程实际 import 的文件，再谈控制效果。

一句话总结：入口脚本来自 install/，包模块看 PYTHONPATH 指向源码还是安装副本，所以 build 规则不同。

6.6 测试布局

6.6.1 包内测试：seam 1 与 seam 2

包内测试主体分两个缝。**seam 2** 是纯逻辑，直接调用 C++ 头文件或 Python 模块，例如 nmpc 的 test_sign_map.py、test_commands.py、test_node_core.py、test_ocp_*.py、test_params.py；它回答「给定输入，逻辑与数值对不对」。**seam 1** 是节点级，把节点作为子进程起起来，用 bench 的假 mavros、假动捕与假轨迹喂话题，断言话题、进程存活、退出码与日志；它回答「接线、QoS、频率、时序对不对」。

nmpc 另有 test_mavros_interface.py(消息字段与 type_mask 契约)与 test_preflight_check.py(纯逻辑、参数与 ast 扫描)，都不起 DUT。bench 自测把域号钉在 73 并硬拒域 0，多夹具同跑要用 colcon test --executor sequential；vrpn_listener 与 bringup 没有测试。两个缝都不替代实飞。

一句话总结：包内 seam 2 直接调纯函数，seam 1 用 bench 夹具起节点子进程看话题，两者都不替代实飞。

6.6.2 仓库级测试：run/tests/ 与 tools/*/test_*.py

run/tests/ 守操作层：菜单号、命令与话题名钉到 z0.ACTIONS 等，安全演练文档钉到 docs/assets/z0.params，录制层测解析与安全边界，test_z0_check_live.py 对着假 mavros 做正控。tools/*/test_*.py 守离线测量与数据处理：电池与悬停推力、飞控参数、梯子基线、SITL 判据、符号映射、时间常数、RTI 收敛与报告数据；需要 bag 或 .ulg 的用例自己取。两组都不测控制律。

跑之前 source /opt/ros/humble、~/px4_msgs_ws、~/mavros_ws 与本仓 install/；缺 px4_msgs 时 seam 1 在 collection 报错，run/tests 与 tools/* 的依赖用例被 importorskip 跳过。全量测试还要设 ACADOS_SOURCE_DIR，否则 acados 会被猜成源码目录。

一句话总结：run/tests 守操作文档与菜单脚本，tools 的测试守离线测量与报告数据，它们都不测控制律。

6.7 证据目录的角色

仓库里叫「证据」的东西分散在四个位置，职责不同，不能混放。

表 18　证据目录。

目录	存什么	谁写	怎么用
bag/	场景目录的 rosbag 与 session.json；飞控 .ulg 若已归档则放在 fcu/；根目录有自动生成的总表 MANIFEST.md，old/ 存归档；另有导出过 .params	录制层 run/rec.py 与飞控	报告里的事故复现、频率与轨迹读数；不手改
tools/*/evidence/	各离线工具的输出：JSON、文本、原始扫描结果	对应工具	支撑参数辨识、SITL 判据、时间常数与报告数据；重跑工具生成，不手填
docs/assets/	z0.params 参数快照、论文 PDF、演练记录、上游项目说明	人工整理	引用飞控参数与外部资料的**固定快照**；改动要留版本
docs/report/	报告章节、data/、figures/、generated/（图题等生成物）	报告写作与数据管线	第 5 阶段的最终产物；结果数据与图由数据管线从 data/ 生成

四类目录的共同纪律是：**原始记录只增不改，派生读数由工具重算。**bag/ 里的一场飞行不能为了「让图好看」去改；tools/*/evidence/ 里的数字不能手填，必须能由工具重跑；docs/assets/z0.params 是某个时点的参数快照，不是活飞控的实时值；docs/report/generated/ 里的内容如果与 data/ 对不上，以数据管线为准。

更细的逐场索引、文件清单与读数口径属于第 4 章、第 10 章与资料索引，本章只固定它们的角色与边界。

一句话总结：bag 存飞行记录，evidence 存读数，docs/assets 存参数与文献，docs/report 存报告产物。

6.8 本章引用清单

表 19　本章引用清单。

主题	文件
包清单、依赖、入口	src/*/package.xml、各包 CMakeLists.txt 与 README.md
上游项目取舍与包来源	docs/reference/code-inventory.md
MPC 的实现分层	docs/control/phase2-implementation.md、src/nmpc/nmpc/__init__.py
节点入口与条件订阅	src/nmpc/nmpc/mpc_node.py
零 ROS 决策逻辑	src/nmpc/nmpc/node_core.py
下行映射	src/nmpc/nmpc/sign_map.py、docs/control/sign-map.md
任务总管与两条输出后端	src/control/README.md、src/control/include/control/fsm.hpp
启动层与操作台	src/bringup/README.md、src/bringup/launch/z0.launch.py
台架夹具与安全域	src/bench/README.md、src/bench/bench/pytest_support.py
动捕相对位姿	src/mocap/README.md
参考轨迹	src/traj/README.md
VRPN 桥的来源与不可改点	src/vrpn_listener/VENDORED.md
人工起链、录制与仓库级测试	run/README.md、run/tests/
录制数据的组织	bag/README.md
报告数据口径与复现	docs/report/data/README.md
入口脚本与包模块的 build 规则	.scratch/phase2/issues/20-vertical-channel-mismatch-offset.md、.scratch/phase4/issues/13-mavros-downlink-switch.md

主题	文件
术语与书写约定	CONTEXT.md

一句话总结：本章每个结构结论都能回到仓库内的源码、包清单或第一手记录。

第 7 章 启动脚本与遥控等交互（含安全/FSM）

本章是 “现场那一页” 的报告版：从摆好飞机、起链、选轨迹、拨开关、按开始键，一直写到降落、录制与出事后的处置。它回答的不是 “代码怎么写”，而是 “这一趟飞下来，人在什么时候做什么、机器在什么时候接手、每一步凭什么判断它可以往下走”。

第 5 章给出了层与层之间的接口和话题，第 6 章按包拆了代码结构；本章不重复它们，只把两条控制链的操作顺序、就绪判据、遥控通道、状态机与失效路径串起来。MPC 的公式见第 8 章，实验设计与数据处理见第 9 章，结果与未验证项见第 10 章。本章的命令与菜单号以 `run/README.md` 为唯一出处，现场动作以 `operation_manual.md` 为准，通道裁定回到 `params-changes.md` §H，安全结论回到 `docs/assets/drill/` 的第一手日志。

7.1 术语与通道编号

本章只用几个固定词，先把它们钉死，后面不再解释。**控制权**回答 “谁在驱动飞机”，只有 `MANUAL` 与 `AUTO` 两个值；**任务态**回答 “这一趟走到哪一步”，有八个值，逐项见7.7 节的表 26。两者是正交的，`/control/state` 把二者拼成一行，例如 `MANUAL/STANDBY` 或 `AUTO/TRACK`。**开始键**指遥控器上的 ch9，**杀车**指 ch8，**软停**指不切断动力的三种停任务方式。回退 (A) 级与 (C) 级沿用项目里的分档：(A) 是 offboard 位置控制，(C) 是 MPC 下发角速率与推力，角速率环与混控仍由 PX4 做。

表 20　本章术语。

词	含义	在哪读
控制权	`MANUAL` / `AUTO`：飞机是否正被我们发的指令驱动	`/control/state` 斜杠前
任务态	`INIT`…`RC_OVERRIDE` 八个值：任务走到哪一步	`/control/state` 斜杠后
开始键	ch9；两次上升沿分别对应起飞与开始轨迹，长按对应自动降落	`/mavros/rc/in` 的 `channels[8]`
杀车	ch8；高值让 PX4 立即停桨	PX4 的 `manual_lockdown`
软停	长按 ch9、打杆、拨 ch6 三条不切断动力的停任务路径	本章 7.8 节
回退 (A) 级	放弃 MPC，回到 offboard 位置链	`run/z0.py up` 第 1 档
通道编号	遥控器侧的 ch 编号，从 1 起	`RC_MAP_*` 参数与 `RC_CHANNELS`

`ch9` 在这个仓库里一律指遥控器通道号，不是数组下标；代码读的是 `/mavros/rc/in` 的 `channels[8]`，两者差 1，现场核对时不要混。

一句话总结：控制权与任务态分开读，通道号从 1 起。

7.2 一键起链：两条链与一个问句

`run/z0.py` 是一张 11 项的交互菜单，本身不含任何飞行逻辑。它把命令送出去、把读数摊开，判据与记录仍在 `bench-runbook.md` 里。现场最常用的是其中四项：第 8 项 “一键启动（动捕 +mavros，手动遥控飞）”、第 9 项 “一键启动全部（起前选控制器；默认 offboard 控制）”、第 10 项 “清除后台节点”、第 11 项 “开始录制（选场景 1–6）”。其余各项分别做环境自检、起链核对，或单独起某一个节点。

表 21　一键起链的入口。

入口	等价于	起什么	起完之后谁在上位
`run/up.sh`	`run/z0.py up`，菜单第 9 项	公共前缀 + 所选尾巴	`control`，或 `mpc_node`（见下）
`run/z0.py up_manual`	菜单第 8 项	mavros → vrpn_listener → mocap	没有人；飞机由遥控器手动飞

入口	等价于	起什么	起完之后谁在上位
run/down.sh	run/z0.py down，菜单第 10 项	按名字停进程	——
run/z0.py check	菜单第 2 项	只读核对表	——

两条一键链的公共前缀逐字相同：mavros（可选带 GCS 桥给 QGC）→ vrpn_listener → mocap，每一步都等判据成立再走下一步。差别只在起链前选出的 “尾巴”。

第一种是手动档。up_manual 只起前缀三步，不起 traj，也不起 control。这意味着整条链里没有任何程序在发布 setpoint；起完之后，操作者直接用遥控器手动起飞。它适合检查动捕与飞控链路，或者单纯把飞机飞起来。

第二种是自动档。菜单第 9 项会先问一句 “这一趟谁在上位控制飞机”，输入数字选尾巴；直接回车等于第 1 档。两档的尾巴都包含 traj，这是操作者 2026-10-16 定下的结构事实——“轨迹必须起没得选”——不是提示语。第 1 档是 traj → control，control 自己发 PX4 setpoint，output_mode 保持默认的 position；第 2 档是 traj → mpc → control，control 以 output_mode:=external 起，mpc_node 带 reference_source:=control 与 offboard_prestream:=true。两档的现场动作逐字相同：选轨迹、ch6 拨到 Offboard、开始键第 1 次上升沿、判稳后第 2 次上升沿并松手、飞完自动降落。区别只在 “最后给 PX4 发指令的人” 是谁。

所以 “谁在上位控制飞机” 问的不是 “谁在管任务”，而是 “谁的指令最后落到 PX4 的执行器 setpoint 上”。两档里 control 始终是任务总管：它管开始键、解锁、切 OFFBOARD、起飞、判稳、记录轨迹原点、发 /traj/start 与降落。第 2 档只是把最后一段换成 mpc_node 求解后写姿态 setpoint。手动档则连任务总管都不起，飞机完全由人飞。

run/down.sh 是这条链的另一头。它按名字扫描 DOWN_TARGETS 白名单——mavros、vrpn_listener、mocap、traj、control、mpc_node 与 bringup 壳——先把 pid 与完整命令行摊开，问一句 y/N，再按 SIGINT → 等 5 s → SIGTERM → 等 3 s → SIGKILL 的阶梯停。它不区分这些进程是谁起的：别的会话、跑着的测试，只要名字对上都会进清单。**停进程不是降落**：setpoint 一停，PX4 在 COM_OF_LOSS_T 之后自己退出 OFFBOARD，按 COM_OBL_RC_ACT 落回 Position；飞机若还在空中，要先用 7.8 节的软停路径处置。

一句话总结：两条链只差最后发指令的人：遥控器或程序。

7.3 启动顺序与判读

自动档的起链顺序是固定的：mavros → vrpn_listener → mocap → traj →（第 2 档才有的 mpc）→ control。顺序不能反，原因分散在每一步的判据里：后面的节点消费前面的话题，前面没起来，后面即使进程在跑也只是空转。

表 22　启动顺序与就绪判据。

步	起什么	等什么判据	上限	现场提示
1	mavros	/mavros/state 的 connected=True	60 s	★ 有帧 ≠ 连上飞控
2	vrpn_listener	/vrpn/< 刚体 >/pose 有帧	30 s	Z0 / z0 两种刚体名都认
3	mocap	/mocap/pose_enu 有帧	30 s	起之前确认 “飞机已摆好、静止、固定？”
4	traj	不等	——	没被选就一帧不发，等它等于等到天荒地老
5	mpc（第 2 档）	它自己的启动日志行	120 s	闸门关着时话题上看不出 “我起来了”
5′	control（两档都有）	/control/state 有帧	30 s	起始态是 MANUAL/INIT
6	终端让给 fly.py	操作者选 1/2/3/4	——	起飞前必须选好轨迹

步	起什么	等什么判据	上限	现场提示
7	前台守着	——	——	打印核对表与起飞提示，Ctrl-C 一起停

第一步的判据值得单独说。`/mavros/state` 有帧只说明 mavros 在发布，而 `connected` 是消息里的一个字段，由 mavros 是否收到飞控心跳决定。看“有没有帧”会把“串口开着但飞控没回话”读成已连接；菜单里那句“有帧 ≠ 连上飞控”就是冲这个来的。第 1 步起之前还会查 `/dev/ttyTHS1` 被谁占着，因为第二个 mavros 打开串口失败的报错长得很像 launch 坏了。

第三步的确认语不是客套。`mocap` 节点在启动那一刻锁定阶段 A 的原点，飞机没摆好、还在动，原点就记歪了；后面所有相对位姿都会带这个偏差。这一步起之前必须让飞机静止、固定。

第四步的 `traj` 刻意不等。它只发参考、不碰飞控，没收到 `/traj/select` 就一帧不发；用“有没有帧”当就绪判据会永远等不到。真正的选轨迹发生在第 6 步，终端交给 `fly.py` 之后，而且必须在起飞前选好。

第 2 档的 `mpc_node` 也不能用话题判断。闸门关着的时候它不发执行器指令，只有 `offboard_prestream` 的纯悬停帧，话题面上没有“我起来了”这个事实。所以判据取它自己的启动日志行 `MPC`（模型预测控制）节点启动:，上限 120 s；等不到就打日志尾巴、把已经起的全部停掉并返回 1。这条路径还带一个环境前提：命令自带 `ACADOS_SOURCE_DIR` 与 `LD_LIBRARY_PATH`，缺一个就会在建求解器时报 `OSError: libacados.so / libhpipm.so`，2026-10-17 的起链实测就死在第 5 步。

第六步之后，`fly.py` 在同一个终端里选轨迹，选完退出；`z0.py` 打印核对表与起飞提示，然后前台守着。菜单第 2 项 `run/z0.py check` 随时可以另开一个终端跑，它把每个话题有没有帧、多少 Hz，`/mavros/state` 的 `connected/armed/mode`，以及 `/control/state` 的原文摊成一张表。

一句话总结：每步等判据成立再走；traj 不等。

7.4 输出路径：同一套任务、两个最后执行者

两档的现场动作相同，区别只在最后一跳。表 23 把两条输出路径并排放在一起。

表 23　两条输出路径。

尾巴	谁写	话题	消息内容	之后由谁算控制量
第 1 档 offboard	`control`	`/mavros/setpoint_raw/local`	绝对 ENU 位置 + yaw，`type_mask=2552`	PX4 的位置环 → 姿态环 → 角速率环 → 混控
第 2 档 MPC	`control`	`/control/target`	绝对 ENU 位置 + 速度 + yaw，`type_mask=2496`	不直接到飞控；`mpc_node` 拿它当参考
第 2 档 MPC	`mpc_node`	`/mavros/setpoint_raw/attitude`	机体三轴角速率（FLU）+ 归一化总推力，`type_mask=128`	PX4 的角速率环 → 混控

第 1 档里，`control` 从 `STANDBY` 起就以 50 Hz 发 `/mavros/setpoint_raw/local`，切 OFFBOARD 之前已经预流满 1 s。预流的作用不是那些帧本身——切模式前 PX4 会把它们丢掉——而是每条 setpoint 触发的 offboard 心跳，它满足 PX4 的“切模式前 setpoint 已在流”前置。切进 OFFBOARD 之后，同样的帧才真正驱动飞机。

第 2 档里，`control` 一帧都不写 `/mavros/setpoint_raw/local`。它只写内部目标 `/control/target`，`mpc_node` 订阅这个目标与原始动捕，解出总推力与机体角速率，再写 `/mavros/setpoint_raw/attitude`。`type_mask=128` 表示忽略姿态四元数，只取角速率与推力。mavros 在边界上把角速率从 FLU 转到 FRD，推力不转；PX4 只保留角速率环、混控与安全机制。这是 (C) 级的定义：我们下到角速率，不下到电机。

两条路在结构上互斥，不靠现场纪律：external 档下 `control` 不创建 local 发布者，offboard 档下不启动 `mpc_node`。因此不存在两路 setpoint 互相覆盖的窗口。

第 2 档还有一道闸门。`/control/target` 断流 0.5 s，`mpc_node` 就以 `no_control_target` 关闸并**停流**——不发布，既不冻结在最后一个目标，也不发零帧。再往后是 PX4 自己的保护：`COM_OF_LOSS_T=1.0` s 内没有 setpoint，退出 OFFBOARD，按 `COM_OBL_RC_ACT=0` 回 Position。求解器不可用时另有 hold → hover 的兜底阶梯，但那不是这条断流闸门的一部分；最后一道始终是 PX4 的超时。

`offboard_prestream` 只负责在 `control` 请求 OFFBOARD 之前让 PX4 看见姿态流。它不 arm、不 set_mode；这两件事仍然只在 `control` 手里。

一句话总结：两条尾巴最后写的 setpoint 话题不同。

7.5 录制：六条场景与一次收口

录制由菜单第 11 项启动。它选场景 1–6，起 `ros2 bag record`，并**占住终端**：屏幕上走倒计时，录满 120 s（场景 1–5）自己收口；想提前停就按 Ctrl-C，这与倒计时到点走的是同一条 SIGINT 收口路径。

表 24　六条录制场景。

场景	名字	时长	用途
1	静止	120 s	噪声基线
2	手飞起飞	120 s	起飞瞬态
3	轨迹 1（悬停）	120 s	悬停判据
4	轨迹 2（正方形）	120 s	有机动的激励
5	轨迹 3（8 字）	120 s	另一类机动
6	任意录制	不到点	自由补录，只有 Ctrl-C 收口

录制的话题走白名单正则 `^/(mavros|mocap|traj|control|vrpn)/`，不是 `-a`；压缩用 zstd，在收口那一刻做。收口的动作是固定的：SIGINT 之后等它写缓存、压包、关库，最多等 45 s，等不到才升级到 SIGTERM / SIGKILL；然后用 `ros2 bag info` 的实测帧数逐条回对预期，写 `session.json`（时长、大小、话题、频率、sha256、缺了什么、为什么收的），再重算 `bag/MANIFEST.md`。

这一层只订阅，不解锁、不切模式、不发帧。这条边界有机器守卫：`test_rec.py` 用 AST 扫录制层的代码，禁止出现 `create_publisher` / `create_service` / `create_client`，也禁止那四个与“让飞机动”有关的动作。它和 `run/z0.py` 的其余部分一样，不给程序起飞入口。

现场没有“停止录制”这一项。菜单第 11 项一启动就占住终端，想提前停只能按 Ctrl-C；要看进度，另开一个终端跑 `run/z0.py rec_status`。手工拷完飞控日志（`.ulg`）之后，可以跑 `run/z0.py rec_manifest` 重算总表。飞控日志本身不由这一层拷：拔 SD 卡，把 `< 卡根 >/log/sessNNN/logMMM.ulg` 手工放进 `bag/< 场景 >/fcu/`。

录制也不是降落。飞机还在空中时按 Ctrl-C 只结束录制进程，不改变飞机状态；要停任务仍要走 7.8 节的软停路径。

一句话总结：录制只订阅、不解锁、不发帧。

7.6 遥控通道：四个开关，四条职责

遥控器的通道分配是 Phase 1 的输入，不是随手定的。`z0.params` 里非零的 `RC_MAP_*` 只有七条：四条摇杆加三个开关。其余的 `RC_MAP_*` 全为 0，ch9 不在占用表里，所以拨它不会切模式、不会解锁、不会杀车。

表 25　遥控通道。

通道	功能	参数	谁读	现场怎么用
ch1–ch4	横滚 / 俯仰 / 油门 / 偏航	RC_MAP_ROLL/PITCH/THROTTLE/YAW = 1/2/3/4	PX4；control 也读，用于打杆判据	飞行中别碰杆，判据看的是变化率
ch5	解锁开关（冗余硬件路径）	RC_MAP_ARM_SW=5	PX4	现场不要用；程序解锁才是主路径
ch6	模式开关	RC_MAP_FLTMODE=6	PX4	第 6 档 = Offboard，整段任务停在它上面
ch8	杀车开关	RC_MAP_KILL_SW=8; RC_KILLSWITCH_TH=0.75	PX4	高值 = 立即停桨，不是“取消任务”
ch9	开始键	start_channel=9，读 channels[8]	control	两次上升沿 + 一次长按

ch6 的三个档位要按实测说。快照里 COM_FLTMODE1=2、COM_FLTMODE4=8、COM_FLTMODE6=7；2026-10-12 的通道读数里，ch6 低档 999 报 POSCTL（Position）、中档 1503 报 STABILIZED、高档 2000 报 OFFBOARD。现场只用第 6 档，也就是 Offboard；第 2、3、5 档未分配，拨过去没有反应。operation_manual.md §1 的括号里仍写着 COM_FLTMODE1/4/6 = 2/1/7，其中第 4 位与快照的 8、实测的 STABILIZED 不符——这是文档里的旧值，现场动作（第 6 档）不受影响，但读参数时要按快照与实测。

ch9 的判据分三层。第一层是归一化阈值：低位 < 0.25、高位 > 0.75，去抖 200 ms。第二层是采样：/mavros/rc/in 是 10 Hz，200 ms 去抖等于连续 3 帧，所以每次压住至少 0.3 s 再松，压得太贴边会有一半概率按不动，而且连“被丢掉”的日志都没有。第三层是动作归属：上电即在高位不算边沿，必须先见到低位；第 1 次上升沿是解锁加起飞；第 2 次按下只“记下”，**松手**那一刻才进 TRACK；在飞时长按 ⩾1 s 是立即自动降落，判据从电平确认起算，物理上要按住约 1.2 s，而且资格看**按下那一刻**——起飞那一下按完一直按着不放，不会让飞机刚离地就降落。

ch5 保留在参数表里，是冗余的硬件解锁路径，但现场不用它。理由有两条第一手事实：在 OFFBOARD 里拨 ch5，PX4 会以 Arming denied: switch to manual mode first 拒绝；即使在别的模式解开，armed 的变化也不是程序发起的，control 会停发 setpoint、告警并退回 MANUAL/INIT，把飞机留在飞行员手里。程序解锁与开始键那条路才是“已解锁且仍在发 setpoint”的入口。

ch8 是最后手段。它的语义链是 RC_MAP_KILL_SW=8 → RC_KILLSWITCH_TH=0.75 → 高值 = ACTION_KILL → manual_lockdown=true，也就是立即停桨；拨它不等于取消任务，空中拨它等于让飞机失去动力掉下来。程序侧根本不读这条通道，所以拨 ch8 不会在我们的状态机里留下任何状态变化。

COM_RC_IN_MODE 是这层开关的总闸，现值 0。它曾经是 3，语义是“先到的源一直用到重启”；QGC 一上来就发 MANUAL_CONTROL，飞控会把 MAVLink 源锁成首选源，而 ManualControl 只在选中源是 SOURCE_RC 时才评估开关。后果是 ch5、ch6、ch8 与内八手势**整层静默失效**，现场症状是“拨 ch6 模式不变”，连一条拒绝的 STATUSTEXT 都没有。2026-09-23 改成 0（只收 RC）后当场复现：ch6 拨档，模式跟着走。这个值不要改回 3。

一句话总结：四个开关职责唯一，互不复用。

7.7 状态机：两个正交维度

状态机把“谁在控制”与“在干什么”拆成两个维度，而不是一个扁平枚举。控制权是 MANUAL / AUTO，任务态是 INIT、STANDBY、TAKEOFF、HOVER、WAIT_START、TRACK、LAND、RC_OVERRIDE。/control/state 以 "< 控制权 >/< 任务态 >" 发布，例如 MANUAL/STANDBY、AUTO/TAKEOFF、MANUAL/RC_OVERRIDE。

表 26　两个维度。

维度	取值	回答的问题
控制权	MANUAL / AUTO	飞机是否正被我们发的指令驱动
任务态	INIT / STANDBY / TAKEOFF / HOVER / WAIT_START / TRACK / LAND / RC_OVERRIDE	任务走到哪一步

AUTO 的定义不是 “我们连上了飞控”，而是 “我们正在发真正驱动飞机的指令”——已解锁、在 OFFBOARD、setpoint 在流。已进入 OFFBOARD 但还没解锁时仍是 MANUAL：那时的 setpoint 不驱动飞机，飞机还在人手里。反过来，夺权时控制权与任务态会同时置位：打杆或模式被切走，Authority 变 MANUAL，Phase 变 RC_OVERRIDE，于是读到 MANUAL/RC_OVERRIDE。

正常路径如下。第 1 步，INIT 等到 state.connected、动捕有数据、local_position 有效，进 STANDBY；STANDBY 已经开始以 50 Hz 发 setpoint，目标就是当前位置与当前机头，但飞机未解锁。第 2 步，收到开始键第 1 次上升沿，或台架上的 /control/takeoff 服务调用，control 发出解锁请求；**受理不等于起飞**，对外仍是 STANDBY。第 3 步，state.armed 出现上升沿，就在这一帧锁定起飞原点与机头，进 AUTO/TAKEOFF，目标高度从原点沿 0.50 m/s 的斜坡升到 +1.00 m，水平两轴钉在原点。第 4 步，飞机当前位置与 “原点 + 1.00 m” 的差小于 0.10 m、垂直速度小于 0.10 m/s 并持续 1.0 s，进 HOVER；悬停判稳用的是更严的一档，位置与速度都小于 0.10 并持续 2.0 s。第 5 步，判稳后记录轨迹原点，进 AUTO/WAIT_START，目标就是轨迹原点；这里等开始键第 2 次上升沿，兜底超时 300 s，超时按正常任务结束请求降落，end_reason=wait_start_timeout。第 6 步，第 2 次上升沿按下只记下，**松手**才进 AUTO/TRACK，control 这时自己发 /traj/start，目标变成轨迹原点加 /traj/reference。第 7 步，/traj/progress 首次达到 1.0（必须先见过小于 1.0），进 MANUAL/LAND：停发 setpoint，请求 AUTO.LAND，end_reason=traj_done；PX4 在 COM_DISARM_LAND=2.0 s 后自己上锁，状态回到 MANUAL/STANDBY。

起飞这一步有三条前置，缺一条解锁就会被拒：已经进入 OFFBOARD、setpoint 已预流满 1 s、遥控在线且位置守卫成立。开始键第 1 次上升沿发出的起飞命令有 5 s 的有效期（takeoff_timeout_s），过期没等到 armed 就作废，要再按一次。在飞的四个任务态里，长按开始键 ⩾1 s 都直接进 LAND，end_reason=land_pressed，这是唯一一条 “仍由程序自主降落、不交还控制权” 的停法。

完整的状态转移表不在这里重抄。z0 的转移条件以 phase1-implementation.md §4.2 为准；参考项目 XI35 的五个状态与十一条转换在 xi35-fsm.md。两者的关键差别在接管语义：XI35 把摇杆位移积分进悬停点，z0 是立即停止发布，让 PX4 按 COM_OBL_RC_ACT 接管。

一句话总结：控制权与任务态正交，可自由组合。

7.8 失效路径与复位

失效先分成两类，处置刻意不同。**控制权被夺走**包括打杆接管 rc_override 与模式被外部切走 mode_lost；处置是立即停发 setpoint、不请求任何模式、交还控制权——既不抢回 OFFBOARD，也不请求 AUTO.LAND，因为那是在跟正在接管飞机的人抢。**输入坏了**包括 state_timeout、mocap_timeout、local_pos_timeout、rc_timeout、battery_low、position_invalid、mocap_stamp_stalled、ekf2_pos_lost、traj_timeout；处置同样是先停发，若已解锁再请求 AUTO.LAND，把降落交给 PX4。

表 27　失效分类。

类	原因 token	停发	请求 AUTO.LAND	落点
控制权被夺走	rc_override、mode_lost	是	否	MANUAL/RC_OVERRIDE

类	原因 token	停发	请求 AUTO.LAND	落点
输入坏了	state_timeout、mocap_timeout、local_pos_timeout、rc_timeout、battery_low、position_invalid、mocap_stamp_stalled、ekf2_pos_lost、traj_timeout	是	已解锁时是	MANUAL/LAND；未解锁时回 INIT

输入超时的口径是逐条定的，不是照抄一个 0.5 s。/mavros/state 实测 1 Hz，超时取 2.5 s，容一次丢帧；/mavros/local_position/pose、/mocap/pose_enu、/mavros/rc/in 取 0.5 s；/mavros/battery 不做超时判据，做的是低电压值判据，因为飞控静默后最后一帧若低，它仍然低；/mavros/estimator_status 取 2.5 s，只作 ekf2_pos_lost 的新鲜度门。/mavros/rc/in 的超时也只在已解锁时才算失效：没解锁时“遥控没开”不会让飞机动。

现场能停任务的有三条软路径，外加一条不是软路径的杀车。

表 28　停任务的三条软路径与杀车。

路径	动作	程序做什么	飞机最后谁管
(a) 长按开始键	ch9 按住约 1.2 s	停发 setpoint + 请求 AUTO.LAND，end_reason=land_pressed	仍由程序自主降落
(b) 打杆	摇杆变化率超过 100 µs/s，且已解锁	0.3 s 内停发、退到 MANUAL/RC_OVERRIDE、不请求任何模式	PX4 按 COM_OBL_RC_ACT=0 回 Position，此后由人飞
(c) 拨 ch6	拨离第 6 档	判 mode_lost，停发并交还，连 AUTO.LAND 都不请求	PX4 用它自己的控制器，此后由人飞
(d) 拨 ch8	拨到高值	程序不读这条通道，状态不变	PX4 立即停桨；这是杀车，不是停任务

复位只有两条路，按键本身不是一条路。第一条是**可信的落地上锁**：connected=true 且 armed=false，并且要排除飞控断链时 mavros 合成的那一帧断连；这条会清闩锁并重置任务。第二条是**外部把模式放回 OFFBOARD**：闩锁期间我们不请求 OFFBOARD，所以那个上升沿只可能来自人；这条只清夺权闩锁。按 ch9 被明确拒绝，因为控制权不在我们手上时不该由我们复位。未解锁时的输入失效不闩锁：判据恢复就自动回 STANDBY，不需要任何动作。

闩锁有两种。kTakeover 由夺权触发，不论是否解锁，只能靠上面两条复位路清；kArmedDegrade 由已解锁时的输入失效触发，只有可信的落地上锁能清——电池不会因为有人切了模式就变满。闩锁期间日志每 10 s 重报一次，状态本身不会自己松动。

一句话总结：软停先停 setpoint；复位只认落地上锁或交回。

7.9 安全路径的第一手证据

这一节只写 2026-10-12 那几场演练里实际发生的事。PX4 侧的原文都能在 docs/assets/drill/ 里按时间戳找到；节点侧的收尾细节（ps 无输出、launch 的 process has died）在 phase4 票 08 的 Comments 里。演练的判据与记录体例在 safety-paths-drill.md。

表 29　2026-10-12 的安全路径实测。

路径	时刻	条件	结果	原始日志
打杆接管	07:23:27.970	装桨、实飞，起飞中被夺权	degrade=rc_override 命中，停发，回 Position	drill21-probe-run9.log
拨 ch6 夺权	03:59:00.046	未解锁；模式被切走	degrade=mode_lost，停发，闩锁不自恢复	takeover-modelost-entry.log

路径	时刻	条件	结果	原始日志
offboard 心跳丢失（SIGINT）	07:52:28.045	实飞；`control` 正常停	PX4 `Failsafe activated`，回 Position，自动降落	`drill23-sigint-probe-run12.log`
节点崩溃（`kill -9`）	07:40:45.279	实飞；进程被强杀	与上一行同路径，`/control/state` 不再更新	`drill24-probe-run10.log`
ch8 杀车	08:35:49.134	拆桨，未解锁	PX4 报 `Kill-switch engaged` / `Flight termination active`，程序零反应	`drill22-ch8-killswitch-probe-run13.log`

打杆接管。07:23:26.268, ch9 从 999 到 2000, 压住 0.35 s; 07:23:26.554 PX4 报 `Armed by external command`; 07:23:26.951 `/control/state` 到 `AUTO/TAKEOFF`，setpoint 保持 50 Hz。07:23:27.970, PX4 报 `Pilot took over using sticks`; 同一拍摇杆 ch3 从 989 到 1059; 07:23:27.973 `/control/state` 到 `MANUAL/RC_OVERRIDE`; 07:23:27.976 日志 `degrade=rc_override`，已闩锁。随后 setpoint 从 50 Hz 掉到 10 Hz、再到 0 Hz, 07:23:28.926 模式落到 `POSCTL`, 07:23:20–07:24:10 之间我方发起的模式请求为 0 条。打杆到停发的时间差是 1 ms（探针 07:23:27.972，节点 07:23:27.973）。07:23:37.514 `Disarmed by landing`，07:23:39.934 回到 `MANUAL/STANDBY`，`degrade=none`：降级已复位（此前 `degrade=rc_override`，持续 `12.0 s`）。

拨 ch6 夺权。03:59:00.046，模式从 `OFFBOARD` 变成 `STABILIZED`; 03:59:00.057 `/control/state` 到 `MANUAL/RC_OVERRIDE`; 03:59:00.059 `degrade=mode_lost`，已闩锁。setpoint 从 50 Hz 掉到 0 Hz，最后一帧停在约 +5.2 s；模式停在操作者拨到的位置，没有一条我方模式命令；窗口剩余 115 s 内每 10 s 重报 “已闩锁”，状态纹丝不动。这一场全程未解锁，所以它证明的是夺权通路本身，不是打杆判据。

offboard 心跳丢失。07:52:26.897 是最后一帧 setpoint（操作者对 `control` 敲 `kill -INT`）; 07:52:28.045 PX4 报 `Failsafe activated`，间隔约 1.15 s，对得上 `COM_OF_LOSS_T=1.0 s` 加余量; 07:52:28.905 模式落到 `POSCTL`; 07:52:36.752 `Landing detected`; 07:52:38.758 `Disarmed by landing`。这条路径不依赖我们的节点还活着，停发超过 1 s 由 PX4 自己接管。

节点崩溃。07:40:44 最后一帧 setpoint; 07:40:45.279 PX4 报 `Failsafe activated`; 07:40:45.915 模式落到 `POSCTL`; 07:40:48.839 `Landing detected`; 07:40:50.844 `Disarmed by landing`。节点侧 `ps` 无输出，launch 报 `process has died [pid …, exit code -9]`，`/control/state` 不再更新。SIGINT 与 `kill -9` 在 PX4 侧走的是同一条路；节点日志在两种情况都停在半句，差别只剩 launch 那一行是 `process has finished cleanly` 还是 `process has died`。

ch8 杀车。08:35:49.134 PX4 报 `Kill-switch engaged`，探针在同一秒的 08:35:49.164 读到 ch8 从 999 到 2000; 08:35:49.188 与 08:35:49.238 连续报 `Preflight Fail: Kill switch engaged` 与 `Preflight Fail: Flight termination active`。程序侧全程 `MANUAL/STANDBY` 一次没动，setpoint 全程 50 Hz 没掉一拍，没有任何 `degrade=` 行，其他通道一帧未动。08:35:53.640 PX4 报 `Kill-switch disengaged`，ch8 回到 999，此后不再重报。这一场拆桨、未解锁，所以它证明的是“PX4 收到杀车并进入终止状态、程序不读这条通道”，**没有**看到空中真停桨。

回退 (A) 级要按实际范围读。2026-10-12 走的是回退步骤里的第 1、4、5 步：确认 Phase 1 整条链仍在、随时可用。证据是 `2026-10-12-chain-up-clean.log`：`/control/state` 为 `MANUAL/STANDBY`，`/mavros/state` 为 `connected=True armed=False mode=OFFBOARD`，`/mavros/setpoint_raw/local` 约 50 Hz，`/traj/reference` 无帧。第 2、3 步——也就是**把正在跑的 (C) 链换成 (A) 链的那个切换动作本身**——从未演练过。2026-10-12 当天的理由是当时没有正在发 setpoint 的 (C) 链、没有可切的对象；到 2026-10-13，(C) 链的归属已经落地，这条理由不再成立，操作者仍裁定明确不做，理由是排期。按 “接受风险·没有实测读数作锚” 收口。同一张票里的 “求解失败注入” 也一样：实机链路上从未触发过求解失败，操作者按同一方式接受风险收口。

读这批材料时要留意的过期处。`safety-paths-drill.md` §0 与 §2.1 仍写着 “打杆试过半场、判据没命中”，那描述的是 03:59 那场 `mode_lost`; 07:23 那场后来按同一判据通过，记录在 phase4 票 08 的 “实测 ③”。同一份演练页 §3.4 说 (C) 链 “没有工具，只能 Ctrl-C”，那是在菜单第 9 项补上 MPC 档之前（票 14/15，2026-10-16 与 10-17）；现在 `down.sh` 的 `DOWN_TARGETS` 已经包含它。

run/z0.py 的 DOWN_WARNING 里还留着一句 NAV_RCL_ACT = 2 会试图返航，那是旧值；飞控现值是 1（Hold，2026-09-28 实测），遥控丢失时 PX4 就地保持，不返航。这些都在文档清扫票的范围里，本章按现状写，不按旧句子写。

一句话总结：每条安全路径都有当天的日志与时间戳。

7.10 现场边界：这条链里没有什么

这条链的边界比它做了什么更值得记住。run/z0.py 没有解锁、起飞、切模式、改参数、发任何帧的入口；这不是遗漏，而是把“开始键之前不发控制指令”这条防线留在飞控与 control 之间。z0.launch.py 也不给程序起飞入口，理由写在它的文件头：那会绕开同一条防线。

表 30　这条链刻意不做的事。

不做	原因	谁做
自动起桨 / 自动解锁	开始键之前不发控制指令	人按 ch9 第 1 次上升沿，或台架 T4 的 /control/takeoff
自动切模式到 Offboard	解锁前置要求“已在 OFFBOARD”	人拨 ch6 第 6 档
自动让飞机起飞	起飞由状态机在解锁后执行，不由起链脚本执行	control 在 AUTO/TAKEOFF 里爬升
运行中在 (A) 与 (C) 之间切换	两条链都会写 PX4 的 OFFBOARD，同时跑就是两个驾驶员	不存在这条路径；档位只在起链时选
用 down.sh 让飞机降落	停进程只是停进程，PX4 还要等超时	软停路径 (a)/(b)/(c)
用录制收口让飞机降落	录制只订阅	同上
用 QGC 抢模式	会与程序的状态机拉锯	现场不用 QGC 切模式
软件杀车通道	急停语义 = 停桨，只有 ch8 一条	PX4 的 manual_lockdown

还有两条容易在术语上滑过去的边界。第一，ch5 虽然保留在参数里，现场不把它当解锁手段：外部解锁会触发 control 的“非我方解锁”守卫，程序停发并退回 MANUAL/INIT，飞机留在人手里。第二，(A) 与 (C) 的互斥是结构性的：external 档下 control 不写 local 话题，offboard 档下不启动 mpc_node。所以“起链时选一次”不是提示语，是这条链唯一支持的切换时机；飞行中切换的整条路径不存在，演练页也明确不编造它的步骤。

一句话总结：起链不给程序起飞入口，停进程不等于降落。

7.11 本章引用清单

表 31 把本章的命令、通道、状态机与安全结论按主题归到仓库内的文件，便于回查。正文里已经随行给过链接，这里只是索引。

表 31　本章引用清单。

主题	文件
菜单、起链顺序、就绪判据、录制、down	run/README.md
现场时间轴、遥控动作、软停路径	operation_manual.md
遥控通道分配、阈值、COM_RC_IN_MODE、NAV_RCL_ACT	params-changes.md §H
状态、转移、降级、闩锁与复位	phase1-implementation.md §4
参考项目状态机与接管语义差异	xi35-fsm.md
安全路径演练判据与记录	safety-paths-drill.md
2026-10-12 演练日志	docs/assets/drill/
飞控参数现值	docs/assets/z0.params
状态机与遥控实现	fsm.hpp、rc_switch.hpp

主题	文件
起链脚本与录制层	`run/z0.py`、`run/rec.py`
录制层的 “只订阅” 守卫	`run/tests/test_rec.py`
起链与现场排障	`bench-runbook.md`
术语	`CONTEXT.md`

一句话总结：现场结论都能回到仓库内文件。

第 8 章 MPC 公式推导与应用

本章从刚体动力学出发，推导 z0 实际运行的十状态最优控制问题，再把它逐层落到 PX4 的接口上。全章按三层写：第一层是牛顿–欧拉方程与四元数运动学，属于物理推导；第二层是坐标系、四元数乘法顺序与符号映射，属于代码约定；第三层是权重、时域与求解器参数，属于工程选择。三层分开写，是为了让读者能判断哪一步是“推出来的”，哪一步是“选出来的”。

第 5 章已给出两条控制链的接口边界，本章不重复话题表；第 9 章讲实验设计，第 10 章给结果。本章只回答一个问题：控制器每一拍解的那个优化问题，究竟长什么样，凭什么能发到 PX4 上。

8.1 坐标系与符号

8.1.1 三个坐标系与转换边界

模型内部只用两套坐标系：世界系 ENU（东—北—天）与机体系 FLU（前—左—上）。PX4 内部用 NED（北—东—地）与 FRD（前—右—下）。两套体系不是同一套轴换了名字，而是轴的方向真的不同；表 32 给出每个量的口径与转换发生的位置。

表 32　坐标系口径与转换边界。

量	模型内部	PX4 内部	转换由谁做
位置、速度	ENU	NED	mavros 边界
姿态四元数	ENU/FLU，q 把机体系矢量旋到世界系（机体相对世界系的姿态）	NED/FRD	mavros 边界
机体角速率	FLU	FRD	mavros 的 `setpoint_raw` 插件
总推力	标量 $F \geq 0$，沿机体 +z	`thrust_body_z`，向上为负	PX4 的 `fill_thrust`

模型的状态输入这一侧没有坐标转换：动捕给的就是 ENU/FLU，`pose_to_state` 原样装进状态向量。转换只发生在输出侧，而且只发生一次。这个选择的代价是本章 8.8 节必须把符号讲清楚；收益是模型内部没有第二套坐标约定，少一个静默出错的转换点。

四元数约定与代码一致：$q = [w, x, y, z]$，q 把**机体系**矢量旋到**世界系**，

$$\mathbf{v}_{\text{world}} = R(q)\,\mathbf{v}_{\text{body}}. \tag{8-1}$$

`q_to_rot_mat` 按这个方向展开旋转矩阵（`model.py:68-75`），`v_dot_q` 用它把推力转到世界系（`model.py:78-80`）。源码注释把 q 记作“机体 ← 世界”（`model.py:25`），这里的箭头是“机体的姿态相对世界系”的意思；旋转矩阵本身的方向以式 (8-1) 为准，不要按箭头把它读成“把世界系矢量旋到机体系”。若把方向读反，推力方向会整体反过来，症状是飞机一解锁就往下扎，而求解器本身不会报任何错。

一句话总结：模型内部用 ENU 世界系与 FLU 机体系，坐标转换只在 mavros 边界发生一次。

8.1.2 符号与代码变量对照

表 33 是本章的符号总表。同一个物理量在论文记号、代码变量与 PX4 字段里往往有三个名字，读公式时以本表为准。

表 33　符号、代码变量与出处。

符号	含义	代码变量	出处
$\mathbf{p} \in \mathbb{R}^3$	世界系位置	`p`，状态 `x[0:3]`	`model.py:163-166`
$\mathbf{v} \in \mathbb{R}^3$	世界系速度	`v`，状态 `x[3:6]`	同上
$q \in \mathbb{R}^4$	把机体系矢量旋到世界系的单位四元数（机体相对世界系的姿态）	`q`，状态 `x[6:10]`	同上
$F \in \mathbb{R}$	沿机体 +z 的总推力，N	`F`，输入 `u[0]`	`model.py:168-170`
$\omega \in \mathbb{R}^3$	机体三轴角速率，rad/s，FLU	`w`，输入 `u[1:4]`	`model.py:169-170`
m	起飞质量，kg	`params.mass`	`params.py:168`
g	重力加速度，$\mathrm{m/s^2}$	`9.81`（`params.G`）	`params.py:39`、`model.py:177`
N	预测步数	`OcpConfig.N`	`ocp.py:57`
T_f	预测时域，s	`OcpConfig.tf`	`ocp.py:58`
Δt	内部步长，s	`OfflineOcp.dt`	`ocp.py:424-427`
$\mathbf{x}, \mathbf{u}$	状态、输入向量	`x`、`u`	`model.py:49-50`
$\mathbf{x}_0$	本拍初值	`x0`	`node_core.py:539-552`
$\mathbf{x}_{\mathrm{ref}}, \mathbf{u}_{\mathrm{ref}}$	参考状态、参考输入	`x_ref`、`u_ref`	`node_core.py:646-776`
Q, Q_e, R	代价权重矩阵	`q_diag`、`q_e_diag`、`r_diag`	`ocp.py:64-87`
$f_{\max}$	电池可持续推力上限，N	`params.f_max`	`params.py:96`、`181`
$t_{\max}$	推力归一化常数，N	`params.t_max`	`params.py:189-207`
T_{hover}	悬停所需总推力，N	`params.t_hover`	`params.py:184-187`
h	悬停归一化推力	`params.hover_thrust`	`params.py:72`、`171`

Q、Q_e、R 在代码里存成对角线元组，装配求解器时才乘 2 变成矩阵（`ocp.py:255-257`）。这个“乘 2”与 acados 的半系数有关，见8.5.2。

还有两处记号容易混。其一，代码里角速率的变量名是 `w`，而四元数的标量分量记作 q_w；`w` 不是 q_w。其二，第 5 章把总推力记作 T，本章按代码变量 `u[0]` 记作 F；t 只表示归一化推力 $F/t_{\max}$。读跨章公式时以这两个对照为准。

8.2 刚体动力学

8.2.1 平动

把飞机当成一个质量固定的刚体。世界系里，位置对时间的导数就是速度：

$$\dot{\mathbf{p}} = \mathbf{v}. \tag{8-2}$$

牛顿第二定律写成

$$m\dot{\mathbf{v}} = \mathbf{F}_{\mathrm{total}}. \tag{8-3}$$

作用在飞机上的力只有两项：螺旋桨推力与重力。推力在机体系里沿 +z，大小记为 F：

$$\mathbf{f}_{\mathrm{body}} = \begin{bmatrix} 0 \\ 0 \\ F \end{bmatrix}, \qquad \mathbf{g} = \begin{bmatrix} 0 \\ 0 \\ -9.81 \end{bmatrix}. \tag{8-4}$$

重力已经写成“单位质量受力”，所以式 (8-3) 除以质量后，重力项不再带 m。推力要先从机体系转到世界系，再做除法：

$$\dot{\mathbf{v}} = \frac{1}{m} R(q) \begin{bmatrix} 0 \\ 0 \\ F \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ -9.81 \end{bmatrix}. \tag{8-5}$$

式 (8-2) 与式 (8-5) 就是代码里的前六行：`force = [0,0,F]`、`a_thrust = v_dot_q(force, q) / mass`、`f_expl = [v; a_thrust + g; …]`（`model.py:177-182`）。g 在代码里写成 `[0,0,-9.81]`，注释明确标了“ENU：重力朝 −z”（`model.py:177`）。

这里没有阻力项。模型假设推力是唯一的非重力外力，空气阻力、桨叶挥舞、机身升力都不进模型。这个假设在低速悬停与 1 m/s 量级的轨迹上够用，但它不是“阻力很小所以可以忽略”的定量结论；第 10 章的实飞误差里包含这一项，不能全部归给参数失配。

8.2.2 转动：四元数运动学

转动部分要回答的是：机体以角速度 ω 相对世界系转动时，表示姿态的四元数 q 怎么随时间变化。设 $q(t)$ 是把机体系矢量旋到世界系的单位四元数，$R(q)$ 是它对应的旋转矩阵。四元数与旋转矩阵的关系是群同态：

$$R(q_1 \otimes q_2) = R(q_1) R(q_2). \tag{8-6}$$

在很短的时间 Δt 内，机体绕机体系瞬时轴转过 $\Delta\theta = \omega \Delta t$。这个微小转动对应的四元数是

$$\delta q \approx \begin{bmatrix} 1 \\ \frac{1}{2}\Delta\theta \end{bmatrix} = \begin{bmatrix} 1 \\ \frac{1}{2}\omega\Delta t \end{bmatrix}. \tag{8-7}$$

姿态的更新可以写成右乘：$q(t+\Delta t) = q(t) \otimes \delta q$。代入式 (8-6)，

$$R(q(t+\Delta t)) = R(q(t))\, R(\delta q) \approx R(q(t))\, (I + [\Delta\theta]_\times), \tag{8-8}$$

其中 $[\cdot]_\times$ 是叉乘矩阵。两边减去 $R(q(t))$ 再除以 Δt，得到旋转矩阵的微分方程

$$\dot{R} = R\, [\omega]_\times. \tag{8-9}$$

四元数这边，把 $q(t+\Delta t) = q(t) \otimes \delta q$ 展开到一阶：

$$\dot{q} = \frac{1}{2}\, q \otimes \begin{bmatrix} 0 \\ \omega \end{bmatrix}. \tag{8-10}$$

这一步没有省掉中间代数：式 (8-10) 就是式 (8-9) 的四元数写法，右乘来自“微小转动作用在机体系”。写成 $q \otimes [0,\omega]^\mathsf{T}$ 而不是 $[0,\omega]^\mathsf{T} \otimes q$，是约定问题；两种写法对应不同的物理含义，差一个符号。

代码里的矩阵形式是

$$M(\omega) = \begin{bmatrix} 0 & -\omega_x & -\omega_y & -\omega_z \\ \omega_x & 0 & \omega_z & -\omega_y \\ \omega_y & -\omega_z & 0 & \omega_x \\ \omega_z & \omega_y & -\omega_x & 0 \end{bmatrix}, \qquad \dot{q} = \frac{1}{2} M(\omega) q. \tag{8-11}$$

把式 (8-10) 按四元数乘法逐项展开，并与式 (8-11) 对照，可以确认 $M(\omega)q = q \otimes [0,\omega]^{\mathsf{T}}$——也就是说，代码实现的是**右乘**。这一点在源码里容易读反：`skew_symmetric` 的 docstring 把它简写成 $\frac{1}{2}\,\mathrm{skew}(\omega)\otimes q$（`model.py:54-57`），但同一段紧接着说明“排布是为四元数右乘写的”。本章按实际实现写式 (8-10)；若按左乘展开，y、z 两个分量的符号会与代码相反。

逐分量写出，就是求解器实际积分的那四行：

$$\begin{aligned}\dot{q}_w &= -\tfrac{1}{2}(\omega_x q_x + \omega_y q_y + \omega_z q_z),\\ \dot{q}_x &= \tfrac{1}{2}(\omega_x q_w + \omega_y q_z - \omega_z q_y),\\ \dot{q}_y &= \tfrac{1}{2}(\omega_y q_w - \omega_x q_z + \omega_z q_x),\\ \dot{q}_z &= \tfrac{1}{2}(\omega_z q_w + \omega_x q_y - \omega_y q_x).\end{aligned} \tag{8-12}$$

式 (8-12) 与 `model.py:182` 的 `1/2 * mtimes(skew_symmetric(w), q)` 逐项一致。写成时另做过一次数值对拍：随机取单位四元数与角速率，`quat_mul(q, [0, ω])` 与 `M(ω)q` 的差在 10^{-15} 量级。这不是“公式看起来对”，而是两条独立的计算路径给出同一个结果；仓库里没有常驻的这条对拍用例，判据只能由写作者当场核。

8.2.3 三条建模假设及其边界

式 (8-5) 与式 (8-12) 不是无条件成立的。模型显式或隐式地用了表 34 里的假设；把它们与“参数不准”分开，是读第 10 章误差的前提。

表 34　建模假设与失效边界。

假设	写在哪里	真实系统里被什么破坏	参数扫描能否发现
内环瞬时：输入的角速率被当成实际角速度（指令即真值）	`model.py:182` 直接拿输入 `w` 进运动学	PX4 角速率环有有限带宽、延迟与饱和	不能；`rate/inertia` 旋钮只探动态品质
无气动阻力	式 (8-5) 没有阻力项	速度越高阻力越大	不能；它不是某个可调参数
推力沿机体 +z、质量常值	`model.py:178`、`params.py:168`	桨叶挥舞、机架变形、电池消耗	质量扫描只覆盖静态质量
四元数保持单位长度	模型没有 $\|q\| = 1$ 的约束	数值积分漂移	不能；漂移是模型精度指标

第一条最关键。q 的运动学里吃的是**指令角速率**；模型把它直接当成真实角速度，等于假设 PX4 的角速率环一收到指令就精确跟上。这个假设在 100 Hz 的外环与 1 kHz 的内环分层下是合理的工程近似，但它不是“参数误差”，无法靠把 `mass` 或 `hover_thrust` 调准来消除。13 状态模型要显式建模这条内环，见8.12。

第三条里的质量常值也需要说清楚：`Z0Params.mass` 是起飞质量，飞行中电池放电会让质量缓慢下降。这个变化远小于本文讨论的 $\pm 30\%$，但在长航时任务里不能一直当成零。

一句话总结：模型把内环当理想环节，并忽略阻力、变质量与非 +z 推力，这些是结构假设。

8.3 十状态模型

8.3.1 状态与输入

把式 (8-2)、式 (8-5)、式 (8-12) 拼成一个一阶系统。状态取

$$\mathbf{x} = \begin{bmatrix} \mathbf{p} \\ \mathbf{v} \\ q \end{bmatrix} \in \mathbb{R}^{10}, \qquad \mathbf{u} = \begin{bmatrix} F \\ \omega \end{bmatrix} \in \mathbb{R}^{4}. \tag{8-13}$$

完整的连续动力学是

$$\dot{\mathbf{x}} = f(\mathbf{x}, \mathbf{u}) = \begin{bmatrix} \mathbf{v} \\ \frac{1}{m} R(q) \begin{bmatrix} 0 \\ 0 \\ F \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ -9.81 \end{bmatrix} \\ \frac{1}{2}\, q \otimes \begin{bmatrix} 0 \\ \omega \end{bmatrix} \end{bmatrix}. \tag{8-14}$$

代码里的维度断言是 `NX = 10`、`NU = 4`（`model.py:49-50`），状态切片 `Q_SLICE = slice(6, 10)` 对应四元数（`model.py:51`）。式 (8-14) 的右端就是 `f_expl`（`model.py:182`），`dynamics()` 与 `step_rk4()` 都调用同一个 CasADi 表达式（`model.py:194-208`）。

状态里没有 ω。角速率是输入，不是状态；这是“内环理想”假设的直接后果，也是十状态与十三状态的分界。十三状态版本会再加三个状态与一行 $\dot{\omega} = k_\omega(\omega_{\text{des}} - \omega)$，但那需要先辨识 k_ω，当前没有做。

一句话总结：模型状态是位置、速度、四元数共十维，输入是推力与三轴角速率共四维。

8.3.2 输入为什么是“角速率指令”而不是力矩

输入选 ω 而不是力矩，是接口决定模型的典型例子。z0 采用 (C) 级架构：控制器只下到角速率，PX4 保留 1 kHz 角速率环、控制分配与混控。模型如果以力矩为输入，就必须知道转动惯量、电机时间常数与分配矩阵；这些量在 z0 上没有全部辨识，而且一旦换桨或换电池就会变。以角速率为输入，模型把内环当成一个已经调好的执行器，只负责外环的预测。

这也是参考实现 `px4-mpc` 的选择（`docs/control/phase2-implementation.md` §2.1），并与 `Sun2022` §III.A、`Foehn2022` §D.1 的分层一致：论文里的高带宽内环承担姿态跟踪，外环只规划推力与角速率。z0 没有实现论文里的 INDI 内环，PX4 的角速率环就是这里的“内环”。

这个选择的代价写在表 34 第一行：模型看不到内环的延迟与饱和。收益是模型维度低、求解快，且在 (C) 级接口上不需要额外的执行器辨识。

Sun2022 是本项目控制器内（PID 角度环）/外环（MPC）的选型依据。

8.3.3 这套控制能不能称为 NMPC

本章标题用“MPC”作统称。按预测模型与优化问题的性质，z0 这套控制器属于**非线性 MPC（NMPC）**，不是线性 MPC。

判据在模型，不在求解器。式 (8-14) 右端有两处非线性：推力先经 $R(q)$ 旋转，四元数导数含 $q \otimes [0, \omega]$ 的双线性项（`model.py:180-182`）。线性 MPC 要求预测模型能写成 $\mathbf{x}_{k+1} = A\mathbf{x}_k + B\mathbf{u}_k$，这里不存在这样的常数矩阵 A, B。

求解器的内部线性化不改变分类。acados 把问题装配成非线性 OCP（`NONLINEAR_LS`，`ocp.py:258-263`），`SQP_RTI` 每拍做一次 SQP 迭代（`ocp.py:287-293`）；每步迭代都要线性化，这是求解非线性问题的常规手段，不等于对被控对象做线性化。术语表把 NMPC 与线性 MPC 区分开（CONTEXT.md），Phase 4 的接口表也把 `mpc_node` 直接写作非线性 MPC 控制器（15-trajectory-flight-roles.md）。

需要保留的边界：内环理想、无阻力、十状态都是**结构近似**，不是线性化；称 NMPC 不表示模型更精确，也不表示每拍都收敛到最优解。

一句话总结：预测模型含姿态旋转与四元数双线性项，这套控制属于非线性 MPC。

8.4 离散化

8.4.1 连续模型的离散形式

求解器不能直接处理连续微分方程，要在时域上离散。取步长

$$\Delta t = \frac{T_f}{N} = \frac{1.0}{50} = 0.02\ \text{s}, \tag{8-15}$$

并假设每一步内输入不变（零阶保持）。离散动力学写成

$$\mathbf{x}_{k+1} = f_d(\mathbf{x}_k, \mathbf{u}_k), \qquad k = 0, 1, \dots, N-1. \tag{8-16}$$

$N = 50$、$T_f = 1.0$ 写在 `OcpConfig` 里（`ocp.py:57-58`），Δt 由 `tf / N` 算出（`ocp.py:424-427`）。50 步、20 ms 的组合意味着预测窗口覆盖 1 s；这个长度要大于外环的主要时间常数，又要短到模型误差不会把远端预测毁掉。它是工程选择，不是从动力学推出来的。

式 (8-16) 里的 f_d 不是解析矩阵指数，而是数值积分。acados 在每一步内部用显式 Runge–Kutta 把式 (8-14) 从 t_k 积到 t_{k+1}；节点与测试看到的离散模型就是 “积分器 + 连续右端” 的组合。

一句话总结：连续模型按 20 ms 步长离散成 50 步，覆盖 1 s 时域，由求解器内部积分。

8.4.2 RK4 与 ERK 的分工

`model.py` 里有一个 `step_rk4`：

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \frac{\Delta t}{6}\left(k_1 + 2k_2 + 2k_3 + k_4\right), \tag{8-17}$$

其中 $k_1 = f(\mathbf{x}_k, \mathbf{u}_k)$、$k_2 = f(\mathbf{x}_k + \frac{\Delta t}{2}k_1, \mathbf{u}_k)$，$k_3$、$k_4$ 同理（`model.py:198-208`）。这个函数只给测试与闭环数值仿真用，源码注释明确写了 “**不是** OCP 里的积分器”。

最优控制问题内部用的是 acados 的 ERK 积分器，配置为 4 级、3 个子步（`ocp.py:289-292` 的 `sim_method_num_stages = 4`、`sim_method_num_steps = 3`）。两者都是显式 Runge–Kutta 家族，但不是同一个实现：`step_rk4` 是单步四级的定步长 RK4，ERK 是在每个 20 ms 区间里再分 3 个子步的显式积分。把 `step_rk4` 的公式当成 OCP 的离散化写进论文，会把 “测试工具” 误写成 “求解器配置”；本章把这两件事分开。

一句话总结：acados 的四级三步 ERK 大幅优化求解器计算实机。

8.5 最优控制问题

8.5.1 一般形式

每一拍，控制器解一个有限时域最优控制问题：

$$
\begin{aligned}
\min_{\mathbf{u}_0,\dots,\mathbf{u}_{N-1}} \quad & J(\mathbf{x}_{0:N}, \mathbf{u}_{0:N-1}) \\
\text{s.t.} \quad & \mathbf{x}_{k+1} = f_d(\mathbf{x}_k, \mathbf{u}_k), \quad k = 0, \dots, N-1, \\
& \mathbf{x}_0 = \hat{\mathbf{x}}, \\
& \mathbf{u}_k \in \mathcal{U}, \quad k = 0, \dots, N-1.
\end{aligned}
\tag{8-18}
$$

$\hat{\mathbf{x}}$ 是本拍的状态估计。它不是直接取最近一帧动捕，而是把位置按速度外推一个固定延迟 $\tau = 50$ ms：

$$
\hat{\mathbf{p}} = \mathbf{p}_{\text{mocap}} + \mathbf{v}_{\text{mocap}}\tau, \qquad \hat{\mathbf{v}} = \mathbf{v}_{\text{mocap}}, \qquad \hat{q} = q_{\text{mocap}}. \tag{8-19}
$$

`extrapolate_state` 只外推位置，速度与四元数原样保留（`node_core.py:539-552`），默认 $\tau = 50$ ms（`node_core.py:110` 的 `TAU_DEFAULT_S`，经 `ControlConfig.tau_s` 下发，`node_core.py:309-337`）。这个外推补偿的是动捕到控制器的端到端延迟；它没有补偿姿态，因为角速率积分需要陀螺仪，而动捕链路没有给角速率。

式 (8-18) 的初值等式在求解器里不是软约束，而是把第 0 级的上下界都设成同一个值：`solver.set(0, "lbx", x0)` 与 `"ubx"`（`ocp.py:339-340`）。因此 $\mathbf{x}_0$ 不参与优化，只参与代价计算。

一句话总结：优化问题在动力学、初值等式与输入盒约束下最小化有限时域代价。

8.5.2 代价函数与权重

代价采用 acados 的 `NONLINEAR_LS` 形式。最小二乘残差取 $\mathbf{y} = [\mathbf{x}; \mathbf{u}]$，终端残差只取 $\mathbf{x}$：

$$
\mathbf{y}_k = \begin{bmatrix} \mathbf{x}_k \\ \mathbf{u}_k \end{bmatrix}, \qquad \mathbf{y}_e = \mathbf{x}_N. \tag{8-20}
$$

代价写成

$$
J = \frac{1}{2} \sum_{k=0}^{N-1} \left\| \mathbf{y}_k - \mathbf{y}_{\text{ref},k} \right\|_W^2 + \frac{1}{2} \left\| \mathbf{x}_N - \mathbf{x}_{\text{ref},N} \right\|_{W_e}^2, \tag{8-21}
$$

其中 $W = \text{diag}(2Q, 2R)$、$W_e = 2Q_e$。acados 的 `NONLINEAR_LS` 定义里带 $\frac{1}{2}$，装配时把权重乘 2（`ocp.py:255-261`），两者正好抵消，所以按对角线权重手算的代价就是

$$
J = \sum_{k=0}^{N-1} \left(\sum_{i=0}^{9} q_i \, e_{x,k,i}^2 + \sum_{j=0}^{3} r_j \, e_{u,k,j}^2 \right) + \sum_{i=0}^{9} q_{e,i} \, e_{x,N,i}^2. \tag{8-22}
$$

`OcpConfig.cost()` 就是按式 (8-22) 实现的（`ocp.py:93-110`）。它单独存在的原因见8.7.1：`SQP_RTI` 下求解器自报的代价不可信。

表 35 给出三组对角线权重与它们的作用。

表 35　代价权重（`ocp.py:64-87`）。

组	顺序	取值	含义
`q_diag`	$[p(3), v(3), q(4)]$	$(10, 10, 10,\ 10, 10, 10,\ 0.1, 2, 2, 2)$	阶段状态代价
`q_e_diag`	同上	$(300, 300, 300,\ 100, 100, 100,\ 0, 0, 0, 0)$	终端状态代价

组	顺序	取值	含义
r_diag	$[F, \omega_x, \omega_y, \omega_z]$	$(10,\ 500, 500, 500)$	输入代价

位置与速度。阶段权重都是 10，终端位置 300、终端速度 100。终端位置是阶段的 30 倍、终端速度是 10 倍，作用是把误差往终点压；终端姿态权重为 0，终点只约束位置与速度。

姿态。阶段四元数段是 $(0.1, 2, 2, 2)$，终端是 0。$q_w = 0.1$ 不是随手填的：若 $q_w = 0$，参考航向 ψ 与它的镜像 $2\pi - \psi$ 只差 w 的符号，代价对两者完全相同，偏航就成了一条死通道。旧权重下实测“参考朝向与镜像朝向的代价都恰好为 0”；把 q_w 提到 0.1 后，镜像代价变成有限值，这条死通道才被打开（ocp.py:60-67、phase2 票 16）。$q_{\text{att}} = 2.0$ 是相对参考实现的 0.1 提高的，方向是增加姿态误差的代价，不是“调参调出来的最优值”。

终端四元数段仍是 0，这是照抄参考实现。镜像井由阶段项消掉，终端不再重复罚姿态；代价是终点没有显式的姿态收敛要求，姿态靠阶段项与动力学传递。

输入。推力权重 10，三轴角速率权重都是 500。角速率权重大，是为了压指令抖动；它并不改变约束，只改变解在可行域里的位置。

三组权重的绝对值照搬参考实现，z0 只改过四元数段（offline-ocp.md §2）；本章不把 10、300、100 这些绝对值包装成推导结果。能讲出理由的是相对大小：终端高于阶段（把误差往终点压），角速率高于推力（压抖动），阶段姿态略高于 0（打开偏航通道）。

关于偏航权重 500，有一条容易被误读的实测：2026-09-29 曾把 ω_z 的权重降到 50（“方案 E”），当天撤回。原因不是“50 不稳定”，而是 10 m 不可行步进下 RTI 迭代收敛极慢：逐拍变化按几何衰减，60 拍的探针里看起来像逐拍乱跳，跑到 400 拍才衰减到 2.6×10^{-4}；权重 500 在同一场景第 50 拍就精确收敛。但也不能反过来说“500 更稳”：同一张扫描里 500 在 6 m（ω_x）与 7 m（F）上各有真极限环，50 在 3/4/5/6 m 上也有。权重是代价，不是增益旋钮；稳定区逐工况变，见 ocp.py:68-87 与 tools/rti-convergence/ 的读数。

偏航通道仍然弱。$q_w = 0.1$、$q_{\text{att}} = 2.0$ 只解决了“代价分不出镜像”，没有把偏航变成高权威通道。0.4 rad 的机头误差在 ω_z 权重 500 下只换来约 3.9×10^{-4} rad/s 的稳态指令，时间常数在 100–1000 s 量级；这不是本章权重能解决的问题，票 16 的收口记录把它留作已知限制。

一句话总结：代价用状态与输入相对参考的加权平方和，终端只罚位置与速度。

8.5.3 参考与前馈

参考状态由 node_core 生成，分三档：保持、轨迹、绝对目标。三档的共同点是姿态都取“水平 + 纯偏航”，位置按参考自己的速度在时域上线性外推：

$$\mathbf{p}_{\text{ref}}(t_k) = \mathbf{p}_{\text{anchor}} + \mathbf{p}_{\text{rel}} + \mathbf{v}\, t_k, \qquad \mathbf{v}_{\text{ref}} = \mathbf{v}, \qquad q_{\text{ref}} = q_{\text{yaw}}(\psi). \tag{8-23}$$

traj_reference 先把基准状态铺满 $N+1$ 个点，再给位置加上 $\tau_k \mathbf{v}$（node_core.py:646-684）；速度与姿态逐点相同。参考输入固定为

$$\mathbf{u}_{\text{ref}} = \begin{bmatrix} mg \\ 0 \\ 0 \\ 0 \end{bmatrix}, \tag{8-24}$$

由 hold_feedforward 返回（node_core.py:713-776）。

式 (8-24) 不是“悬停时随便填的数”，它让参考自己成为模型的一条精确解。把式 (8-23)、式 (8-24) 代入式 (8-14)：q 常量、$\omega = 0$ 给出 $\dot{q} = 0$；推力项 $R(q_{\text{yaw}})[0, 0, mg]^{\mathsf{T}}/m = [0, 0, g]^{\mathsf{T}}$，与重力抵消，给出 $\dot{\mathbf{v}} = 0$；而 $\dot{\mathbf{p}} = \mathbf{v}$ 正是式 (8-23) 的时间导数。于是

$$f(\mathbf{x}_{\text{ref}}, \mathbf{u}_{\text{ref}}) = \dot{\mathbf{x}}_{\text{ref}}. \tag{8-25}$$

这条等式对“保持”与“轨迹”两档都成立，判据是 `test_node_core.py::test_` 轨迹参考是模型的一条精确解（拿 `step_rk4` 逐步往前推）。它成立的前提是参考族只有匀速平动与常量偏航；一旦参考带持续加速度或持续角速率，式 (8-25) 立刻失效。

$\mathbf{u}_{\text{ref}}$ 是代价参考，不是指令。真正发出去的是求解器输出的 $\mathbf{u}_0$。这一点在垂直通道上被实测放大过：把质量失配的积分偏置加进 $\mathbf{u}_{\text{ref}}$，实测灵敏度

$$\frac{\partial u_0[0]}{\partial u_{\text{ref}}[0]} = -0.0616\ \text{N/N} \tag{8-26}$$

是负的，积分作用会变成正反馈：越沉，偏置越大，解出的推力反而越小。该落点已被证伪并删除（`node_core.py:713-776` 的 ☒ 段）。现在的垂直偏置加在**发出去的指令**上：

$$u_{\text{cmd}}[0] = \text{clip}\left(u_0[0] + \Delta F,\ 0,\ f_{\max}\right), \tag{8-27}$$

ΔF 是纯积分状态，$e_z = z_{\text{ref}} - z_{\text{meas}}$，增益 $k_i = 1\ \text{N}/(\text{m}\cdot\text{s})$，状态夹在 $[-T_{\text{hover}}, f_{\max} - T_{\text{hover}}]$ 内（`mpc_node.py:1094-1120`、`1241`、`484`）。夹状态是为了“退得回来”，夹输出是为了“不越界”，两者不能合并。式 (8-27) 不属于优化问题，它是发布前的附加项；本章把它写在这里，是为了让读者知道十状态模型之外还有一层慢积分。

一句话总结：参考前馈取悬停推力与零角速率，参考本身因此是模型的一条精确解。

8.6 约束

8.6.1 推力上下限

优化问题对输入只有盒约束。推力那一路是

$$0 \le F \le f_{\max} = 11.8\ \text{N}. \tag{8-28}$$

代码里写成 `lbu = [0, -ω_max]`、`ubu = [f_max, ω_max]`、`idxbu = [0,1,2,3]`（`ocp.py:272-278`）。下界取 0，是因为 FLU 下推力沿机体 +z，反向推力不是这台飞机的控制量；负推力会让模型规划出“吸住”飞机的解，而 PX4 的混控器兑现不了。

上界 $f_{\max} = 11.8$ N 是**电池可持续**口径：VTC6 持续放电 15 A，按 22.2 V 与台架效率外推，四电机合计约 11.8 N（`params.py:83-96`、`docs/report/chapters/03-hardware.md` §3.4）。它不是电机与桨的满量程。

$f_{\max}$ 与 $t_{\max}$ 是两个量。归一化常数

$$t_{\max} = \frac{T_{\text{hover}}}{h} = \frac{mg}{0.1589} = 46.3027\ \text{N} \tag{8-29}$$

只负责把牛顿换算成 PX4 的归一化推力；它由悬停点反推，是满电口径的满量程。$t_{\max}$ 是 $f_{\max}$ 的 3.92 倍。把 $t_{\max}$ 当 F 的上界，求解器会规划出电池兑现不了的推力，而且不会报错：节点照参数表飞、测试按同一个模型算预期，可能全绿而飞机掉高。这个坑真实踩过——回填 `hover_thrust` 后 `ocp.py` 的 `F ≤ T_max` 悄悄从“按电池”漂成“按电机”，最后是“又远又高的目标该把推力顶到上限”那条测试以 `38.515 == 46.303` 红了一次（`docs/control/phase2-implementation.md` §3.3.3）。

一句话总结：推力下界为零、上界取电池可持续的 11.8 N，归一化常数不参与约束。

8.6.2 角速率上下限

三轴角速率约束是

$$-3.84 \le \omega_x \le 3.84, \qquad -3.84 \le \omega_y \le 3.84, \qquad -6.28 \le \omega_z \le 6.28 \ \mathrm{rad/s}. \tag{8-30}$$

取值来自飞控参数：`MC_ROLLRATE_MAX` 与 `MC_PITCHRATE_MAX` 是 220 °/s，换算为 3.8397 rad/s；`MC_YAWRATE_MAX` 是 360 °/s,换算为 6.2832 rad/s。代码取三位有效数字 3.84 与 6.28(`params.py:74-81`)。

偏航上限比横滚、俯仰更大，这是四旋翼的常见配置，也是参考实现写错的地方之一：参考实现把偏航写成横滚的一半，z0 改成独立给（`ocp.py:277-280`）。约束加在**指令**上，不是实测角速率上；PX4 的角速率环可能饱和或滞后，模型对此没有建模。

一句话总结：角速率约束取飞控参数换算后的横滚俯仰 3.84、偏航 6.28 rad/s。

8.6.3 没有实现的约束

当前 OCP 的约束只有式 (8-28) 与式 (8-30) 的输入盒，加上式 (8-18) 的初值等式。以下约束**没有实现**：

- **倾角约束**。路线图 §4.3 规划了 $R(q)\mathbf{e}_z \cdot \mathbf{e}_z \ge \cos 45^\circ$，但 `ocp.py` 里只有 `lbu/ubu`，没有任何状态约束。当前模型可以规划出任意倾角，只要推力与角速率在盒内。
- **位置、速度硬约束**。位置与速度只进代价，不是硬约束。`MPC_XY_VEL_MAX=2.0` 是 PX4 位置控制器的软限速，不是这个优化问题的约束。
- **四元数单位长度约束**。模型没有 $\|q\| = 1$ 的等式约束。正常时漂移在 10^{-8} 量级，涨起来才是轨迹发散的信号；它不进 `degraded` 判据（`docs/control/offline-ocp.md` §4.3）。
- **终端约束集与障碍物约束**。都没有。

没有倾角约束的后果不是“飞机一定会翻”，而是“模型不会主动阻止大倾角”。实际飞行里先撞上的是 $f_{\max}$ 与角速率上限：45° 悬停所需推力约 $mg/\cos 45^\circ = 10.4$ N，已经占 $f_{\max}$ 的 88%。这说明推力上限间接限制了可持续倾角，但它不是几何硬约束，瞬时解仍可能规划出更大的倾角。把 45° 写成现状，是本章明确避免的错误。

一句话总结：倾角、位置与四元数范数约束都未实现，当前只有输入盒约束。

8.7 求解器

8.7.1 acados 配置与实时迭代语义

表 36 是 `_build_solver()` 实际下发的配置（`ocp.py:243-302`）。

表 36　求解器配置。

项	取值	出处
预测步数 N	50	`ocp.py:57`
时域 T_f	1.0 s	`ocp.py:58`
代价	`NONLINEAR_LS`，阶段与终端	`ocp.py:258-263`
NLP 求解器	`SQP_RTI`	`ocp.py:290`
QP 求解器	`PARTIAL_CONDENSING_HPIPM`	`ocp.py:287`
Hessian 近似	`GAUSS_NEWTON`	`ocp.py:288`
积分器	`ERK`，4 级 3 步	`ocp.py:289-292`
部分凝聚步数	`qp_solver_cond_N = N`	`ocp.py:293`

`SQP_RTI` 是实时迭代（real-time iteration）：它把 SQP 的一步拆成“准备”和“反馈”两半，每个控制周期只做一次迭代，用上一周期的解作为热启动。它追求的是**低延迟**，不是每拍的最优性。单

次迭代的解是迭代映射的一步，不是收敛解；第一拍用的是求解器的出厂初值，不能当控制指令。实测中目标 +x 10 m、全新实例的第 0 次 solve 给出 $\omega_y = 0$、$F = 7.8111$ N，与悬停初值一样，从第 1 次起才开始动（`docs/control/offline-ocp.md` §4.1）。

这条语义解释了节点里的两个设计：预热 5 拍不发（`WARMUP_CYCLES=5`，`node_core.py:127`），以及测试必须先热身再断言（`conftest.py` 的 `solve_warm`）。它也是 `SQP_RTI` 下三条原厂读数不可信的原因：残差恒为 0、`alpha` 直接抛异常、`get_cost()` 与返回轨迹的代价差两个数量级。节点因此自己按式 (8-22) 重算代价，不读 `solver.get_cost()`。

为什么选这套组合：部分凝聚把长时域 QP 的变量规模压下来，与 10 状态、50 步的规模匹配；Gauss–Newton 用雅可比近似 Hessian，牺牲一点收敛速度换每拍耗时；ERK 对这类非刚性动力学足够，且显式积分不需要线性求解。这些是照着参考实现与 acados 的常见配置选的，不是本章推导出来的。

一句话总结：求解器每周期只做一次实时迭代，因此单拍解是迭代步而不是最优解。

8.7.2 实时预算与实测耗时

控制器的目标频率是 100 Hz（`mpc_node.py:444` 的 `rate_hz = 100.0`），也就是每拍 10 ms。表 37 是三趟 2026-10-17 仿真闭环的求解耗时，数据来自 DUT 的 JSONL，`solve_ms` 取 acados 自报的 `time_tot`（`mpc_node.py:1385`）。

表 37　三趟仿真的求解耗时（ms）。

场景	有效求解帧	均值	p50	p95	稳态最坏	预热首解	退化帧	求解失败	饱和帧
`R-hover`	7797	3.127	3.154	3.624	20.138	11.44	0	0	0
`R-square`	5679	2.985	3.071	3.313	6.462	12.916	0	0	0
`R-lemniscate`	6572	3.133	3.160	3.613	10.738	9.435	0	0	0

读法是：中位约 3.1 ms，p95 约 3.3–3.6 ms，占 10 ms 预算的三分之一；稳态最坏在 6.5–20.1 ms 之间，偶尔超过一拍预算。三趟的退化帧、求解失败帧与饱和帧都是 0。

“最坏超过 10 ms” 不等于 “丢帧”。节点跑在 ROS 定时器上，不是硬实时系统；一次超时会让这一拍晚到，下一拍继续，上一帧 setpoint 仍被 PX4 锁存。真正要担心的是持续超时把有效频率拉低，而不是单次 20 ms。表里 `R-hover` 的 20.138 ms 是稳态窗口里的单帧；这三个汇总只给最大值，回答不了 “它是否连续出现”。`R-square` 的 12.916 ms 落在预热首解，稳态最坏只有 6.462 ms；`R-lemniscate` 的 10.738 ms 也在稳态窗口内。

这些是**仿真**读数，不是真机保证。仿真跑在机载电脑上，被控对象是 SIH，没有真实电调的电磁干扰、没有动捕 WiFi 的抖动，也没有并行会话抢 CPU。真机的耗时证据应在第 10 章按同一口径给出；本章不把仿真最坏值写成 “实飞最坏值”。

一句话总结：三趟仿真的求解中位约 3 ms、p95 约 3.6 ms，未出现求解失败或退化。

8.8 应用到 PX4

8.8.1 CTBR 接口

解出来的第一拍是 $\mathbf{u}_0 = [F, \omega_x, \omega_y, \omega_z]$。它通过 `mavros_msgs/AttitudeTarget` 发到 `/mavros/setpoint_raw/attitude`，字段映射是

$$\text{body_rate}_x = \omega_x, \quad \text{body_rate}_y = \omega_y, \quad \text{body_rate}_z = \omega_z, \quad \text{thrust} = \frac{F}{t_{\max}}, \tag{8-31}$$

`type_mask=128`，即忽略姿态四元数，只使用角速率与推力（`mpc_node.py:411-418`、`1296-1306`）。这就是 CTBR（collective thrust + body rate）：总推力加机体三轴角速率。

PX4 收到后保留 1 kHz 角速率环与控制分配（`docs/control/mpc-roadmap.md:330`），把角速率跟踪成电机指令。位置环与姿态环由 MPC 承担；这是 (C) 级接口的定义。`AttitudeTarget` 的 `orientation` 字段不填，因为掩码已经声明忽略它；填了也不会被使用，但会在日志里制造一个看似有意义的姿态量。

只有正常帧更新 `_last_sp`（`mpc_node.py:1304-1305`）。兜底帧与正常帧走同一个发布函数、同样的字段，区别只在内容来自兜底阶梯；这样“保持”在话题上是可观测的（同一组数值连续出现，时间戳在前进），而不是“节点什么都不发”的黑箱。

一句话总结：下行只发总推力与机体三轴角速率，姿态四元数由掩码忽略。

8.8.2 ENU/FLU 到 FRD 的符号映射

FLU 与 FRD 的关系是绕机体 x 轴转 180°：

$$\mathbf{e}_x^{\mathrm{FRD}} = \mathbf{e}_x^{\mathrm{FLU}}, \qquad \mathbf{e}_y^{\mathrm{FRD}} = -\mathbf{e}_y^{\mathrm{FLU}}, \qquad \mathbf{e}_z^{\mathrm{FRD}} = -\mathbf{e}_z^{\mathrm{FLU}}. \tag{8-32}$$

这不是镜像：对角矩阵 $\mathrm{diag}(1,-1,-1)$ 的行列式是 $+1$，它是一个真旋转。把同一个角速度矢量投影到两套基上，得到

$$\omega_x^{\mathrm{FRD}} = \omega_x^{\mathrm{FLU}}, \qquad \omega_y^{\mathrm{FRD}} = -\omega_y^{\mathrm{FLU}}, \qquad \omega_z^{\mathrm{FRD}} = -\omega_z^{\mathrm{FLU}}. \tag{8-33}$$

也就是“roll 不变、pitch 与偏航取反”，没有额外的轴矢量负号。推力同理：物理推力是 $F\mathbf{e}_z^{\mathrm{FLU}}$，它在 FRD 的 z 轴上的分量是 $-F$，归一化后

$$\mathrm{thrust_body}_z = -\frac{F}{t_{\max}}. \tag{8-34}$$

表 38 给出端到端的所有权。关键点是：我们发出的 `body_rate` 是 FLU 原样、`thrust` 是正号；FLU→FRD 由 mavros 在 `setpoint_raw.cpp:323-324` 用 $R_x(\pi)$ 做一次；推力的负号由飞控的 `fill_thrust` 在 `mavlink_receiver.cpp:1475` 做一次。

表 38　CTBR 的端到端符号映射。

量	我们发出的字段	mavros	PX4 最终字段
roll 角速率	`body_rate.x = +ωx`	FLU→FRD，x 不变	`roll = +ωx`
pitch 角速率	`body_rate.y = +ωy`	FLU→FRD，y 取反	`pitch = −ωy`
yaw 角速率	`body_rate.z = +ωz`	FLU→FRD，z 取反	`yaw = −ωz`
总推力	`thrust = +F/t_max`	不转换	`thrust_body_z = −F/t_max`

不要在节点里抢先转一次。如果我们在 `mavros_attitude_target` 里先把 FLU 转成 FRD，mavros 会再转一次，两次 $R_x(\pi)$ 互相抵消，飞控最终收到的数值与写对时完全一样。症状极隐蔽：roll 不受影响，pitch 与偏航静默反向。所以断言不能落在“PX4 收到了什么”上，只能落在我们发出的字段上（`sign_map.py:126-154`、`docs/control/sign-map.md` §5）。想复现这一档，可以用 `offboard_smoke.py --break pitch-preconvert`。

一句话总结：FLU 到 FRD 是绕机体 x 轴转 180°，角速率取反两根轴，推力在飞控侧取负。

8.8.3 推力归一化链

PX4 的 offboard 角速率分支对 `thrust_body` 不做重标定，而是原样交给混控器（`Multicopter-RateControl.cpp:182`；`MC_BAT_SCALE_EN=0`）。因此模型里的牛顿值必须经过一个固定常数换成归一化推力：

$$t = \frac{F}{t_{\max}}, \qquad t_{\max} = \frac{T_{\text{hover}}}{h} = \frac{mg}{h}. \tag{8-35}$$

代入 z0 的数：$T_{\text{hover}} = 0.75 \times 9.81 = 7.3575$ N，$h = 0.1589$，得到 $t_{\max} = 46.3027$ N（`params.py:184-207`）。悬停时 $F = T_{\text{hover}}$，于是

$$t_{\text{hover}} = \frac{T_{\text{hover}}}{t_{\max}} = h = 0.1589. \tag{8-36}$$

式 (8-36) 就是自洽条件：`hover_thrust` 与 $1/t_{\max}$ 必须出自同一次测量，因为它们是同一条等式的两端。代码用 `t_max = m*g/hover_thrust` 把两者锁死，并用 `check_hover_consistency` 在启动时检查（`params.py:130-160`）。若一边填实测 h、一边填一个独立取的 $t_{\max}$，悬停前馈会整体偏，而求解器不会报错。

`MPC_THR_HOVER` 不在这条链上。它是 PX4 位置控制器与悬停推力估计器的参数，z0 现值仍是默认的 0.5，不是实测的 0.1589。在 (C) 级接口下，我们发的是 `thrust_body`，不经过 PX4 位置控制器的悬停油门重标定；把 `hover_thrust` 填成 0.5 会让归一化链整体错约 3.15 倍（$0.5/0.1589$）；`docs/control/phase2-implementation.md` §3.3.2 把这一项记作 3.9 倍；3.9 与 $t_{\max}/f_{\max} = 3.92$ 同值，不是这里的悬停油门比，按 $t_{\max} = mg/h$ 现算应为 3.15。两者的名字撞脸，但消费者不同。

链上还有一个运行时前置：mavros 的 `thrust_scaling` 必须是 1.0。缺失时 `thrust != 0` 的帧会被整帧丢弃，值为 0 时推力恒为 0；两种都是静默的。`preflight_check` 会读活着的 mavros 节点上的这个参数，而不是读启动文件（`preflight_check.py:365-462`）。

一句话总结：PX4 飞控不能精准控制推力大小，只能通过拟合油门-推力曲线，来通过控制油门从而控制推力

8.9 参考姿态为什么是水平 + 纯偏航

参考姿态取水平加纯偏航，不是为了让图好看，而是因为这一族参考在无阻力模型里是精确解。

设参考位置按常量速度前进，参考姿态是常量偏航：

$$\mathbf{p}_{\text{ref}}(t) = \mathbf{p}_0 + \mathbf{v}t, \qquad \mathbf{v}_{\text{ref}} = \mathbf{v}, \qquad q_{\text{ref}} = q_{\text{yaw}}(\psi). \tag{8-37}$$

把式 (8-37) 与式 (8-24) 代入式 (8-14)：

$$\dot{\mathbf{v}}_{\text{ref}} = \frac{1}{m} R(q_{\text{yaw}}) \begin{bmatrix} 0 \\ 0 \\ mg \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ -g \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ g \end{bmatrix} + \begin{bmatrix} 0 \\ 0 \\ -g \end{bmatrix} = \mathbf{0}. \tag{8-38}$$

同时 $\dot{q}_{\text{ref}} = 0$（因为 $\omega = 0$），$\dot{\mathbf{p}}_{\text{ref}} = \mathbf{v}$。三项都与参考自己的导数一致，所以式 (8-25) 成立。

物理上，这个结果只说了一件事：**倾角是产生水平加速度的手段，不是维持水平速度的手段**。在无阻力模型里，匀速平动不需要任何水平力，水平姿态加悬停推力就够了。真实飞机有阻力，维持匀速确实需要一点倾角；这一点被模型忽略，表现为稳态跟踪误差，而不是参考姿态写错。

`/traj/reference` 只给位置与速度,加速度字段在掩码里被忽略(`type_mask=2496`,`node_core.py:588-634`)。参考源没有加速度信息，控制器就算想算 “维持速度所需的倾角” 也无从算起；它只能选择与模型自洽的水平姿态。若将来解开加速度掩码，参考族就要换成带加速度的形式，`u_ref` 也要按参考的导数重算，式 (8-25) 不再对当前形式成立。

这与第 10 章姿态图的目标线是同一件事。第 10 章将画姿态误差；按这里写定的约定，roll 与 pitch 的参考线取 0，因为参考姿态是水平，偏航的参考线是该段恒定参考航向，误差目标同样是 0。图上的 0 不是 “实测姿态恰好为 0”，而是 “参考姿态的水平分量与当前航向就是比较基准”。

一句话总结：无阻力模型里匀速平动不需要倾角，所以参考姿态是水平加纯偏航。

8.10 模型失配与鲁棒性

8.10.1 参数失配的证据现状

讨论失配前必须先说证据的有效期。`docs/control/mismatch-sweep.md` 页首有一条 ☒：该页全部读数出自 2026-10-09 之前的旧同构被控对象，旧机型常数 `SIH_T_MAX = 2.95 N`（总 11.8 N），重搭后 `T_MAX_TOTAL = 46.3027 N`。因此那张 13 场扫描表、§3–§4 的结论以及 “等效刚度 $k = 3.57$ N/m” 全部作废。作废的是读数，不是方法：五个旋钮、夹具与判据层的设计仍然有效。

重搭之后，目前只有一场质量失配的证据：`tools/px4-sitl/evidence/tier-b/20261009-rebuild/T0-summary.json`。被控对象质量 0.975 kg（比控制器模型高 30%），悬停推力 9.56475 N；控制器模型仍是 0.75 kg、`hover_thrust=0.1589`。判定结果：

- 判据 ① 稳态窗口内，水平误差均值 0.0107 m、最大 0.0169 m；垂直误差均值 −0.00013 m、绝对均值 0.00035 m。
- 倾角最大 15.46°（起飞瞬态），稳态最大 0.55°。
- 求解稳态 p50 2.931 ms、最坏 5.932 ms；退化 0、求解失败 0、兜底保持 0、安全悬停 0。
- 全程未发散。

这一场说明：质量加三成，在这条轨迹、这个控制器参数、开启垂直偏置积分的条件下，稳态误差仍远小于 0.15 m 的判据界。它**不能**外推成 “所有 ±30% 参数都鲁棒”：推力 −30%、惯量、内环时间常数、角速率增益都没有重搭后的读数；旧机型上的读数已随页首 ☒ 作废，不再引用其结论。本章因此只写这一场的结论，不复制旧表的 “只有 mass 与 thrust 过线” 那句话。

一句话总结：代码通过质量加三成误差小于判据，来判断参数是否失配。

8.10.2 结构假设与参数误差不是一回事

表 39 把 “模型结构” 和 “模型参数” 分开。参数失配扫描只能扫右列，扫不到左列。

表 39　结构假设与参数误差。

类别	具体内容	能不能靠 ±30% 扫描发现
结构假设	内环瞬时、无阻力、推力沿 +z、无执行器动态、无电池电压跌落	不能
参数误差	质量、推力标度、惯量、角速率环增益、延迟	部分能；旧扫描覆盖过，重搭后只重跑过质量

“内环理想” 是最容易被误当成参数误差的一条。模型把 ω 当实际角速率，真实系统里它是指令，PX4 的角速率环有带宽、延迟与饱和。把 `rate` 旋钮调 ±30% 只能探 “内环增益变了以后闭环品质如何”，不能回答 “模型少了一个状态” 这个问题；后者的修法是十三状态模型加 k_ω，不是把某个参数调准。

工程上的鲁棒性来自三处：反馈本身、1 s 的短时域、以及保守的 $f_{\max}$。短时域让模型误差来不及把预测推得很远；反馈每 10 ms 修正一次；$f_{\max}$ 按电池而不是电机取值，避免规划出兑现不了的推力。这

些是设计上的余量，不是形式化的鲁棒性保证；本章不写 “对 ±30% 参数鲁棒” 这种没有边界的话。

一句话总结：参数失配扫描探不到结构误差，内环理想假设需要十三状态模型才能显式建模。

8.11 求解失败兜底

8.11.1 “可用” 与 “退化” 的判据

求解器返回后，`solution_usable` 先判这一拍能不能发（`node_core.py:516-537`）：状态码成功、解形状为 $(4,)$、四个分量全有限；任一不满足即不可用。

“退化” 与 “不可用” 分开：状态码非零、u 越界、代价或耗时非有限而解本身仍有限时，节点照样发、打警告、计数。合并两者，会让一次数值噪声触发不必要的姿态切换。

发布前再由 `frame_is_finite` 检查 `body_rate_x/y/z` 与 `thrust` 四个数（`node_core.py:490-514`）。顺序是 `solution_usable` → 垂直偏置 → `sign_map` 入参检查 → 整帧校验（`mpc_node.py:1206-1250`）。PX4 对角速率有 NaN 回退、对推力没有，最后一道不能省。

一句话总结：求解状态码异常、形状不对或含非有限值时判为不可用，不进入发布。

8.11.2 兜底阶梯

求解不可用时走三档阶梯（`FallbackLadder`，`node_core.py:1438-1526`）：解可用就发解；连续不可用不超过 10 拍，保持上一帧可行指令、只换时间戳；超过 10 拍，切悬停推力加零角速率并置事件标志。

不能一路保持：PX4 会无限期锁存最后一条 setpoint（`MulticopterRateControl.cpp:177-184`），“我们不发” 与 “飞控沿用旧帧” 是同一件事，陈旧指令可能正打满角速率。也不能一失败就悬停：10 拍在 100 Hz 下约 100 ms，够跨过单次超时，又不会让旧指令持续太久（`FALLBACK_HOLD_FRAMES=10`，`node_core.py:135`）。安全悬停不依赖状态；第一拍就没有 “上一帧” 时，`can_hold=False` 直接进悬停。

阶梯只处理 “求解不可用”，不处理 “闸门关闭”：未解锁、不在 OFFBOARD、状态或测量超时，节点不发帧、阶梯复位，PX4 在 `COM_OF_LOSS_T=1.0` s 后自行退出 Offboard（`docs/assets/z0.params:290`）。不抛异常是刻意的：进程崩掉后 PX4 仍锁存最后一条指令，切到已知安全指令比崩溃更可控。

一句话总结：不可用时先保持上一帧可行指令十拍，再切到不依赖状态的安全悬停。

8.12 已实现与规划中

表 40 把本章涉及的机制按 “已实现” 与 “规划中” 分开。这张表是本章的边界声明：正文里的公式只描述左列。

表 40　已实现与规划中的机制。

机制	状态	证据 / 规划出处
十状态模型 $[\mathbf{p}, \mathbf{v}, q]$、输入 $[F, \omega]$	已实现	`model.py:49-51、162-191`
四元数运动学右乘约定	已实现	`model.py:54-65、182`
50 步、1 s、20 ms 离散	已实现	`ocp.py:57-58、424-427`
`NONLINEAR_LS` 代价与三组权重	已实现	`ocp.py:64-87、255-263`
推力 $[0, f_{\max}]$ 与角速率盒约束	已实现	`ocp.py:272-278`
悬停前馈 $\mathbf{u}_{\text{ref}} = [mg, 0, 0, 0]$	已实现	`node_core.py:713-776`
发布前垂直偏置积分	已实现	`mpc_node.py:1094-1120、1241`
求解失败三档兜底	已实现	`node_core.py:1438-1526`

机制	状态	证据 / 规划出处
十三状态模型与 $\dot{\omega} = k_\omega(\omega_{\text{des}} - \omega)$	**未实现**	docs/control/mpc-roadmap.md §4.2; src/nmpc/ 无 k_omega 代码（仅 model.py:36 的方向声明）; k_ω 未辨识
45° 倾角约束	**未实现**	docs/control/mpc-roadmap.md §4.3; ocp.py 只有 lbu/ubu
INDI 内环	**未实现**	docs/control/mpc-roadmap.md §3.1; src/nmpc/ 无 INDI 实现
四元数单位长度约束	**未实现**	模型无该约束；漂移作精度指标
加速度参考与前馈导数	**未实现**	/traj/reference 的加速度掩码被忽略
位置、速度硬约束、障碍物约束	**未实现**	ocp.py 无状态约束

十三状态版本的公式在路线图 §4.2 里已经写出，但它不是现状：k_ω 需要给角速率阶跃、看 IMU 响应来辨识，这一步没有做。倾角约束同理，45° 只是规划值，当前 OCP 不会阻止模型规划更大的倾角。把这两项写成 “已实现”，会让第 10 章的误差解释失真。

一句话总结：已实现的是十状态模型与前馈参考，十三状态、倾角约束与 INDI 是未来可拓展工作。

8.13 本章引用清单

表 41 把本章的公式、常数与判据按主题归到仓库内的文件与论文。正文里已经随行给过 file:line，这里只是便于回查。

表 41　本章引用清单。

主题	文件 / 论文
十状态动力学、四元数工具、RK4	src/nmpc/nmpc/model.py
OCP 装配、代价、约束、求解器配置	src/nmpc/nmpc/ocp.py
质量、悬停推力、归一化常数、电池上限	src/nmpc/nmpc/params.py
参考生成、前馈、初值外推、兜底阶梯	src/nmpc/nmpc/node_core.py
发布路径、垂直偏置、消息字段	src/nmpc/nmpc/mpc_node.py
FLU/FRD 映射与推力符号	src/nmpc/nmpc/sign_map.py、docs/control/sign-map.md §5
离线求解器行为、RTI 首帧与代价读数	docs/control/offline-ocp.md §2、§4
规划中的十三状态、倾角约束、INDI	docs/control/mpc-roadmap.md §3–4
十状态来源与推力上限推导	docs/control/phase2-implementation.md §2.1、§3.3
偏航镜像井与权重讨论	phase2 票 16
失配扫描的作废声明与方法	docs/control/mismatch-sweep.md 页首、§1.2
质量加三成的重搭后证据	T0-summary.json
三趟仿真的求解耗时	R-hover、R-square、R-lemniscate
四旋翼动力学与 (C) 级分层	Sun2022 §III.A、Foehn2022 §D.1
实时迭代	Houska2011（Automatica 2011）

一句话总结：本章的公式、权重与判据都能回到源码、证据文件或论文。

第 9 章 实验设计与数据处理

本章说明第 10 章的数据从哪里来、怎样组织，以及怎样重跑。四个源——参考目标、offboard 实飞、MPC 仿真、MPC 实飞——来自六份话题包和三趟 SITL；本章给出四源的定义、包与场景的映射，以及复现命令。

全章只讲数据，不讲结果。四个源由同一套管线生成 100 Hz 长表；数据窗口、重采样与指标的具体口径以 `docs/report/data/README.md` 与 `tools/report/` 的实现为准。

话题、状态机与坐标系的分工见第 5、7 章；模型、参考姿态与控制器约定见第 8 章。本章不重复推导，只说明数据来源与组织方式。

9.1 数据组织

9.1.1 四个源

表 42 给出四个源的定义。目标与 offboard 来自同一份包：`/traj/reference` 是相对轨迹原点的参考，加上该包的原点才是绝对目标；offboard 的实际轨迹取 `/mavros/vision_pose/pose`。MPC 仿真取 SITL summary 的 `x0`，MPC 实飞取实飞包的 VP。

表 42　四个源。

source	是什么	数据	时间零点	轨迹原点	控制器反馈
`target`	参考目标	offboard 包 `/traj/reference`（相对量）+ 原点	同一份 offboard 包	同一份 offboard 包	无（目标不是测量）
`offboard`	PX4 内置 PID 实飞	`/mavros/vision_pose/pose`	同一份 offboard 包	`/mavros/setpoint_raw/local`	EKF2 位置估计（ROS 侧 LP，约 30 Hz）
`mpc_sim`	MPC 在 SITL 闭环	summary `frames[].x0`（合成动捕）	由墙钟反推（`tools/report/extract.py`）	`x_ref0`（前摇段）模型参考首点	SITL 合成 mocap
`mpc_real`	MPC 实飞	`/mavros/vision_pose/pose`	同一份实飞包	`/control/target`	`/mocap/pose_enu`（原始动捕）

一句话总结：四个源分别来自同场实飞包与 SITL 状态，是后续所有对比的输入。

9.1.2 三份产物

三份产物是 `docs/report/data/<scene>.csv`、同名的 `<scene>.json` 与数据字典 `docs/report/data/README`。CSV 是长表：一行一个源、一个网格点，列固定为 `source, t_s, x_m, y_m, z_m, x_abs_m, y_abs_m, z_abs_m, vx_mps, vy_mps, vz_mps, roll_rad, pitch_rad, yaw_rad, qw, qx, qy, qz, u_thrust_n, wx_radps, wy_radps, wz_radps, valid`。相对列 `x_m` 等已减各自原点；绝对列 `x_abs_m` 等保留原值，便于回包核对。`valid=0` 表示该网格点落在该源原始时间范围之外，值是端点保持的结果，不是新测到的样本。

JSON 存的是方法元数据：窗口、原点、完成原因、输入 sha256、参考发布缺口与逐源读数。CSV 只存网格上的数，不重复这些口径。换包或改窗口后，两份产物必须一起重生成；只重画图不会更新 JSON。

一句话总结：四个源统一落成一张 100 Hz 长表，相对列与绝对列并存，口径元数据在同名 JSON。

9.2 数据集

9.2.1 六份话题包与场景映射

三个场景各用两份包：一份 offboard 实飞包，同时提供 `/traj/reference` 与 PX4 内置 PID 的实际轨迹；一份 MPC 实飞包。表 43 是本章与 `tools/report/build.py` 的 `SCENES` 字典逐项对应的映射，改路径必须两处一起改。

表 43　六份话题包。

场景	控制器	包	包内首条消息（UTC）	包时长	TRACK 情况
悬停	PX4 内置 PID（offboard）	`bag/3_traj1_hover`	2026-10-08T09:39:51.222Z	116.4 s	录到 39.65 s 截断，`progress` 终值 0.661
悬停	MPC 实飞	`bag/6_free__20261017_070103`	2026-10-17T07:01:06.495Z	90.0 s	完整，60.02 s
正方形	PX4 内置 PID（offboard）	`bag/4_traj2_square`	2026-10-08T09:44:12.981Z	36.9 s	完整，8.02 s
正方形	MPC 实飞	`bag/6_free__20261017_070446`	2026-10-17T07:04:49.276Z	38.0 s	完整，8.01 s
8 字	PX4 内置 PID（offboard）	`bag/5_traj3_lemniscate`	2026-10-08T10:18:31.477Z	49.6 s	完整，12.21 s
8 字	MPC 实飞	`bag/6_free__20261017_070935`	2026-10-17T07:09:38.175Z	46.3 s	完整，12.20 s

一句话总结：3 条飞行轨迹 X 2 种控制方式。

9.2.2 三趟 nominal 仿真

仿真格不是另造一条参考，而是把实飞包的参考逐字回放。三趟都用同一个被控对象（`z0.param`，`--mismatch none`）与同一套控制器参数（`mass=0.75 kg`、`hover_thrust=0.1589`、`t_max=46.3027 N`），参考分别取自 `bag/3`、`bag/4`、`bag/5`。表 44 给出三趟的配置。

表 44　三趟 nominal SITL。

tag	参考包	参考形状	span	hold_s	产物
`R-hover`	`bag/3_traj1_hover`	悬停，参考恒为 0	39.6417 s	78	`R-hover-summary.json`、`solver-summary.json`
`R-square`	`bag/4_traj2_square`	1 m 正方形	17.8832 s	56	`R-square-summary.json`、`solver-summary.json`
`R-lemniscate`	`bag/5_traj3_lemniscate`	8 字，$a = 1.0$ m	26.2236 s	65	`R-lemniscate-summary.json`、`solver-summary.json`

三趟的 `ref_hold_s` 都是 30 s，`ref_z_offset_m` 都是 1.0 m，`hold_s` 按 `ref_hold_s + span + 8.0` 向上取整到秒（8.0 s 是验证器留的稳态窗口）。`span` 是模型参考的 `bag` 段长度；正方形与 8 字的参考在完成之后还会继续发零，所以 `span` 比 TRACK 时长长（17.88 s 对 8.02 s、26.22 s 对 12.21 s）。悬停那趟只录到 39.65 s，模型参考也按 39.64 s 回放，没有补到 60 s。

参考模型的生成命令与 sha256 在 `tools/report/evidence/sitl/README.md`。正方形与 8 字用 `mocap_model.py dump` 从对应包导出；悬停的 TRACK 窗口在包尾截断，`mocap_model.py` 的 `track_window()` 会返回零长窗口，因此改用 `build_hover_model.py`，只把窗口终点改成包尾，其余逐字复用 `build_model()`。三份模型的 sha256 已复算过，重跑逐字节一致。

已有的 Tier A（`hold` 参考）与 Tier B 的 `T0`（质量 +30% 的失配对象）不作为四源图的仿真格。它们的参考或对象与实飞不同，放进四源图会把 “失配鲁棒性” 混进 “同口径跟踪”；第 10 章只在边界一节说明这一点，不把它们的数值并进结果表，失配问题另见 `docs/control/tier-b-sitl-seam3.md`。

一句话总结：三趟 nominal 仿真复用实飞参考与同一套控制器参数，只补上缺失的仿真格。

9.3 复现命令

以下命令都从仓库根目录执行；数据管线需要 ROS 2 Humble 的 Python 环境，先 source 再跑：

```
source /opt/ros/humble/setup.bash
python3 tools/report/build.py --scene all
python3 tools/report/plot_traj.py --scene all --candidates
python3 tools/report/plot_pitch_roll.py --scene all
python3 -m pytest tools/report -q
```

`build.py` 的输入与逐字节可复现说明见 `docs/report/data/README.md`。第 10 章误差、姿态与控制量图的命令以 `docs/report/generated/figure-captions.md` 的图注为准；三趟 SITL 的模型导出与闭环参数见 `tools/report/evidence/sitl/README.md`。全量测试需要设置 `ACADOS_SOURCE_DIR`。

一句话总结：数据、图与测试都能从仓库根目录用给定命令重跑，仿真需另起 PX4 与 mavros。

9.4 本章引用清单

表 45 是本章结论的落点。路径都相对仓库根；带行号的源码位置是写作时的快照，改代码后以实际文件为准。

表 45　引用与证据。

路径	内容	用在哪
`docs/report/data/README.md`	数据字典：四源、窗口、列、原点、已知坑	全章
`docs/report/data/<scene>.json`	窗口、原点、完成原因、输入 sha256、逐源读数	9.1、9.2
`tools/report/build.py`	管线入口、`SCENES` 映射与产物	9.1–9.3
`tools/report/extract.py`	读包、原点提取、仿真 t0 反推	9.1、9.2
`tools/report/evidence/sitl/README.md`	三趟 nominal SITL 的配置、命令、产物	9.2.2、9.3
`tools/report/evidence/mpc-normalization.json`	三趟启动行原文、`t_max` 与日志哈希	9.2.2
`bag/README.md`、`bag/MANIFEST.md`	包来源、会话起点、时长、轨迹归属	9.2.1
`src/traj/config/traj.yaml`	参考频率、相对量、形状与尺度	9.2.1
`docs/control/tier-b-sitl-seam3.md` §1	B 档状态源、真值判据、失配对象	9.2.2

本章的数值快照取自 2026-10-17/18 生成的三份 `<scene>.json`。包、模型或控制器参数一变，窗口与逐源读数都要重算；不要把本章的表当成永久常量。

第 10 章 实验结果与汇总

本章给出三个场景（悬停、正方形、8 字）× 四个源（目标、offboard 实飞、MPC 仿真、MPC 实飞）的图与指标，并单列能比到哪一层、哪些事还没验证。口径见第 9 章；图与表由脚本从 `docs/report/data/` 与 `tools/report/evidence/sitl/` 现算，正文不手抄结果数，也不给总评式的过关结论。

10.1 读数口径

10.1.1 公共窗与各源完成时刻

四个源先各自回到 `/traj/start` 时间零点与轨迹原点，再切到公共窗 `[0, min(各源完成时刻)]`。表 46 给出网格点数、公共窗与各源完成时刻。

表 46　三场景的公共窗与各源完成时刻。

场景	网格点/源	公共窗 (s)	target 完成 (s)	offboard 完成 (s)	mpc_sim 完成 (s)	mpc_real 完成 (s)	右端由谁决定
悬停	3966	0 – 39.6494	39.6501 (ref_end)	39.6501 (ref_end)	39.6494 (ref_end)	60.0163 (progress)	mpc_sim = 39.649415 s
正方形	802	0 – 8.0096	8.0213 (progress)	8.0213 (progress)	17.8923 (ref_end)	8.0096 (progress)	mpc_real = 8.009599 s
8 字	1222	0 – 12.2021	12.2130 (progress)	12.2130 (progress)	26.2338 (ref_end)	12.2021 (progress)	mpc_real = 12.202111 s

表注：取自 `data/<scene>.json` 的 `grid` 与 `sources`；`progress` 为首个 `/traj/progress ≥ 1.0`，`ref_end` 为参考流末帧。仿真时间是 SITL 自己的时间线，不能与墙钟相减。悬停 offboard 包在参考流末尾被录制截断，`progress` 终值 0.661，不是控制失败。

一句话总结：四源先对齐时间零点与轨迹原点，再切到各源完成时刻的最小值；悬停窗口停在 offboard 包的录制截断处。

10.1.2 图与表的生成方式

图来自 `plot_traj.py`、`plot_attitude.py`、`plot_error.py`、`plot_control.py`，只读 `docs/report/data/*.csv`，图注读同名 `.json` 并写入 `figure-captions.md` 的分块。表由 `report_metrics.py` 现算到 `generated/metrics.md`，同一份块写进本章的 `phase5-metrics:*`；脚本逐项与 JSON 的 `metrics_vs_target` 对账，对不上就报错。重跑 `build.py --scene all` 与 `report_metrics.py`，表随数据更新。

一句话总结：图与表都由脚本从 CSV、JSON 与求解摘要现算，正文只解释方向，不手抄结果数。

10.2 轨迹图（T1–T3）

三张轨迹图都是左 XY 俯视、右 Z–t，四源叠加；目标虚线，实际源实线。XY 等比例同刻度，悬停因此缩成一个点，不是轨迹丢失。

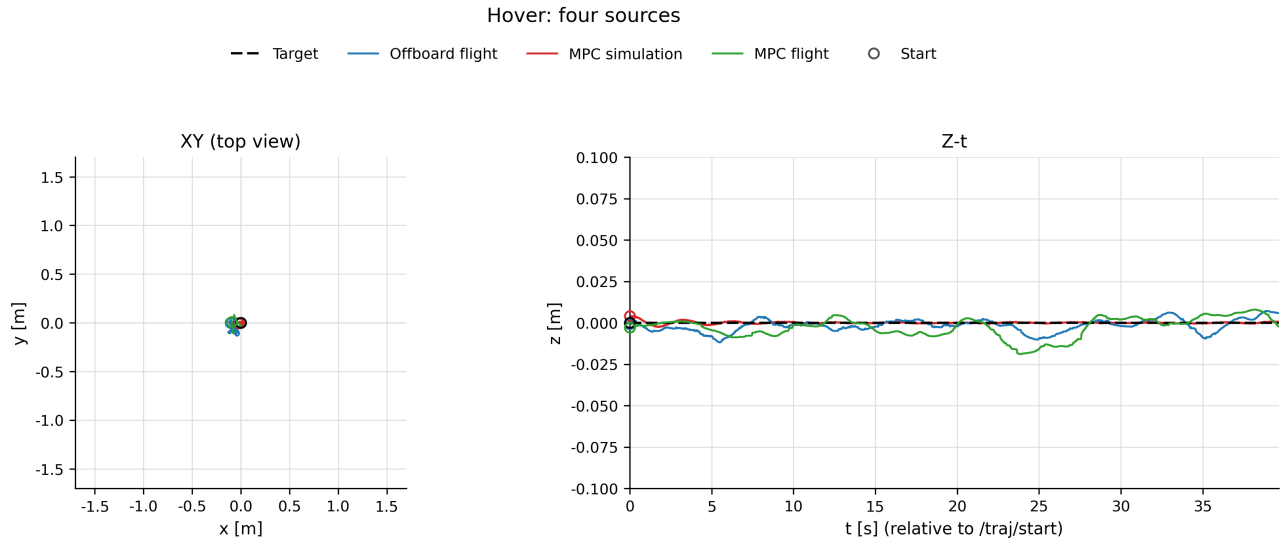


图 1　悬停（hover）轨迹：四源对比。

- 文件：`figures/fig-traj-hover.png`、`figures/fig-traj-hover.svg`（300 dpi）
- 数据：`data/hover.csv`（同名 `.json` 记输入 sha256 与窗口口径）
- 窗口：0.0000 - 39.6494 s（四源完成时刻取最小：mpc_sim = 39.649415 s）
- 来源:目标与 offboard 来自 `bag/3_traj1_hover`;MPC 仿真来自 `tools/report/evidence/sitl/R-hover/R-hover-summary.json`；MPC 实飞来自 `bag/6_free__20261017-070103`。
- 坐标：四源各自减去自身的轨迹原点（`AUTO/WAIT_START` 期间的指令值），单位为 m；时间零点为 `/traj/start` 的包记录时间戳。
- 实际轨迹：实飞两源取 `/mavros/vision_pose/pose`；仿真源取 SITL 的 `x0`（合成动捕）。offboard 与 MPC 两组控制器看到的反馈源不同，详见 `data/README.md`。
- 样式：目标为虚线，实际轨迹为实线；四源起点用同一种空心圆标记；左图 XY 等比例俯视，右图为高度-时间。
- 坐标轴统一：三张轨迹图的 XY 与 3D 候选共用同一组范围——XY 两轴 -1.7 到 1.7 m；3D 视图的 z 轴 -0.3 到 0.3 m；范围由全部数据现算。Z–t **不统一**：时间轴到本场景公共窗终点，z 按本场景数据范围。
- 生成：`python3 tools/report/plot_traj.py --scene all --candidates`
- 注意：offboard 包截止 39.65 s（录制截断，不是控制失败）。
- 裁剪口径：公共窗右端取各源最小：mpc_sim = 39.649415 s；录制截断：progress 终值 0.661 < 1.0；截止到参考流末尾 39.650080 s（包尾 39.661156 s）

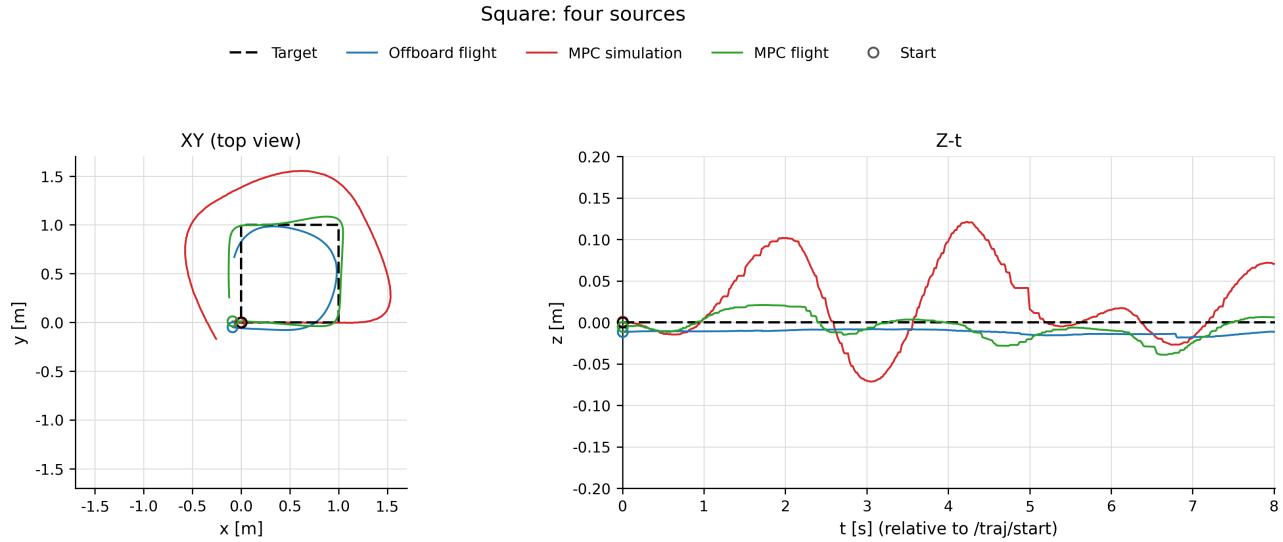


图 2　正方形（square）轨迹：四源对比。

- 文件：`figures/fig-traj-square.png`、`figures/fig-traj-square.svg`（300 dpi）
- 数据：`data/square.csv`（同名 `.json` 记输入 sha256 与窗口口径）
- 窗口：0.0000 - 8.0096 s（四源完成时刻取最小：mpc_real = 8.009599 s）
- 来源：目标与 offboard 来自 `bag/4_traj2_square`；MPC 仿真来自 `tools/report/evidence/sitl/R-square/R-square-summary.json`；MPC 实飞来自 `bag/6_free__20261017-070446`。
- 坐标：四源各自减去自身的轨迹原点（`AUTO/WAIT_START` 期间的指令值），单位为 m；时间零点为 `/traj/start` 的包记录时间戳。
- 实际轨迹：实飞两源取 `/mavros/vision_pose/pose`；仿真源取 SITL 的 `x0`（合成动捕）。offboard 与 MPC 两组控制器看到的反馈源不同，详见 `data/README.md`。
- 样式：目标为虚线，实际轨迹为实线；四源起点用同一种空心圆标记；左图 XY 等比例俯视，右图为高度-时间。
- 坐标轴统一：三张轨迹图的 XY 与 3D 候选共用同一组范围——XY 两轴 -1.7 到 1.7 m；3D 视图的 z 轴 -0.3 到 0.3 m；范围由全部数据现算。Z–t **不统一**：时间轴到本场景公共窗终点，z 按本场景数据范围。
- 生成：`python3 tools/report/plot_traj.py --scene all --candidates`

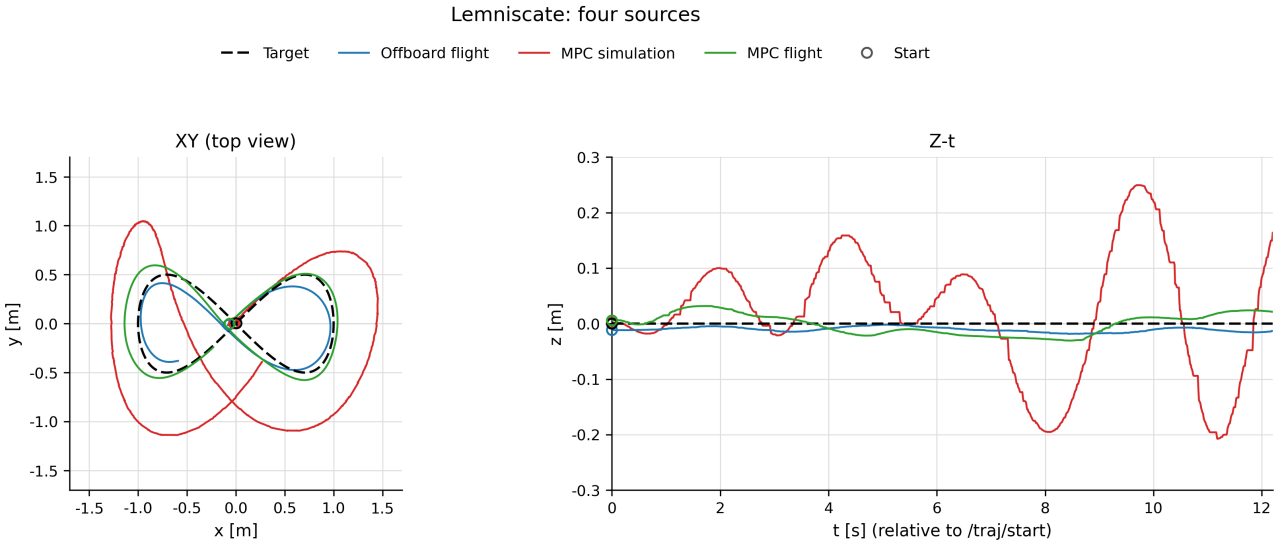


图 3　8 字（lemniscate）轨迹：四源对比。

- 文件：`figures/fig-traj-lemniscate.png`、`figures/fig-traj-lemniscate.svg`（300 dpi）
- 数据：`data/lemniscate.csv`（同名 `.json` 记输入 sha256 与窗口口径）
- 窗口：0.0000 - 12.2021 s（四源完成时刻取最小：mpc_real = 12.202111 s）
- 来源：目标与 offboard 来自 `bag/5_traj3_lemniscate`；MPC 仿真来自 `tools/report/evidence/sitl/lemniscate/R-lemniscate-summary.json`；MPC 实飞来自 `bag/6_free__20261017-070935`。
- 坐标：四源各自减去自身的轨迹原点（`AUTO/WAIT_START` 期间的指令值），单位为 m；时间零点为 `/traj/start` 的包记录时间戳。
- 实际轨迹：实飞两源取 `/mavros/vision_pose/pose`；仿真源取 SITL 的 `x0`（合成动捕）。offboard 与 MPC 两组控制器看到的反馈源不同，详见 `data/README.md`。
- 样式：目标为虚线，实际轨迹为实线；四源起点用同一种空心圆标记；左图 XY 等比例俯视，右图为高度-时间。
- 坐标轴统一：三张轨迹图的 XY 与 3D 候选共用同一组范围——XY 两轴 -1.7 到 1.7 m；3D 视图的 z 轴 -0.3 到 0.3 m；范围由全部数据现算。Z–t **不统一**：时间轴到本场景公共窗终点，z 按本场景数据范围。
- 生成：`python3 tools/report/plot_traj.py --scene all --candidates`

一句话总结：三张轨迹图里实飞两源都跟住了形状，仿真的拐角外扩与高度起伏最大；逐点差别由误差图表给出。

10.3 姿态图（A1–A3）

姿态图每张三行：roll、pitch、yaw（deg），四源叠加。目标为控制器约定的 “水平 + 纯偏航”，故三行目标线均为 0。

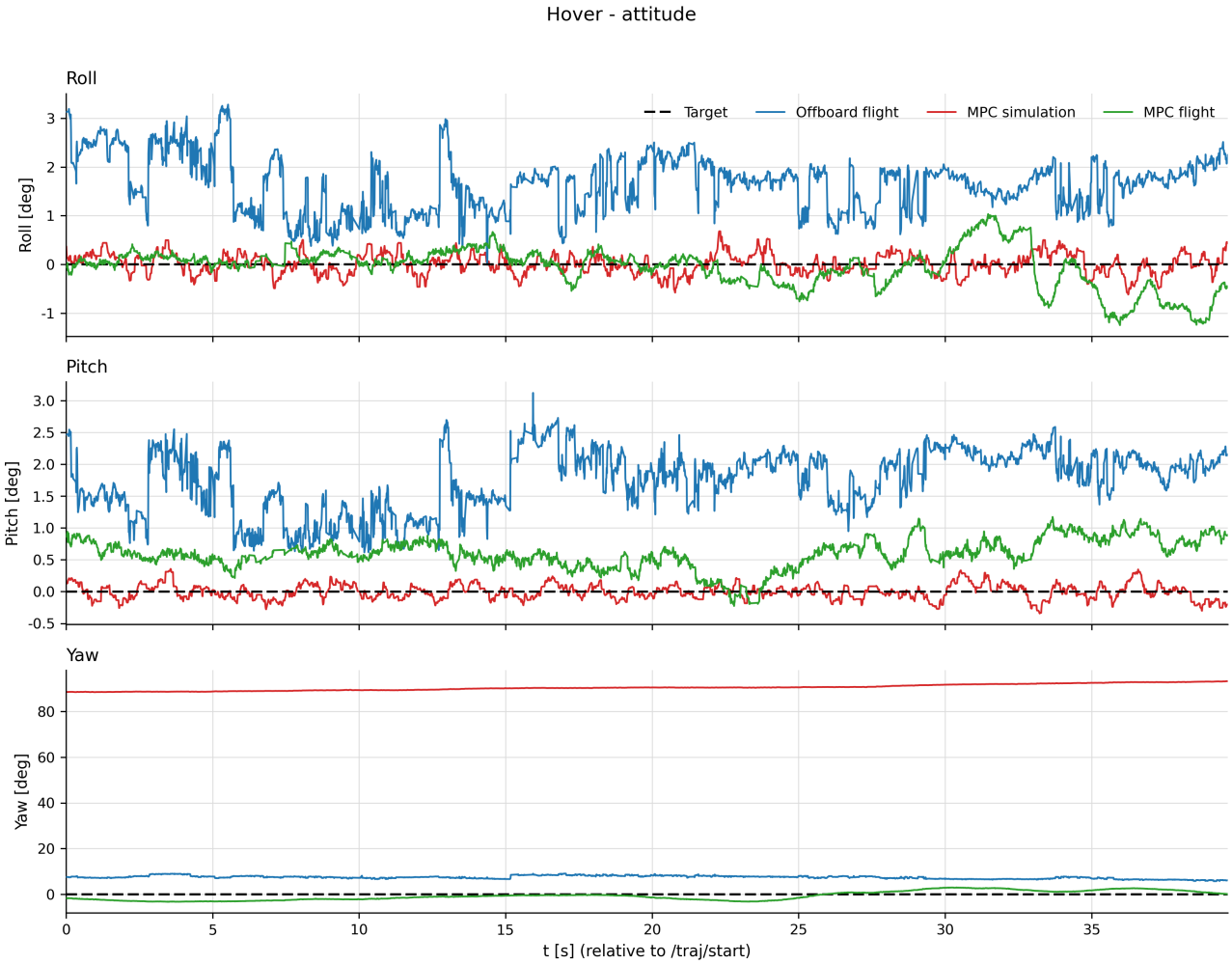


图 4　悬停（hover）姿态角：四源对比。

- 文件：`figures/fig-att-hover.png`、`figures/fig-att-hover.svg`（300 dpi）
- 数据：`data/hover.csv`（同名 `.json` 记输入 sha256 与窗口口径）
- 窗口：0.0000 - 39.6494 s（公共窗右端取各源最小：mpc_sim = 39.649415 s；录制截断：progress 终值 0.661 < 1.0；截止到参考流末尾 39.650080 s（包尾 39.661156 s））
- 坐标：时间零点为 `/traj/start` 的包记录时间戳；三行分别是 roll、pitch、yaw，单位为 deg；三行纵轴各自按该轴数据范围，不共用。
- 来源：目标来自 `/traj/reference` 的偏航（控制器约定为水平 + 纯偏航；本报告三条目标线均为 0）；offboard 来自 `bag/3_traj1_hover` 的 `/mavros/vision_pose/pose`；MPC 仿真来自 `tools/report/evidence/sitl/R-hover/R-hover-summary.json` 的 `x0` 四元数；MPC 实飞来自 `bag/6_free__20261017-070103` 的 `/mavros/vision_pose/pose`。
- 目标姿态口径：按控制器约定取「水平 + 纯偏航」，即 roll = pitch = yaw = 0；这不是用参考加速度反解出的理想倾角，理由见第 8 章。
- 姿态口径：实飞姿态取 `/mavros/vision_pose/pose`（约 190 Hz），它与 30 Hz EKF 姿态有约 **+0.03 rad** 的滚转系统偏差，不要把实际滚转均值读成跟踪误差；仿真姿态由 `x0[6:10]` 四元数解算，旧日志里名叫 roll/pitch/yaw 的字段是角速率，不能当姿态角。
- 生成：`python3 tools/report/plot_attitude.py --scene all`

- 注意：offboard 包截止 39.65 s（录制截断，不是控制失败）。

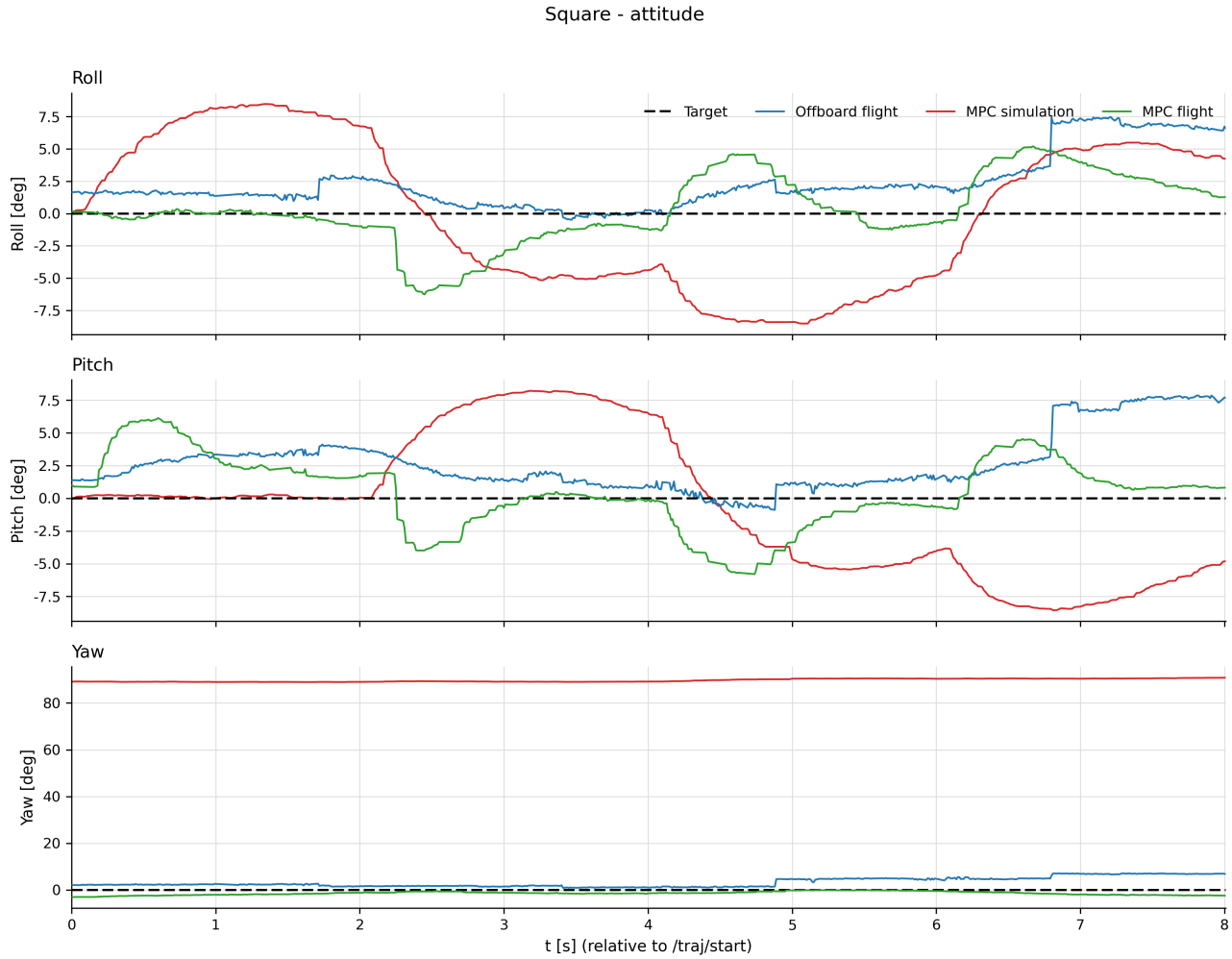


图 5　正方形（square）姿态角：四源对比。

- 文件：`figures/fig-att-square.png`、`figures/fig-att-square.svg`（300 dpi）
- 数据：`data/square.csv`（同名 `.json` 记输入 sha256 与窗口口径）
- 窗口：0.0000 - 8.0096 s（公共窗右端取各源最小：mpc_real = 8.009599 s；`/traj/progress` 首次 ⩾ 1.0：8.021292 s）
- 坐标：时间零点为 `/traj/start` 的包记录时间戳；三行分别是 roll、pitch、yaw，单位为 deg；三行纵轴各自按该轴数据范围，不共用。
- 来源：目标来自 `/traj/reference` 的偏航（控制器约定为水平 + 纯偏航；本报告三条目标线均为 0）；offboard 来自 `bag/4_traj2_square` 的 `/mavros/vision_pose/pose`；MPC 仿真来自 `tools/report/evidence/sitl/R-square/R-square-summary.json` 的 `x0` 四元数；MPC 实飞来自 `bag/6_free__20261017-070446` 的 `/mavros/vision_pose/pose`。
- 目标姿态口径：按控制器约定取「水平 + 纯偏航」，即 roll = pitch = yaw = 0；这不是用参考加速度反解出的理想倾角，理由见第 8 章。
- 姿态口径：实飞姿态取 `/mavros/vision_pose/pose`（约 190 Hz），它与 30 Hz EKF 姿态有约 **+0.03 rad** 的滚转系统偏差，不要把实际滚转均值读成跟踪误差；仿真姿态由 `x0[6:10]` 四元数解算，旧日志里名叫 roll/pitch/yaw 的字段是角速率，不能当姿态角。
- 生成：`python3 tools/report/plot_attitude.py --scene all`

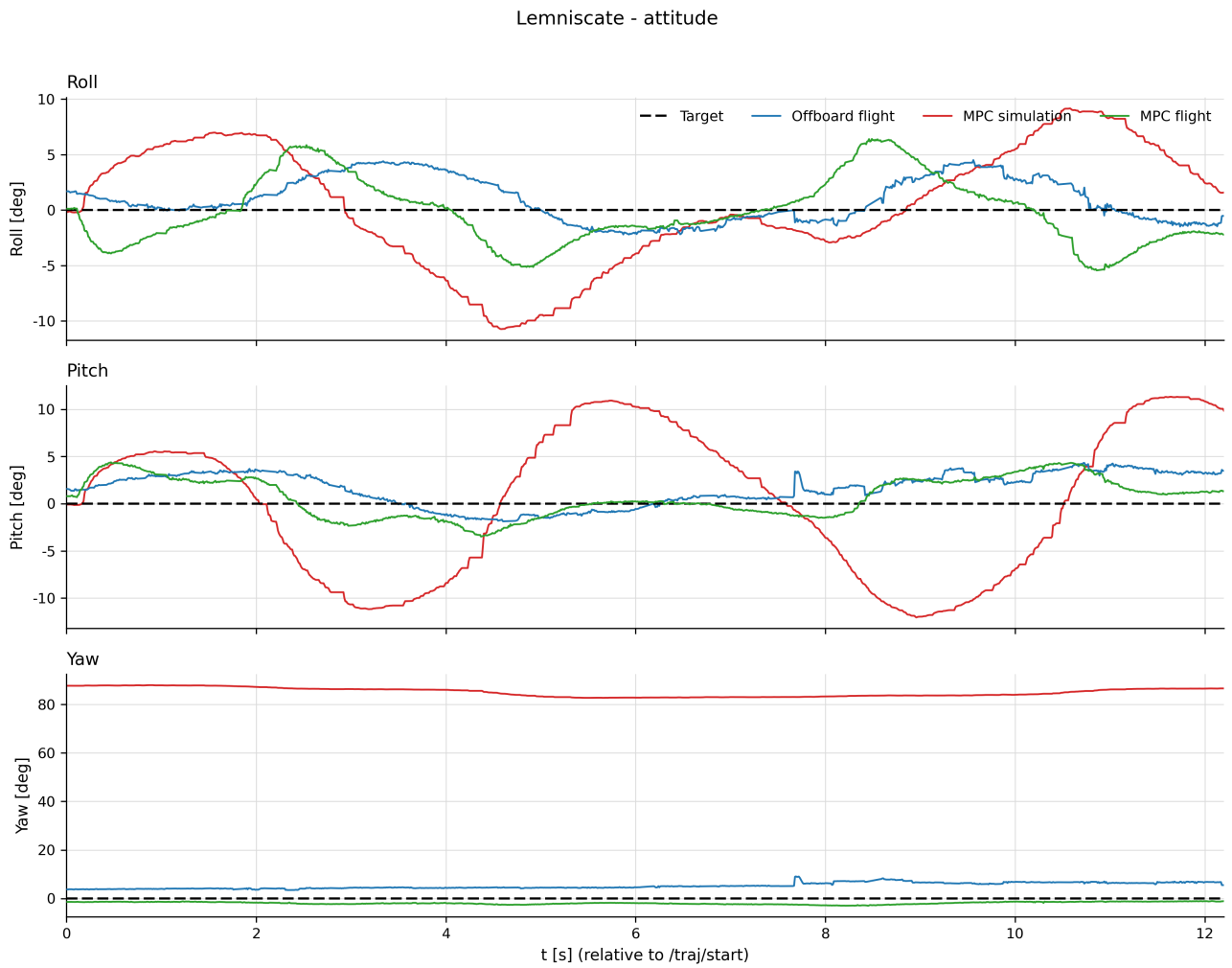


图 6　8 字（lemniscate）姿态角：四源对比。

- 文件：`figures/fig-att-lemniscate.png`、`figures/fig-att-lemniscate.svg`（300 dpi）
- 数据：`data/lemniscate.csv`（同名 `.json` 记输入 sha256 与窗口口径）
- 窗口：0.0000 - 12.2021 s（公共窗右端取各源最小：mpc_real = 12.202111 s；`/traj/progress` 首次 ⩾ 1.0：12.213041 s）
- 坐标：时间零点为 `/traj/start` 的包记录时间戳；三行分别是 roll、pitch、yaw，单位为 deg；三行纵轴各自按该轴数据范围，不共用。
- 来源：目标来自 `/traj/reference` 的偏航（控制器约定为水平 + 纯偏航；本报告三条目标线均为 0）；offboard 来自 `bag/5_traj3_lemniscate` 的 `/mavros/vision_pose/pose`；MPC 仿真来自 `tools/report/evidence/sitl/R-lemniscate/R-lemniscate-summary.json` 的 `x0` 四元数；MPC 实飞来自 `bag/6_free__20261017-070935` 的 `/mavros/vision_pose/pose`。
- 目标姿态口径：按控制器约定取「水平 + 纯偏航」，即 roll = pitch = yaw = 0；这不是用参考加速度反解出的理想倾角，理由见第 8 章。
- 姿态口径：实飞姿态取 `/mavros/vision_pose/pose`（约 190 Hz），它与 30 Hz EKF 姿态有约 **+0.03 rad** 的滚转系统偏差，不要把实际滚转均值读成跟踪误差；仿真姿态由 `x0[6:10]` 四元数解算，旧日志里名叫 roll/pitch/yaw 的字段是角速率，不能当姿态角。
- 生成：`python3 tools/report/plot_attitude.py --scene all`

实飞 roll 带约 +0.03 rad 的 VP/EKF 零点差，图与表 47 均**未扣**，判断滚转控制前应先扣掉这个零点差。仿真偏航整场在 90° 附近，是真实读数，原因尚未判定，不要当控制失败扣掉。表 47 把总夹角拆成倾角与偏航差；仿真源的总夹角几乎全由偏航差贡献。

表 47　姿态误差（相对目标：水平 + 纯偏航）。

场景	源	总夹角均值 (°)	总夹角最大 (°)	倾角均值 (°)	倾角最大 (°)	偏航差均值 (°)	偏航差最大 (°)
悬停	offboard	7.87	9.77	2.43	4.03	7.48	9.09
悬停	mpc_sim	90.51	93.27	0.21	0.69	90.51	93.26
悬停	mpc_real	2.01	3.35	0.71	1.56	1.81	3.28
正方形	offboard	5.02	12.35	3.73	10.42	3.25	7.12
正方形	mpc_sim	90.05	91.25	7.66	9.83	89.71	90.84
正方形	mpc_real	3.69	7.47	3.25	7.42	1.41	3.09
8 字	offboard	6.15	9.78	3.02	5.63	5.25	9.09
8 字	mpc_sim	85.48	87.96	8.91	12.94	85.08	87.95
8 字	mpc_real	3.97	7.18	3.34	6.68	1.90	3.02

表注：窗口为表 46 的公共窗；姿态误差取四元数最小旋转角，倾角去掉偏航，偏航差 wrap 到 $[-\pi, \pi]$。实飞为 VP，仿真为 `x0[6:10]`，测量源不同，差值不能全归给控制器。

逐源的 pitch/roll 细节另有一组 9 张图（见 `docs/report/figures/`，文件名形如 `fig-pitchroll-<scene>-<source>.png`），本章不重复展开。

一句话总结：实飞机头基本保持，倾角随机动增大；仿真的偏航差整场约九十度。

10.4 位置误差（E1–E3）

10.4.1 误差–时间图

误差 = 实际 − 目标，三个分量各一行；目标行恒为 0，只标零误差位置，不是第四条实际轨迹。

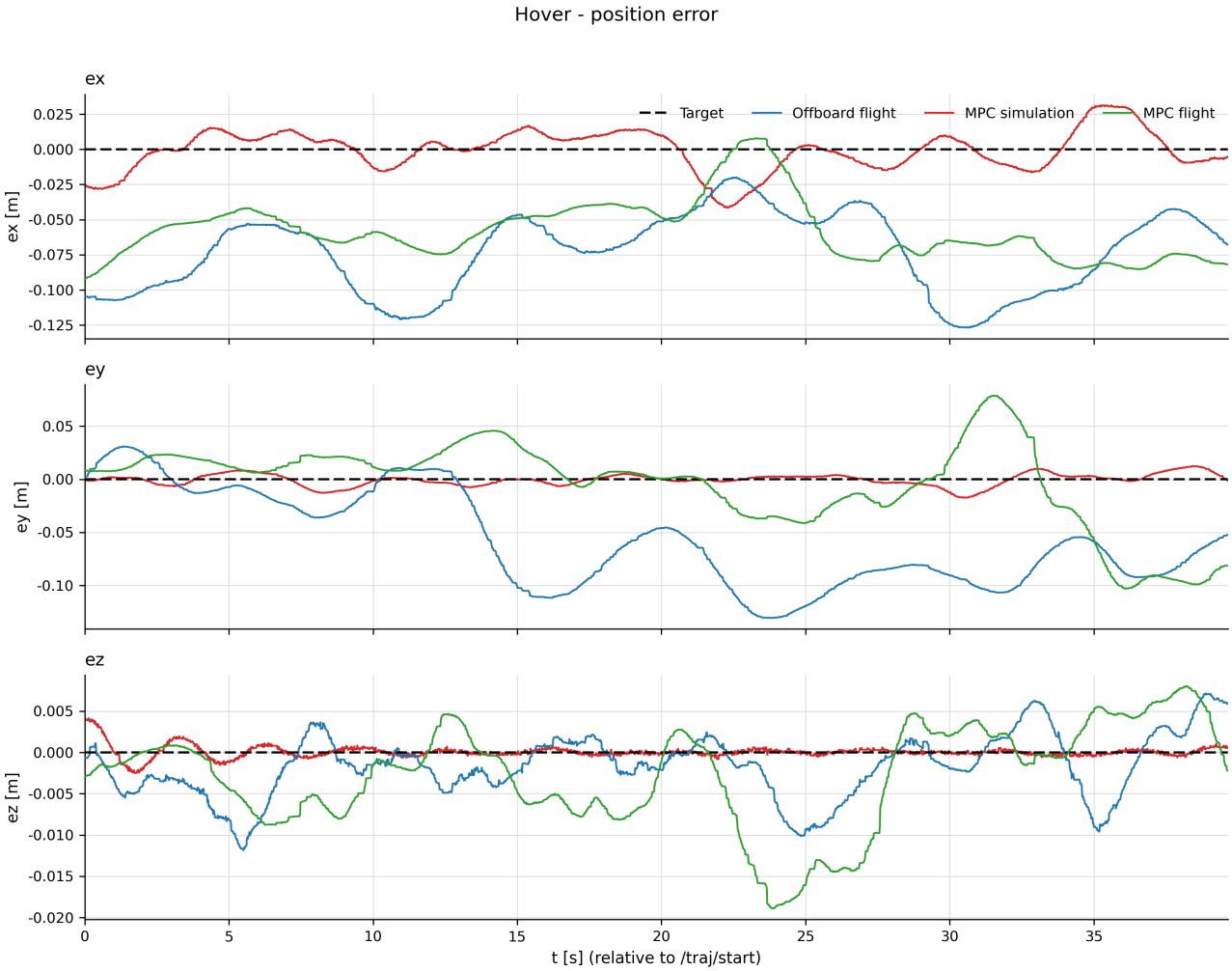


图 7　悬停（hover）位置误差：四源对比。

- 文件：`figures/fig-error-hover.png`、`figures/fig-error-hover.svg`（300 dpi）
- 数据：`data/hover.csv`（同名 `.json` 记输入 sha256 与窗口口径）
- 窗口：0.0000 - 39.6494 s（公共窗右端取各源最小：mpc_sim = 39.649415 s；录制截断：progress 终值 0.661 < 1.0；截止到参考流末尾 39.650080 s（包尾 39.661156 s））
- 误差定义：**实际 − 目标**（不是目标 − 实际）；三个分量分别为 `ex`、`ey`、`ez`，单位 m；坐标均为轨迹原点相对系。
- 来源：offboard 来自 `bag/3_traj1_hover` 的 `/mavros/vision_pose/pose`；MPC 仿真来自 `tools/report/evidence/sitl/R-hover/R-hover-summary.json` 的 `x0` 位置；MPC 实飞来自 `bag/6_free__20261017-070103` 的 `/mavros/vision_pose/pose`；目标行是 目标自身（`/traj/reference`），因此误差恒 0。
- 目标姿态与位置均来自 `/traj/reference`；参考在 TRACK 窗内才成立，跑完后参考继续发零，不能拿末帧参考当完成判据。
- 生成：`python3 tools/report/plot_error.py --scene all`
- 注意：offboard 包截止 39.65 s（录制截断，不是控制失败）。

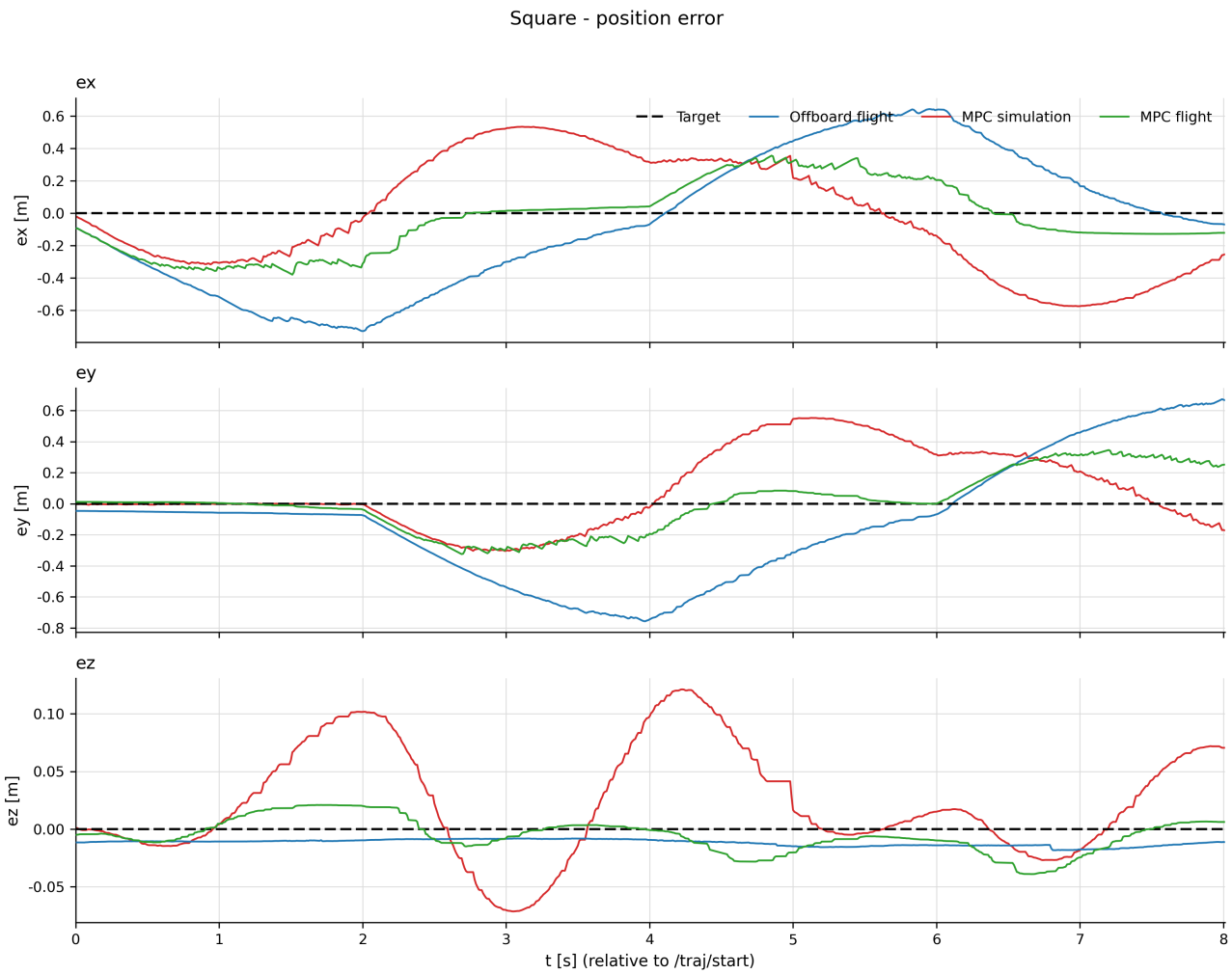


图 8　正方形（square）位置误差：四源对比。

- 文件：`figures/fig-error-square.png`、`figures/fig-error-square.svg`（300 dpi）
- 数据：`data/square.csv`（同名 `.json` 记输入 sha256 与窗口口径）
- 窗口：0.0000 - 8.0096 s（公共窗右端取各源最小：mpc_real = 8.009599 s；`/traj/progress` 首次 ⩾ 1.0：8.021292 s）
- 误差定义：**实际 − 目标**（不是目标 − 实际）；三个分量分别为 `ex`、`ey`、`ez`，单位 m；坐标均为轨迹原点相对系。
- 来源：offboard 来自 `bag/4_traj2_square` 的 `/mavros/vision_pose/pose`；MPC 仿真来自 `tools/report/evidence/sitl/R-square/R-square-summary.json` 的 `x0` 位置；MPC 实飞来自 `bag/6_free__20261017-070446` 的 `/mavros/vision_pose/pose`；目标行是 目标自身（`/traj/reference`），因此误差恒 0。
- 目标姿态与位置均来自 `/traj/reference`；参考在 TRACK 窗内才成立，跑完后参考继续发零，不能拿末帧参考当完成判据。
- 生成：`python3 tools/report/plot_error.py --scene all`

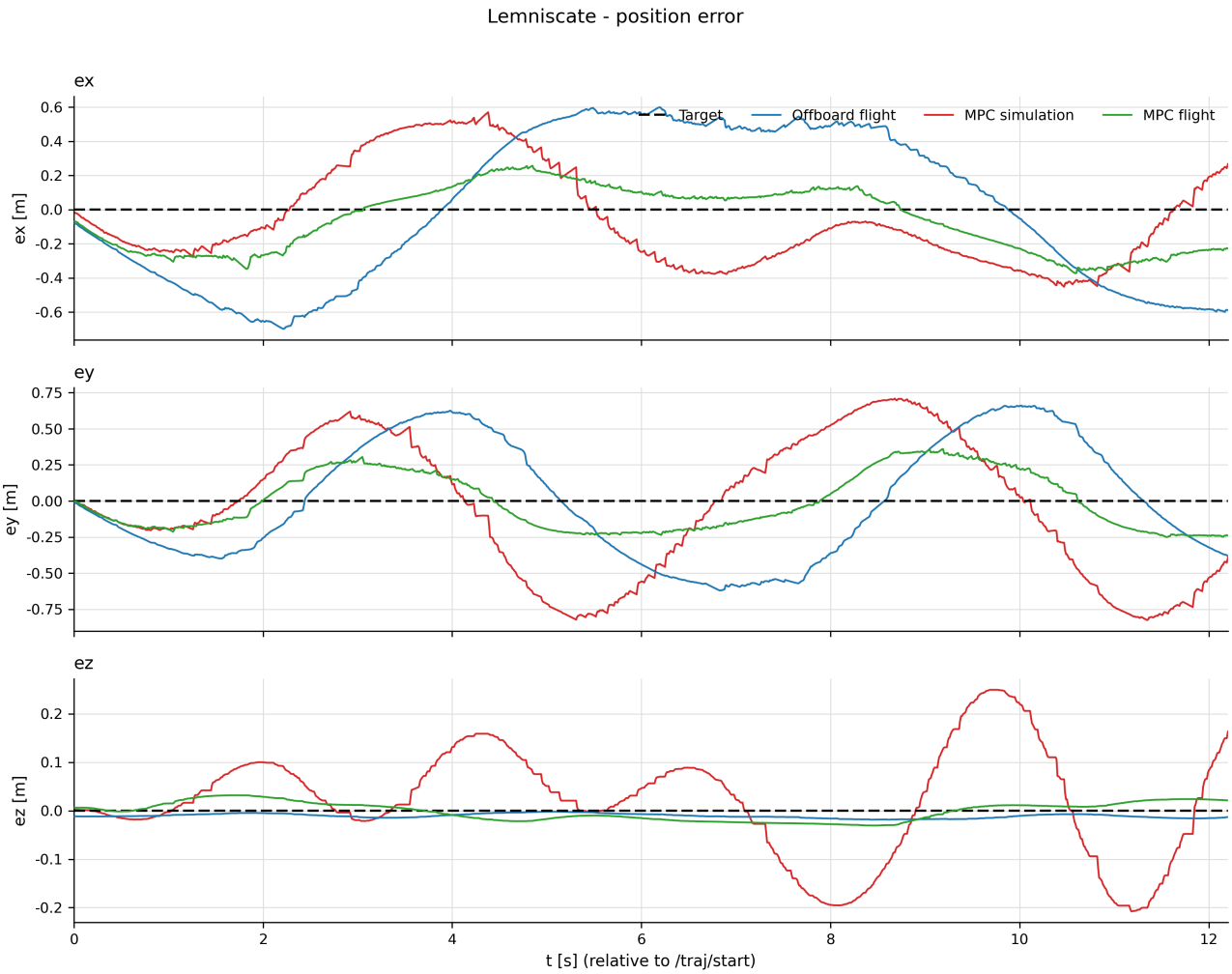


图 9　8 字（lemniscate）位置误差：四源对比。

- 文件：`figures/fig-error-lemniscate.png`、`figures/fig-error-lemniscate.svg`（300 dpi）
- 数据：`data/lemniscate.csv`（同名 `.json` 记输入 sha256 与窗口口径）
- 窗口：0.0000 - 12.2021 s（公共窗右端取各源最小：mpc_real = 12.202111 s；`/traj/progress` 首次 ⩾ 1.0：12.213041 s）
- 误差定义：**实际 − 目标**（不是目标 − 实际）；三个分量分别为 ex、ey、ez，单位 m；坐标均为轨迹原点相对系。
- 来源：offboard 来自 `bag/5_traj3_lemniscate` 的 `/mavros/vision_pose/pose`；MPC 仿真来自 `tools/report/evidence/sitl/R-lemniscate/R-lemniscate-summary.json` 的 `x0` 位置；MPC 实飞来自 `bag/6_free__20261017-070935` 的 `/mavros/vision_pose/pose`；目标行是 目标自身（`/traj/reference`），因此误差恒 0。
- 目标姿态与位置均来自 `/traj/reference`；参考在 TRACK 窗内才成立，跑完后参考继续发零，不能拿末帧参考当完成判据。
- 生成：`python3 tools/report/plot_error.py --scene all`

悬停误差在厘米级；正方形与 8 字大一个量级，运动场景的逐点误差主要是**时间上的跟随滞后**，不是空间形状走错。

一句话总结：悬停误差在厘米级；正方形与 8 字的误差主要是匀速段的时间滞后，拐角处出现尖峰。

10.4.2 位置指标与悬停稳态

表 48 给出逐轴 RMSE、水平均值/最大值、垂直绝对值均值/最大值；逐轴 RMSE 与水平均值不可互换。

表 48　位置误差指标（实际 − 目标，公共窗）。

场景	源	RMSE ex (m)	RMSE ey (m)	RMSE ez (m)	水平均值 (m)	水平最大 (m)	垂直均值 (m)	垂直最大 (m)
悬停	offboard	0.0791	0.0730	0.0042	0.1047	0.1610	0.0032	0.0119
悬停	mpc_sim	0.0144	0.0056	0.0007	0.0132	0.0415	0.0004	0.0042
悬停	mpc_real	0.0618	0.0407	0.0067	0.0694	0.1333	0.0051	0.0189
正方形	offboard	0.4214	0.4069	0.0121	0.5720	0.7608	0.0117	0.0183
正方形	mpc_sim	0.3498	0.2646	0.0534	0.4073	0.6253	0.0403	0.1212
正方形	mpc_real	0.2133	0.1833	0.0156	0.2753	0.3802	0.0124	0.0390
8 字	offboard	0.4485	0.4080	0.0115	0.5935	0.8241	0.0107	0.0185
8 字	mpc_sim	0.2918	0.4724	0.1112	0.5204	0.8650	0.0868	0.2495
8 字	mpc_real	0.1962	0.1977	0.0188	0.2691	0.3950	0.0167	0.0317

表注：窗口为表 46 的公共窗；单位 m；所有源在同一张 100 Hz 网格、各自轨迹原点相对系上逐点相减。完成时刻见表 46。

表 49 只列悬停公共窗最后 10 s（29.65–39.65 s）的稳态指标。只有悬停的目标是定点，运动场景末 10 s 不构成稳态；悬停包被截断，这 10 s 不是 60 s 悬停的末段。

表 49　悬停末 10 s 稳态误差。

源	水平均值 (m)	水平最大 (m)	垂直绝对值均值 (m)	网格点
offboard	0.1194	0.1610	0.0037	1001
mpc_sim	0.0160	0.0313	0.0003	1001
mpc_real	0.1024	0.1333	0.0035	1001

表注：窗口为公共窗最后 10 s；水平取 xy 范数，垂直取绝对值；单位 m。

一句话总结：逐轴 RMSE 与水平均值回答不同问题；悬停末十秒的稳态误差单独列表，且不是六十秒悬停的末段。

10.5 控制量（C1–C3）

控制量图四行:推力与 wx、wy、wz,只画 MPC 仿真与实飞。offboard 发的是 /mavros/setpoint_raw/local 位置设定点，与推力/角速率不是同一层，混画会造成 “看起来可比” 的假象；两组只在位置跟踪上可比（10.6.1）。推力 N 值 = 归一化推力 × t_max，逐趟读取；CSV 按 100 Hz 零阶保持取样。

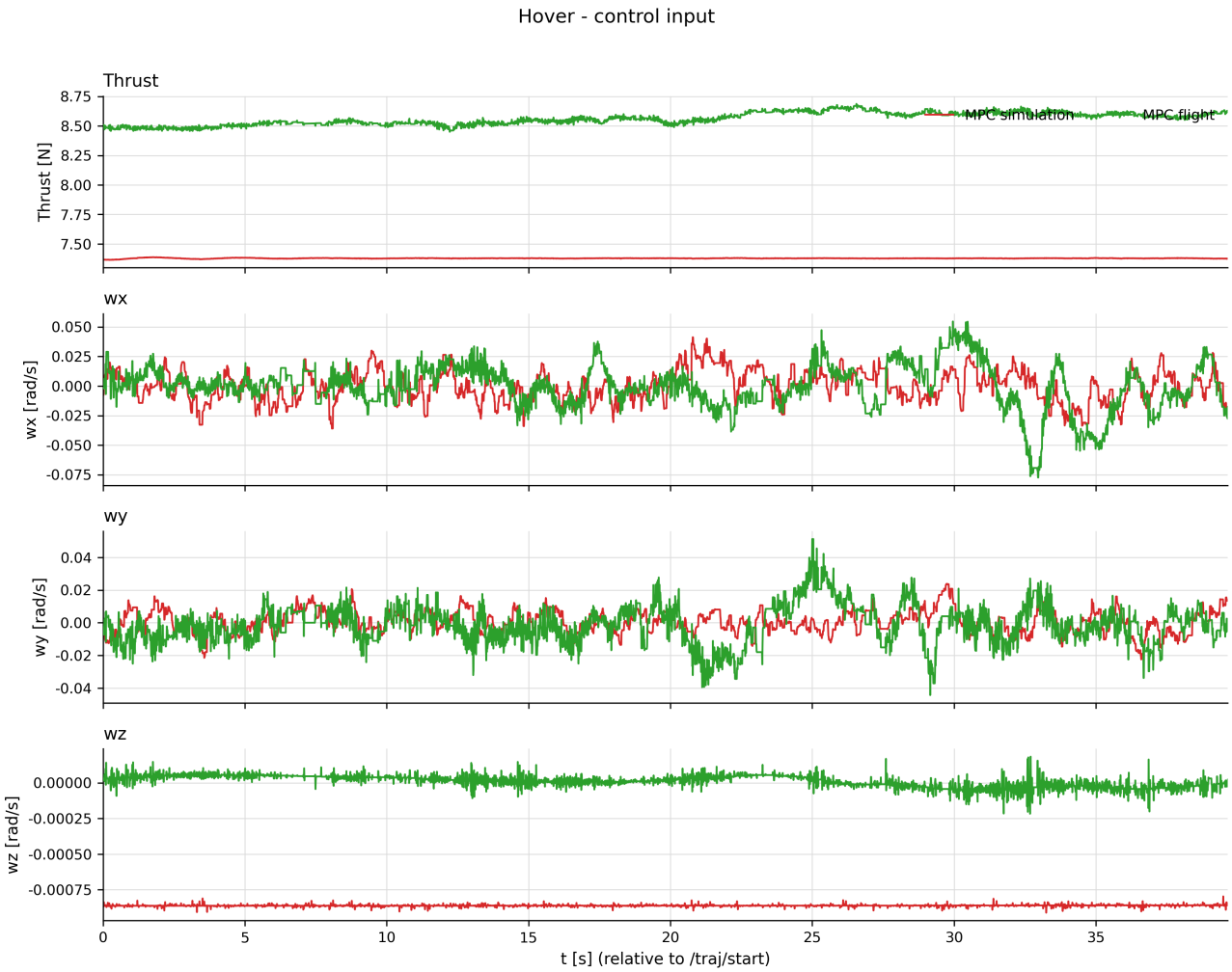


图 10　悬停（hover）控制量：MPC 仿真 vs 实飞。

- 文件：`figures/fig-control-hover.png`、`figures/fig-control-hover.svg`（300 dpi）
- 数据：`data/hover.csv`（同名 `.json` 记输入 sha256 与窗口口径）
- 窗口：0.0000 - 39.6494 s（公共窗右端取各源最小：mpc_sim = 39.649415 s；录制截断：progress 终值 0.661 < 1.0；截止到参考流末尾 39.650080 s（包尾 39.661156 s））
- 四行：总推力 `F [N]`、机体角速率 `wx`、`wy`、`wz [rad/s]`（FLU，机体系指令）；横轴 `t [s]` 相对 `/traj/start`。
- 只有两源：MPC 仿真（`tools/report/evidence/sitl/R-hover/R-hover-summary.json` 的求解指令（SITL 控制器参数））与 MPC 实飞（`bag/6_free__20261017-070103` 的 `/mavros/setpoint_raw/attitude`（`body_rate` + `thrust`））。
- 为什么没有 offboard：offboard 的控制量是 `/mavros/setpoint_raw/local` 的**位置设定点**，与“推力 + 角速率”不是同一层；画在一起会造成“看起来可比”的假象。两组只在位置跟踪这一层可比（见误差图）。
- 推力换算：`u_thrust_n = thrust × t_max`，`t_max = 46.3027 N`；仿真取 SITL 控制器参数，实飞取该趟 `mpc_node` 启动行（`tools/report/evidence/mpc-normalization.json`）。CSV 里的控制量按 100 Hz 网格**零阶保持**取样。
- 生成：`python3 tools/report/plot_control.py --scene all`
- 注意：offboard 包截止 39.65 s（录制截断，不是控制失败）。

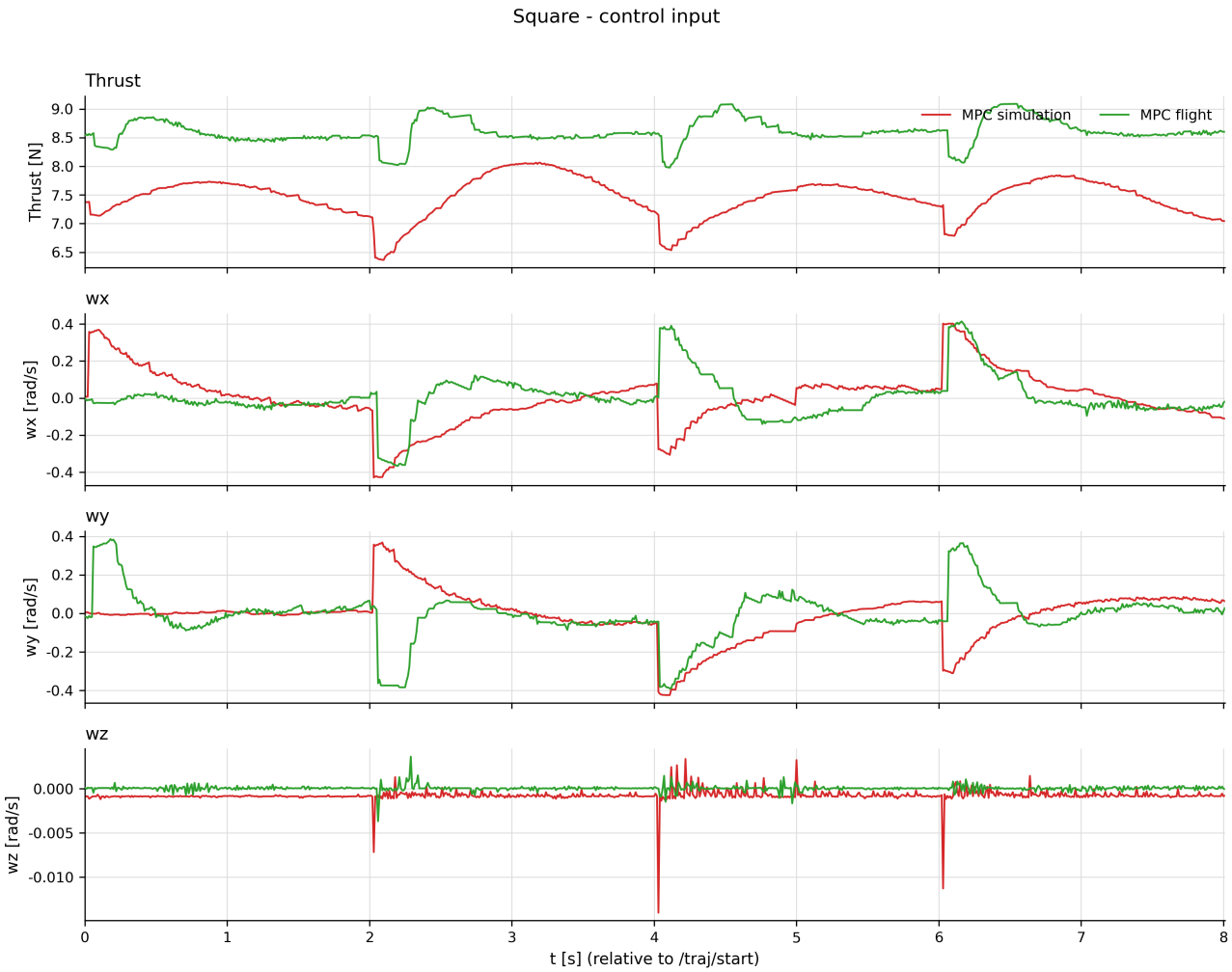


图 11　正方形（square）控制量：MPC 仿真 vs 实飞。

- 文件：`figures/fig-control-square.png`、`figures/fig-control-square.svg`（300 dpi）
- 数据：`data/square.csv`（同名 `.json` 记输入 sha256 与窗口口径）
- 窗口：0.0000 - 8.0096 s（公共窗右端取各源最小：mpc_real = 8.009599 s；`/traj/progress` 首次 ⩾ 1.0：8.021292 s）
- 四行：总推力 `F [N]`、机体角速率 `wx`、`wy`、`wz [rad/s]`（FLU，机体系指令）；横轴 `t [s]` 相对 `/traj/start`。
- 只有两源：MPC 仿真（`tools/report/evidence/sitl/R-square/R-square-summary.json` 的求解指令（SITL 控制器参数））与 MPC 实飞（`bag/6_free__20261017-070446` 的 `/mavros/setpoint_raw/attitude`（`body_rate` + `thrust`））。
- 为什么没有 offboard：offboard 的控制量是 `/mavros/setpoint_raw/local` 的**位置设定点**，与“推力 + 角速率”不是同一层；画在一起会造成“看起来可比”的假象。两组只在位置跟踪这一层可比（见误差图）。
- 推力换算：`u_thrust_n = thrust × t_max`，`t_max = 46.3027 N`；仿真取 SITL 控制器参数，实飞取该趟 `mpc_node` 启动行（`tools/report/evidence/mpc-normalization.json`）。CSV 里的控制量按 100 Hz 网格**零阶保持**取样。
- 生成：`python3 tools/report/plot_control.py --scene all`

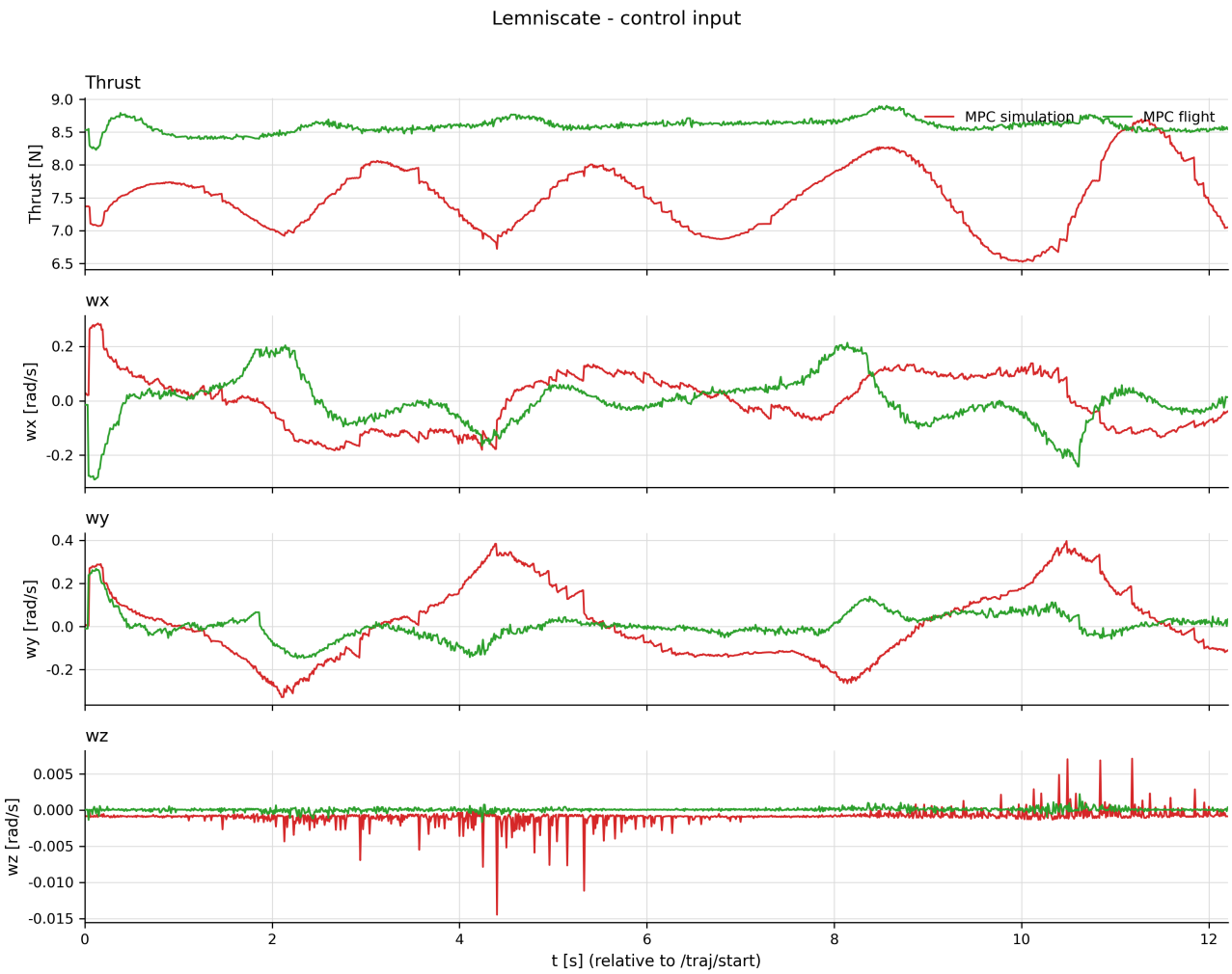


图 12　8 字（lemniscate）控制量：MPC 仿真 vs 实飞。

- 文件：`figures/fig-control-lemniscate.png`、`figures/fig-control-lemniscate.svg`（300 dpi）
- 数据：`data/lemniscate.csv`（同名 `.json` 记输入 sha256 与窗口口径）
- 窗口：0.0000 - 12.2021 s（公共窗右端取各源最小：mpc_real = 12.202111 s；`/traj/progress` 首次 ⩾ 1.0：12.213041 s）
- 四行：总推力 `F [N]`、机体角速率 `wx`、`wy`、`wz [rad/s]`（FLU，机体系指令）；横轴 `t [s]` 相对 `/traj/start`。
- 只有两源：MPC 仿真（`tools/report/evidence/sitl/R-lemniscate/R-lemniscate-summary.json` 的求解指令（SITL 控制器参数））与 MPC 实飞（`bag/6_free__20261017-070935` 的 `/mavros/setpoint_raw/attitude`（`body_rate` + `thrust`））。
- 为什么没有 offboard：offboard 的控制量是 `/mavros/setpoint_raw/local` 的**位置设定点**，与“推力 + 角速率”不是同一层；画在一起会造成“看起来可比”的假象。两组只在位置跟踪这一层可比（见误差图）。
- 推力换算：`u_thrust_n = thrust × t_max`，`t_max = 46.3027 N`；仿真取 SITL 控制器参数，实飞取该趟 `mpc_node` 启动行（`tools/report/evidence/mpc-normalization.json`）。CSV 里的控制量按 100 Hz 网格**零阶保持**取样。
- 生成：`python3 tools/report/plot_control.py --scene all`

悬停推力近乎直线，角速率很小；运动场景在拐角处出现台阶或凹陷，`wx`、`wy` 换向时到每秒零点几弧度，`wz` 全程很小。实飞推力均值高于仿真，数值见表 50。

表 50　控制量统计（仅 MPC 两源）。

场景	源	推力均值 (N)	推力最小 (N)	推力最大 (N)	角速率模长均值 (rad/s)	角速率模长最大 (rad/s)	|wx|max (rad/s)	|wy|max (rad/s)	|wz|max (rad/s)	饱和拍数
悬停	mpc_sim	7.378	7.364	7.388	0.0132	0.0416	0.0413	0.0238	0.0009	0
悬停	mpc_real	8.561	8.451	8.687	0.0185	0.0811	0.0774	0.0513	0.0002	0
正方形	mpc_sim	7.451	6.366	8.063	0.1357	0.5639	0.4292	0.4245	0.0140	0
正方形	mpc_real	8.601	7.977	9.089	0.1151	0.5511	0.4137	0.3910	0.0037	0
8 字	mpc_sim	7.513	6.523	8.693	0.1763	0.4227	0.2847	0.3964	0.0145	0
8 字	mpc_real	8.598	8.229	8.901	0.0821	0.3914	0.2895	0.2679	0.0022	0

表注：窗口为表 46 的公共窗；推力单位 N，角速率单位 rad/s（FLU，机体系）。饱和拍数按生产判据：推力界 `[0, f_max]`，roll/pitch 3.84、yaw 6.28 rad/s，相对容差 1e-3。仿真的饱和拍数取自 `solver-summary.json` 的 `saturation.frames_with_sat`；实飞在 100 Hz 网格上逐拍套同一判据。两种计数都以“拍”为单位，不是秒。

表 50 的饱和拍数全为 0：三场里没有一拍被推到约束边界，也不表示输出被后处理裁剪；节点不做裁剪。

一句话总结：控制量图只比较 MPC 的仿真与实飞，三场推力与角速率都没有顶到约束边界。

10.6 三组对比

10.6.1 offboard 与 MPC（位置层）

这一组只比位置跟踪。offboard 反馈 EKF 位置（约 30 Hz）、发位置设定点；MPC 实飞反馈原始动捕 `/mocap/pose_enu`、发推力与角速率。比较的是跟踪结果，不是控制器输入，也不能据此判断哪个控制器更强；差异同时含算法、反馈源与估计滞后，本批数据不能拆开。

表 48 的方向清楚：三场景里 offboard 的逐轴 RMSE、水平均值与最大值都高于 MPC 实飞；垂直方向相反。即 MPC 实飞水平更贴目标，offboard 高度更稳；这只是本批数据的方向，不带阈值。悬停末 10 s 稳态同向；正方形与 8 字末段仍在运动，没有稳态窗。

一句话总结：同一批包上，MPC 实飞的水平跟踪优于 offboard，高度误差更大；两组反馈源不同。

10.6.2 MPC 仿真与实飞

这一组同参考、同公共窗、同网格，最接近“只换被控对象与反馈源”。仍有两处不同：仿真状态来自 SITL 合成动捕 `x0`，实飞来自真实动捕经 VP；仿真时间是 SITL 自己的时间线。

读数与“仿真应更准”的直觉不一致：悬停时仿真误差更小；正方形与 8 字时仿真的水平与垂直误差反而更大，偏航差整场约 90°。不能据此说“实飞控制器优于仿真”：被控对象、状态源与时间对齐都不同，误差里混着模型失配、反馈源差异与时间滞后。本章只报读数，不判定原因；仿真的拐角外扩与 10.2 的轨迹图一致，偏航 90° 的未判定原因见10.3。

一句话总结：悬停时仿真更准，运动场景时仿真的水平与垂直误差反而更大；本章只报读数，不判定原因。

10.6.3 求解耗时与退化

表 51 给出三趟标称 SITL 的稳态求解耗时、退化与饱和拍数；稳态统计去掉 `fallback == 'warmup'` 的拍。均值与 p95 回答常态，最坏回答单拍尖峰。

表 51　标称 SITL 求解耗时与退化。

场景	稳态求解拍数	均值 (ms)	p95 (ms)	最坏 (ms)	退化拍数	饱和拍数
悬停	7792	3.126	3.624	20.138	0	0
正方形	5674	2.983	3.312	6.462	0	0
8 字	6567	3.132	3.612	10.738	0	0

表注：每场的数据来自 `tools/report/evidence/sitl/<tag>/solver-summary.json`；稳态求解拍数 是去掉 warmup 后的拍数；p95 用最近秩；退化拍数来自原始 DUT JSONL 的 `degraded` 字段，饱和拍数来自 `sat` 字段。耗时是 SITL 宿主机的读数，不是飞控上的读数。

三场均值与 p95 都在数毫秒，退化与饱和拍数为 0。最坏值要带限定读：正方形已在空闲时重跑；悬停与 8 字保留首轮读数，首轮与本机全量测试同时跑，可能被负载抬高。退化判的是解的质量与约束越界，不是耗时，“退化 0” 不等于每拍都在 10 ms 内解完。

一句话总结：三场求解均值与 p95 都在数毫秒，退化与饱和均为零；悬停与 8 字的最坏值可能受本机负载影响。

10.7 边界与未验证项

10.7.1 五张 `wontfix` 票的缺口

五张票由操作者裁定收口为 `wontfix`——是 “裁定收、不做了”，不是 “做完了”；下列缺口没有证据作锚，本章不写成已解决。

- **phase4/01 台架几何与外参**：动捕刚体到 IMU 的 xy 外参未测（节点落 0.0），“动捕 z = 0 平面即地面” 未证，`EKF2_EV_POS_*` / `EKF2_IMU_POS_*` 未上机读回，倾斜段实飞判据未跑。
- **phase4/04 固件重编重刷**：按操作者裁定不重刷，沿用现行固件与参数；无重编、重刷，也无刷前/刷后参数 diff。这是决定不做，不是验证通过。
- **phase4/09 阶梯 1+2**：阶梯 2 判据 ② 判不了、④ 水平红（最后撞地）；阶梯 1 第 4 框只落一半；同日操作者决定跳过阶梯 2 直接做闭环。
- **phase4/14 起链前控制器选择**：被 phase4/15 取代后收口；菜单 9 的 MPC 档改为 `traj → mpc → control`，`control` 成为唯一任务总管。
- **phase4/16 阶梯 3/4 收尾缺口**：τ 实飞 A/B 未做，冻结值未上实飞；回退能力未触发、未取证；两级共同日志项缺失（无 `/mpc/*`、无逐拍求解耗时/内部状态/饱和标志）。

一句话总结：五张 wontfix 票的缺口都是 “裁定收、不做了”，没有任何一条被本章或本轮数据补上。

10.7.2 本轮读数的边界

以下限制读表时要一并带上。

- **悬停 offboard 包录制截断**：只录到参考流末尾，`progress` 终值 0.661；所有指标只到公共窗右端，末段未进图或表。
- **仿真完成时刻不是墙钟**：仿真按参考 `span` 回放，完成时刻只在 SITL 时间线上成立；表 46 的仿真列不能与实飞列相减。
- **仿真偏航约 90°**：从首帧起就在 90° 附近、整场未转到 0；是数据读数，原因尚未判定，不要扣掉后再比较姿态。
- **VP 与 EKF 的 +0.03 rad 滚转零偏**：实飞姿态取 VP，与 30 Hz EKF 有约 +0.03 rad 系统差；实飞 roll 均值不在 0 主要是这个零点差，不是跟踪误差。
- **反馈源不同**：offboard 用 EKF 位置，MPC 实飞用原始动捕；比较的是跟踪结果，不是控制器输入。
- **求解最坏值可能受负载影响**：悬停与 8 字保留首轮读数，首轮与本机全量测试同时跑；正方形已在空闲时重跑。均值与 p95 影响较小，最坏值按此前提读。

- **起飞段欠阻尼**：三趟标称 SITL 都在离地后冲到目标高度以上再回落，约十几秒收敛；公共窗包含这段瞬态，悬停稳态不受影响，运动场景早期误差会受影响。
- **正方形此前从未在 SITL 跑过**：`R-square` 是本轮补上的第一趟，历史上 B 档判据曾把它删掉；本章读数只代表这一趟 nominal。

一句话总结：悬停被截断、仿真时间线独立、偏航九十度与滚转零点差都限制读法；最坏求解耗时可能含本机负载。

10.7.3 未验证项与不作外推的部分

- **τ 的实飞 A/B 未做**：四趟实飞都用节点默认 τ，冻结值未上实飞；“换 τ 更好或更坏” 在本轮没有证据。
- **回退能力未在实飞中取证**：四趟实飞都未触发回退，phase4/08 的演练不能当本轮证据。
- **Tier A 与 Tier B 的 T0 失配证据不进入四源图**：参考或被控对象与实飞不同，放进来会把 “失配鲁棒性” 混进 “同口径跟踪”。Tier A 的 ±30% 扫描读数出自被控对象重搭之前，`docs/control/mismatch-sweep.md` 页首已整页标作废；本章不引用。B 档矩阵见 `docs/control/tier-b-sitl-seam3.md`，回答的是另一个问题。
- **不把 “没有异常” 写成 “已通过”**：饱和与退化计数为 0 只说明这三趟 nominal 没触发，不覆盖失配、外扰、电池变化或更长飞行。
- **不做跨层级比较**：offboard 与 MPC 的控制量不可比；本章没有把位置设定点换算成推力或角速率，也没有反过来做。

一句话总结：τ 的 A/B、实飞回退与失配鲁棒性都未取证；三场 nominal 的 “零饱和、零退化” 不能外推。

10.8 小结

三个场景 × 四个源的图与表已齐。位置层 MPC 实飞水平更优、offboard 高度更稳；姿态层实飞机头基本保持，仿真偏航约 90°；控制量只有 MPC 两源可比，三场无饱和、无退化。仿真在运动场景的误差反而更大，本章只报读数、不判因。

边界：悬停包录制截断、仿真时间线独立、VP/EKF 有 +0.03 rad 零点差、反馈源不同、最坏求解值可能受负载影响；τ 的实飞 A/B、回退能力、失配鲁棒性均未取证，五张 `wontfix` 的缺口保持原样。没有统一阈值的地方，只给读数与未验证项。

一句话总结：四源三场景的图与指标已齐，水平跟踪与高度跟踪各有胜负；所有未取证项都单列，不作通过结论。